

Introdução a *Machine Learning* e Redes Neurais

Autoria:

Rafael Divino Ferreira Feitosa
Danilo Turkievicz dos Santos
Rafael Teixeira Sousa

Organizadores:

Taciana Novo Kudo
Renata Dutra Braga
Deborah Silva Alves Fernandes
Cristiane Bastos Rocha Ferreira
Arlindo Rodrigues Galvão Filho



Universidade Federal de Goiás

Reitora

Angelita Pereira de Lima

Vice-Reitor

Jesiel Freitas Carvalho

Diretora do Cegraf UFG

Maria Lucia Kons

Conselho Editorial da Coleção Formação no AKCIT

Anderson da Silva Soares

Arlindo Rodrigues Galvão Filho

Deborah Silva Alves Fernandes

Juliana Pereira de Souza Zinader

Renata Dutra Braga

Taciana Novo Kudo

Telma Woerle de Lima Soares

Equipe de produção:

Amanda Souza Vitor

Ana Laura de Sene Amâncio Zara Brisolla

Ana Luísa Silva Gonçalves

Caio Barbosa Dias

Daiane Souza Vitor

Dandra Alves de Souza

Davi Oliveira Gomes

Guilherme Correia Dutra

Iuri Vaz Miranda

Layane Grazielle Souza Dias

Luciana Dantas Soares Alves

Luis Felipe Ferreira Silva

Luiza de Oliveira Costa

Luma Wanderley de Oliveira

Suse Barbosa Castilho

Wanderley de Souza Alencar

Introdução a *Machine Learning* e Redes Neurais

Autoria:

Rafael Divino Ferreira Feitosa
Danilo Turkievicz dos Santos
Rafael Teixeira Sousa

Organizadores:

Taciana Novo Kudo
Renata Dutra Braga
Deborah Silva Alves Fernandes
Cristiane Bastos Rocha Ferreira
Arlindo Rodrigues Galvão Filho

Cegraf UFG
2024

© Cegraf UFG, 2024

© Taciana Novo Kudo

Renata Dutra Braga

Deborah Silva Alves Fernandes

Cristiane Bastos Rocha Ferreira

Arlindo Rodrigues Galvão Filho

© Universidade Federal de Goiás, 2024

© AKCIT, 2024

Revisão Técnica

Deborah Silva Alves Fernandes

Revisão Editorial

Ana Laura de Sene Amâncio Zara Brisolla

Capa

Iuri Vaz Miranda

Editoração Eletrônica

Luma Wanderley de Oliveira

Layane Grazielle Souza Dias



Esta obra é disponibilizada nos termos da Licença Creative Commons – Atribuição – Não Comercial – Compartilhamento pela mesma licença 4.0 Internacional. É permitida a reprodução parcial ou total desta obra, desde que citada a fonte.

<https://doi.org/10.5216/FEI.int.ebook.978-85-495-1018-1/2024>

Dados Internacionais de Catalogação na Publicação (CIP) (Câmara Brasileira do Livro, SP, Brasil)

Feitosa, Rafael Divino Ferreira
Introdução a Machine Learning e redes neurais
[livro eletrônico] / Rafael Divino Ferreira Feitosa,
Danilo Turkievicz de Souza, Rafael Teixeira Souza. --
Goiânia, GO : Cegraf UFG, 2024.

PDF

Bibliografia.

ISBN 978-85-495-1018-1

1. Ciência da computação 2. Inteligência artificial 3. Redes neurais (Ciência da computação)
I. Souza, Danilo Turkievicz de. II. Souza, Rafael Teixeira. III. Título.

24-241695

CDD-004

Índices para catálogo sistemático:

1. Ciência da computação 004

Eliete Marques da Silva - Bibliotecária - CRB-8/9380

Introdução a *Machine Learning* e Redes Neurais

Instituições responsáveis

Universidade Federal de Goiás (UFG)

Centro de Competência Embrapii em Tecnologias Imersivas, denominado AKCIT (Advanced Knowledge Center for Immersive Technologies)

Centro de Excelência em Inteligência Artificial (CEIA)

Instituições financiadoras

Empresa Brasileira de Pesquisa e Inovação Industrial (Embrapii)

Governo do Estado de Goiás

Empresas parceiras do AKCIT

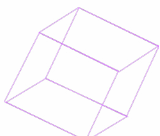
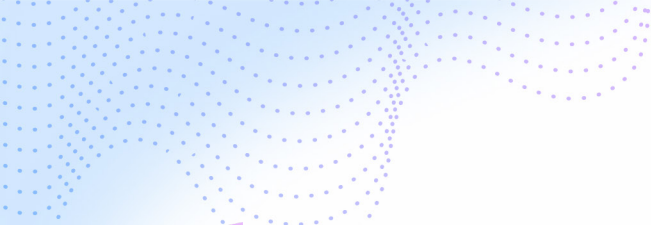
Apoio

Universidade Federal de Goiás (UFG)

Pró-Reitoria de Pesquisa e Inovação (PRPI-UFG)

Instituto de Informática (INF-UFG)





Lista de Abreviaturas e Siglas

1D	1 Dimensão
2D	2 Dimensões
3D	3 Dimensões
4D	4 Dimensões
AUC	Área Sob a Curva
BPTT	<i>Backpropagation Through Time</i> - Retropropagação Através do Tempo
CNN	<i>Convolutional Neural Network</i> - Rede Neural Convolucional
CUDA	<i>Compute Unified Device Architecture</i> - Arquitetura Unificada de Cálculo de Dispositivos
DBSCAN	<i>Density-Based Spatial Clustering of Applications with Noise</i> - Agrupamento de Aplicações Baseado em Densidade com Ruído
DL	<i>Deep Learning</i> - Aprendizado Profundo
GANs	<i>Generative Adversarial Networks</i> - Redes Generativas Adversárias
GPUs	<i>Graphics Processing Units</i> - Unidades de Processamento Gráfico
HOG	<i>Histograms of Oriented Gradients</i> - Histogramas de Gradientes Orientados
HSV	<i>Hue, Saturation and Value</i> - Matiz, Saturação e Valor
Hz	<i>Hertz</i>
IA	Inteligência Artificial
IID	<i>Independent and Identically Distributed</i> - Independentes e Identicamente Distribuídos



kNN	<i>k-Nearest Neighbors</i> - k-Vizinhos Mais Próximos
LASSO	<i>Least Absolute Shrinkage and Selection Operator</i> - Operador de Encolhimento e Seleção Absoluto Mínimo
LDA	<i>Linear Discriminant Analysis</i> - Análise Discriminante Linear
LightGBM	<i>Light Gradient Boosting Machine</i>
MAE	<i>Mean Absolute Error</i> - Erro Médio Absoluto
MFCC	<i>Mel-Frequency Cepstral Coefficients</i> - Coeficientes Cepstrais de Frequência Mel
ML	<i>Machine Learning</i> - Aprendizado de Máquina
MLP	<i>Multi-Layer Perceptron</i> - Perceptron de Múltiplas Camadas
MSE	<i>Mean Squared Error</i> - Erro Médio Quadrado
OCR	Reconhecimento Óptico de Caracteres - Reconhecimento Óptico de Caracteres
PCA	<i>Principal Component Analysis</i> - Análise de Componentes Principais
PLN	Processamento de Linguagem Natural
QDA	<i>Quadratic Discriminant Analysis</i> - Análise Discriminante Quadrática
RDA	Regularized Discriminant Analysis - Análise Discriminante Regularizada
ReLU	<i>Rectified Linear Unit</i> - Unidade Linear Retificada
RFE	<i>Recursive Feature Elimination</i> - Eliminação Recursiva de Atributos
RGB	<i>Red, Green and Blue</i> - Vermelho, Verde e Azul
RMSE	<i>Root Mean Squared Error</i> - Raiz do Erro Médio Quadrado
RNAs	<i>Artificial Neural Networks</i> - Redes Neurais Artificiais
RNN	<i>Recurrent Neural Network</i> - Rede Neural Recorrente
ROC	<i>Receiver Operating Characteristic</i> - Curva de Característica de Operação do Receptor

SGBD

Database Management Systems - Sistemas de Gestão de Bancos de Dados

SSR

Sum of Squared Residuals - Soma dos Quadrados dos Resíduos

SST

Total Sum of Squares - Soma Total dos Quadrados

SUV

Sports Utility Vehicles - Veículos Utilitários Desportivos

SVM

Support Vector Machines - Máquinas de Vetores de Suporte

t-SNE

t-Distributed Stochastic Neighbor Embedding - Embutimento Estocástico de Vizinhos Distribuídos t

TF-IDF

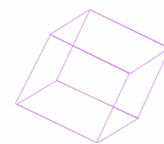
Term Frequency-Inverse Document Frequency - Frequência de Termos-Inversa Frequência de Documentos

TPUs

Tensor Processing Units - Unidades de Processamento de Tensor

XGBoost

Extreme Gradient Boosting



Lista de Figuras, Quadros e Tabelas

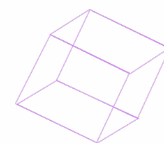
Figura 1 - Etapas de construção de um modelo de <i>machine learning</i>	24
Figura 2 - Aplicações de <i>machine learning</i> em diferentes setores	26
Figura 3 - Estreita relação entre inteligência artificial, <i>machine learning</i> e <i>deep learning</i>	27
Figura 4 - Série temporal com dados normais e uma anomalia em destaque	31
Figura 5 - Exemplo de reconhecimento de face	36
Figura 6 - Extração de características de uma imagem de íris	38
Figura 7 - Sequência de ácido desoxirribonucleico (DNA)	39
Figura 8 - Árvore de decisão para escolha de um carro	39
Figura 9 - Disposição de objetos em um espaço vetorial de três dimensões	42
Figura 10 - Imagem de classe, objeto e característica	42
Figura 11 - Processo de reconhecimento de padrões em <i>machine learning</i>	44
Figura 12 - Tipos de aprendizagem de máquina	48
Figura 13 - Treinamento de um algoritmo de aprendizagem supervisionada	49
Figura 14 - Treinamento de um algoritmo de aprendizagem não supervisionada	50
Figura 15 - Treinamento de um algoritmo de aprendizagem semi-supervisionada	51
Figura 16 - Treinamento de um algoritmo de aprendizagem por reforço	52
Figura 17 - Dados unidimensionais sobre altura (em cm)	57
Figura 18 - Dados bidimensionais que relacionam peso (em kg) e altura (em cm) de cinco pessoas	58
Figura 19 - Dados tridimensionais que relacionam peso (em kg), altura (em cm) e idade (em anos) de cinco pessoas	59

Figura 20 - Espaços de características bidimensional denso (à esquerda) e tridimensional esparsa (à direita)	60
Figura 21 - Espaço de características de peso e altura não normalizado (à esquerda) e normalizado (à direita)	62
Figura 22 - Transformação do espaço de características	63
Figura 23 - Exemplo tipo de fronteiras	70
Figura 24 - Na classificação binária utilizou-se uma função discriminante, na multi-classes, múltiplas funções discriminantes são utilizadas	70
Figura 25 - Matriz de covariância e as relações de co-dependência entre as variáveis	72
Figura 26 - Aplicação de transformações em vetor, (a) exemplo de autovalor e autovetor, (b) exemplo de um não autovetor e autovalor	73
Figura 27 - Gráfico com os dados do dataset iris	83
Figura 28 - Gráfico dos valores do X_{train_lda}	85
Figura 29 - Gráfico dos valores do X_{train_lda}	86
Figura 30 - Gráfico dos valores do X_{train_lda}	88
Figura 31 - Exemplo de dados estruturados e não estruturados	98
Figura 32 - Exemplo de tipos de dados para diferentes modelos	99
Figura 33 - Exemplo de técnicas para criação de variáveis categóricas	103
Figura 34 - Exemplo divisão do conjunto de dados	105
Figura 35 - Exemplo <i>k-fold</i>	106
Figura 36 - Exemplo de dados desbalanceados	107
Figura 37 - Exemplo do funcionamento do <i>Synthetic Minority Over-sampling Technique</i> (SMOTE)	109
Figura 38 - Exemplo de ajustes de curvas	111
Figura 39 - Exemplo de carregamento de imagem	115
Figura 40 - Exemplo de um conjunto de dados não supervisionados	120

Figura 41 - Visualização de valores ausentes	126
Figura 42 - Tratamento de Outliers	129
Figura 43 - Distribuição de dados antes da normalização - MinMax Scaling	133
Figura 44 - Distribuição de dados após normalização - MinMaxScaling	135
Figura 45 - Distribuição de dados após normalização - ZCore	136
Figura 46 - Exemplo da utilização do SMOTE	147
Figura 47 - Underfitting vs Overfitting	153
Figura 48 - Underfitting: Modelo Polinomial de Grau 1	157
Figura 49 - Overfitting: Modelo Polinomial de Alto Grau	158
Figura 50 - Fitting Adequado: Modelo Polinomial de Grau 3	159
Figura 51 - Exemplo R^2	165
Figura 52 - Exemplo de erro quadrático médio (MSE)	166
Figura 53 - Exemplo de erro absoluto médio (MAE)	168
Figura 54 - Exemplo de raiz do erro quadrático médio (RMSE)	169
Figura 55 - Exemplo de árvore de regressão - dados	175
Figura 56 - Exemplo de árvore de regressão - árvore	175
Figura 57 - Comparação entre redes neurais, RNN e Feed-Forward	177
Figura 58 - Relação entre Horas Estudadas e Pontuação no Teste	180
Figura 59 - Relação entre Horas Estudadas e Pontuação no Teste - Regressão Linear	181
Figura 60 - Gráfico SSR / SST	184
Figura 61 - Gráfico dos Resíduos ao quadrado e MSE	186
Figura 62 - Gráfico dos Erros Absolutos e do MAE	188
Figura 63 - Gráfico do Quadrado dos Resíduos e do RMSE	191
Figura 64 - Comparação Dados da Corrida de Automóveis	195
Figura 65 - Comparação das Métricas entre conjuntos de dados	196

Figura 66 - Gráfico para exemplificar o processo de Regressão Linear	198
Figura 67 - Gráfico de Plotagem dos Dados	199
Figura 68 - Gráfico da Soma do Quadrado dos Resíduos	201
Figura 69 - Gráfico do SSR do Novo Modelo	202
Figura 70 - Gráfico de Regressão Linear treinado com dados apenas de bmi	206
Figura 71 - Gráfico de Regressão Ridge treinado com dados apenas de bmi	207
Figura 72 - Gráfico da Árvore de Regressão treinado com dados apenas de bmi	208
Figura 73 - Gráfico RNN treinado com dados apenas de bmi	209
Figura 74 - Exemplos de aplicações de classificação	214
Figura 75 - Exemplos de diferentes amostras de uma classe “gato”	217
Figura 76 - Mapa mental classificadores na literatura de aprendizado de máquina	217
Figura 77 - Exemplos das três espécies de flores Iris	218
Figura 78 - Distribuição das amostras de flores com base no comprimento e largura da sépala	219
Figura 79 - Exemplo de aplicação do KNN em um novo dado	219
Figura 80 - Fronteira de discriminação de um KNN	220
Figura 81 - Matriz de confusão e seus componentes	222
Figura 82 - Exemplos de Curva ROC de diferentes classificadores	227
Figura 83 - Exemplo de uma árvore de decisão sobre o problema de decidir sair de casa ou não	228
Figura 84 - Ilustração do efeito do viés e variância	230
Figura 85 - Gráfico de linha representando os valores das acurácias do modelo k-NN	238
Figura 86 - Mapa mental com os algoritmos de agrupamento	242
Figura 87 - Ilustração de um <i>K-Means</i> aplicado com três agrupamentos	243
Figura 88 - Exemplos de aplicação do <i>K-means</i> e DBSCAN	245

Figura 89 - Exemplos de agrupamentos e índice de silhueta com uso do KNN com diferentes valores de k	246
Figura 90 - Gráfico - Seaborn	250
Figura 91 - Gráfico - Visualização de Cluster	253
Figura 92 - Esquema de um neurônio artificial - Perceptron	257
Figura 93 - Funções de ativação step e sigmoide	258
Figura 94 - Esquema básico de um <i>Multi-Layer Perceptron</i>	259
Figura 95 - Esquema básico de um <i>Multi-Layer Perceptron</i> com indicação de fluxo de dados no <i>feedforward</i> e <i>backpropagation</i>	260
Figura 96 - Exemplo de vetor gradiente em uma função com duas variáveis	261
Figura 97 - Funções de ativação sigmoide e ReLU	262
Figura 98 - Esquema básico de um <i>Multi-Layer Perceptron</i> ao receber uma imagem	264
Figura 100 - Esquema da AlexNet	265
Figura 101 - Exemplo de imagens do ImageNet	266
Figura 102 - Visualização das convoluções obtidas ao treinar uma CNN	267
Figura 103 - Visualização da atenção em um problema de tradução	268
Quadro 1 - Detecção, previsão e reconhecimento de <i>machine learning</i>	32
Quadro 2 - Comparativo entre as métricas abordadas resume essa seção	170
Tabela 1 - Comparativo entre problemas de detecção, reconhecimento e previsão	32
Tabela 2 - Exemplos de classes e objetos pertencentes a elas	43
Tabela 3 - Resumo das vantagens e desvantagens dos algoritmos que usam funções discriminantes	94
Tabela 4 - Resumo dos conjuntos de dados Iris, MNIST e CIFAR-10	100



Sumário

Apresentação	19
Unidade I - <i>Machine Learning</i>	20
1.1 O Que é <i>Machine Learning</i> ?	21
1.1.1 Etapas de Construção de um Modelo de <i>Machine Learning</i>	22
1.1.1.1 Coleta de Dados	22
1.1.1.2 Processamento de Dados	22
1.1.1.3 Treinamento de Modelos	23
1.1.1.4 Aplicação e Tomada de Decisão	23
1.2 Aplicações de <i>Machine Learning</i>	26
1.3 Inteligência Artificial x <i>Machine Learning</i> x <i>Deep Learning</i>	26
Unidade II - Detecção, reconhecimento e previsão	29
2.1 Detecção de padrões e anomalias	30
2.2 Detecção, Previsão e Reconhecimento	31
Unidade III - Reconhecimento de padrões	34
3.1 O Que é Reconhecimento?	35
3.1.1 Treinamento de Modelos	36
3.2 Padrões	37
3.2.1 Arranjos de Padrões	37
3.2.2 Objetos e Características	40
3.2.3 Classes	41

Unidade IV - Tipos de Aprendizagem de Máquina	46
4.1 Aprendizagem Supervisionada	48
4.2 Aprendizagem Não Supervisionada	49
4.3 Aprendizagem Semi-supervisionada	50
4.4 Aprendizagem por Reforço	51
 Unidade V - Espaço de Características	 54
5.1 Extração de Características	55
5.2 Seleção de características	56
5.3 Características vs. Dimensionalidade	57
5.4 Maldição da Dimensionalidade	60
5.5 Normalização do Espaço de Características	61
5.6 Transformação do Espaço de Características	62
5.7 Redução da Dimensionalidade	63
5.8 Seleção de Características vs. Redução da Dimensionalidade	64
 Unidade VI - Funções Discriminantes	 66
6.1 Introdução	67
6.2 Aplicações	67
6.3 Conceitos Importantes	68
6.3.1 Espaço de Características	69
6.3.2 Fronteiras de Decisão	69
6.3.3 Classificação Binária vs. Multiclasse	70
6.4 Principais Algoritmos	71
6.4.1 Conceitos Matemáticos Importantes	71
6.4.1.1 Matriz de Covariância ou de Dispersão	71
6.4.1.2 Autovetores e Autovalores	73
6.4.2 Análise Discriminante Linear (LDA)	74
6.4.2.1 Detalhamento do Código	75

6.4.3. Análise Discriminante Quadrática (QDA)	77
6.4.4 Análise Discriminante Regularizada (RDA)	79
6.5 Notebook Colab	81

Unidade VII - Conjuntos de dados 95

7.1 A Importância dos Dados para a Construção de Modelos de ML	96
7.2 Dados Estruturados vs. Dados Não Estruturados	97
7.3 Conjuntos de Dados Supervisionados, Não Supervisionados e Semi-supervisionados	98
7.4 Limpeza de Dados (Tratamento de Valores Ausentes, Outliers , Erros de Digitação)	101
7.5 Pré-processamento (Normalização, Codificação de Variáveis Categóricas)	102
7.6 Seleção e Engenharia de Características (Feature Selection e Feature Engineering)	104
7.7 Divisão em Conjuntos de Treino, Validação e Teste	104
7.8 Dados Desbalanceados e Técnicas para Balanceamento	107
7.8.1 Técnica de Oversampling	108
7.8.2 Técnica de Undersampling	109
7.9 Subajuste (Underfitting) e Sobreajuste (Overfitting)	110
7.10 Notebook Colab:	112

Unidade VIII - Regressão 162

8.1 Métricas de Avaliação	164
8.1.1 Coeficiente de Determinação R^2	164
8.1.2 Erro Quadrático Médio (MSE)	165
8.1.3 Erro Absoluto Médio (MAE)	167
8.1.4 Raiz do Erro Quadrático Médio (RMSE)	168
8.2 Tipos de Regressão	170
8.2.1 Regressão Linear	170

8.2.2 Regressão Ridge , Lasso e Elastic Net	171
8.2.3 Árvores de Regressão	173
8.2.4 Recurrent Neural Network (RNN) ou Rede Neural Recorrente	176
8.3 Notebook Colab	178

Unidade IX - Classificação 213

9.1 Classificadores	216
9.1.1 KNN: k-Nearest Neighbors	218
9.2 Avaliação de Classificadores	220
9.2.1 Métricas para Classificação	221
9.2.1.1 Acurácia	222
9.2.1.2 Precisão	223
9.2.1.3 Recall	223
9.2.1.4 F1- score	224
9.2.1.5 Matriz de confusão	224
9.2.1.6 Acurácia em Detecção de Fraude	225
9.2.1.7 Precisão em Diagnóstico Médico (Detecção de Câncer)	225
9.2.1.8 F1- Score em Detecção de Spam	225
9.2.1.9 Curva ROC	226
9.3 Classificadores Modernos	227
9.3.1 XGBoost	228
9.3.2 LightGBM	229
9.4 Notebook Colab	230

Unidade X - Agrupamento 241

10.1 K-Means	243
10.2 DBSCAN	244
10.3 Avaliação de Agrupamentos	245
10.4 Notebook Colab	246

Unidade XI - Redes Neurais	255
11.1 Neurônio Artificial	256
11.2 <i>Multi-Layer Perceptron</i>	258
11.3 <i>Deep Learning</i>	262
11.3.1 CNNs: Redes Neurais Convolucionais	263
11.3.2 <i>Transformer</i>	267
11.3.3 Rede Neurais Generativas	268
11.3.4 <i>Frameworks</i> para DL	269
Unidade XII - Encerramento	271
12.1 Considerações Finais	272
Referências	274

Apresentação

Prezado(a) Participante,

Seja bem-vindo(a) ao Microcurso **Introdução a Machine Learning e Redes Neurais!**

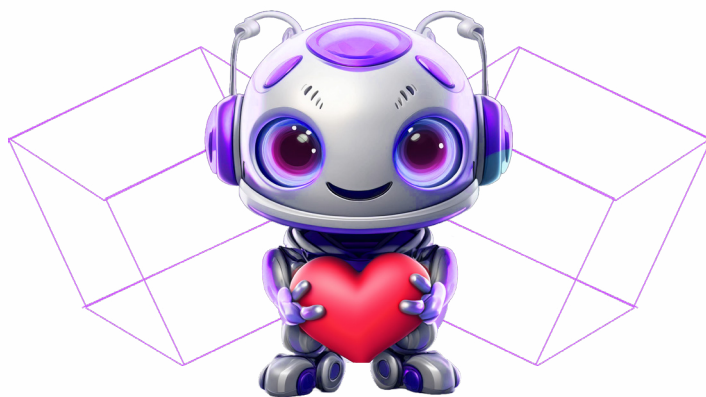
Este microcurso faz parte da Coleção Formação e Capacitação do Centro de Competências Imersivas, uma parceria entre a Embrapii e a Universidade Federal de Goiás (UFG).

Neste Microcurso, você será introduzido aos conceitos fundamentais de *machine learning*, aprendendo a construir modelos, desde a coleta e o processamento de dados até a aplicação prática em problemas reais. Começaremos com uma visão geral do aprendizado de máquina, explorando suas principais etapas e diferenciando-o de outras áreas da inteligência artificial.

À medida que avançamos, aprofundaremos em técnicas de reconhecimento de padrões, essenciais para desenvolver modelos precisos. Abordaremos os diferentes tipos de aprendizado de máquina e exploraremos o espaço de características, discutindo como selecionar e transformar dados para otimizar o desempenho dos modelos.

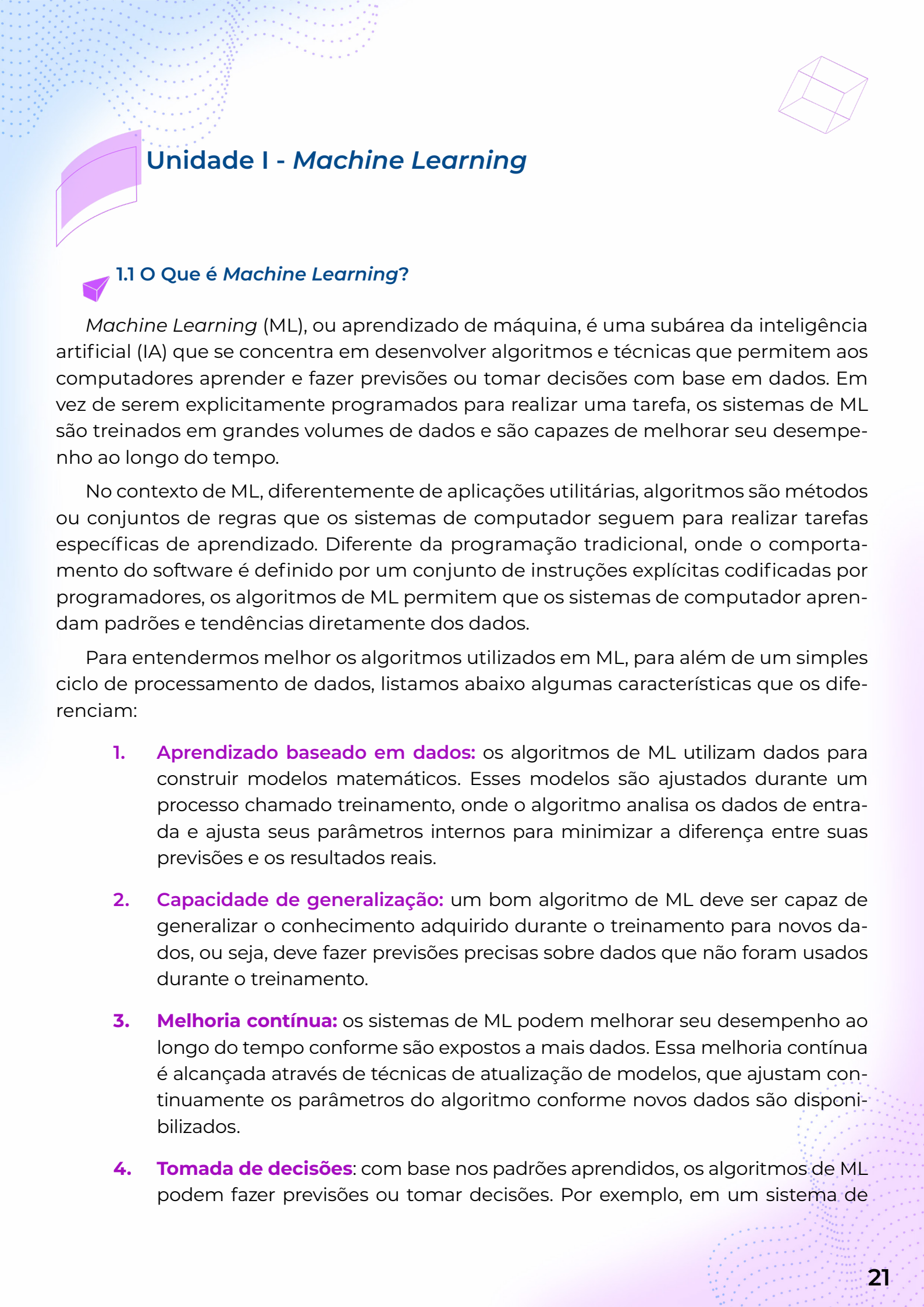
Na etapa final, exploraremos técnicas avançadas, como funções discriminantes e as aplicações práticas de regressão e classificação, além de uma introdução às redes neurais. Você terá a oportunidade de aprender sobre arquiteturas neurais modernas, compreender seus princípios e ver como aplicá-las em problemas que ilustram o potencial dessas tecnologias.

Este Microcurso oferece uma visão abrangente e prática que o prepara para os desafios do aprendizado de máquina, equipando você com as ferramentas e o conhecimento necessários para seguir em frente na sua jornada na inteligência artificial.



Desejamos a você uma excelente jornada de aprendizado!

Unidade I
Machine Learning



Unidade I - Machine Learning

1.1 O Que é Machine Learning?

Machine Learning (ML), ou aprendizado de máquina, é uma subárea da inteligência artificial (IA) que se concentra em desenvolver algoritmos e técnicas que permitem aos computadores aprender e fazer previsões ou tomar decisões com base em dados. Em vez de serem explicitamente programados para realizar uma tarefa, os sistemas de ML são treinados em grandes volumes de dados e são capazes de melhorar seu desempenho ao longo do tempo.

No contexto de ML, diferentemente de aplicações utilitárias, algoritmos são métodos ou conjuntos de regras que os sistemas de computador seguem para realizar tarefas específicas de aprendizado. Diferente da programação tradicional, onde o comportamento do software é definido por um conjunto de instruções explícitas codificadas por programadores, os algoritmos de ML permitem que os sistemas de computador aprendam padrões e tendências diretamente dos dados.

Para entendermos melhor os algoritmos utilizados em ML, para além de um simples ciclo de processamento de dados, listamos abaixo algumas características que os diferenciam:

1. **Aprendizado baseado em dados:** os algoritmos de ML utilizam dados para construir modelos matemáticos. Esses modelos são ajustados durante um processo chamado treinamento, onde o algoritmo analisa os dados de entrada e ajusta seus parâmetros internos para minimizar a diferença entre suas previsões e os resultados reais.
2. **Capacidade de generalização:** um bom algoritmo de ML deve ser capaz de generalizar o conhecimento adquirido durante o treinamento para novos dados, ou seja, deve fazer previsões precisas sobre dados que não foram usados durante o treinamento.
3. **Melhoria contínua:** os sistemas de ML podem melhorar seu desempenho ao longo do tempo conforme são expostos a mais dados. Essa melhoria contínua é alcançada através de técnicas de atualização de modelos, que ajustam continuamente os parâmetros do algoritmo conforme novos dados são disponibilizados.
4. **Tomada de decisões:** com base nos padrões aprendidos, os algoritmos de ML podem fazer previsões ou tomar decisões. Por exemplo, em um sistema de

recomendação, um algoritmo pode prever quais produtos um usuário pode gostar com base em seu histórico de compras e comportamento de navegação.

1.1.1 Etapas de Construção de um Modelo de *Machine Learning*

A transformação de dados brutos em decisões inteligentes utilizando ML é um processo sistemático que envolve várias etapas essenciais, desde a coleta inicial dos dados até a aplicação prática de modelos treinados. Cada etapa desempenha um papel crucial na construção de modelos que podem aprender com os dados e fazer previsões ou tomar decisões com base neles. A seguir, explicamos as principais etapas desse processo.

1.1.1.1 Coleta de Dados

A primeira etapa é a coleta de dados brutos, que são os dados não processados coletados diretamente da fonte. Esses dados podem vir de diversas fontes, como sensores, *logs* de atividades, interações de usuários, registros financeiros, dados de redes sociais, entre outros. A qualidade e a quantidade dos dados coletados são fundamentais, pois eles formam a base sobre a qual todo o processo de ML será construído. Essa etapa é importante pois:

- » **A coleta adequada garante que os dados capturem as informações relevantes para o problema que se deseja resolver.**
- » **Dados variados e de alta qualidade aumentam a capacidade do modelo de generalizar e fazer previsões precisas.**

1.1.1.2 Processamento de Dados

Após a coleta, os dados brutos passam por um processo de limpeza e preparação, conhecido como pré-processamento. Esta etapa envolve a remoção de ruídos, tratamento de valores ausentes, transformação de variáveis categóricas, normalização, e extração de características. O objetivo é transformar os dados brutos em um formato estruturado e adequado para a modelagem. Essa etapa é dividida nas seguintes sub-etapas:

- » **Limpeza de dados:** remoção de inconsistências, duplicatas e valores ausentes.
- » **Transformação de dados:** conversão de dados categóricos em numéricos, normalização e padronização.
- » **Extração de características:** identificação e seleção de atributos relevantes que ajudarão na construção do modelo.

Podemos destacar os seguintes pontos que tornam essa etapa igualmente importante:

- » **Dados bem processados reduzem a complexidade do modelo e aumentam sua precisão.**
- » **Ajuda a prevenir problemas como o *overfitting* (sobreajuste) e melhora a performance do modelo.**



1.1.1.3 Treinamento de Modelos

Nesta etapa, os dados processados são utilizados para treinar o modelo de ML. O treinamento envolve a aplicação de algoritmos que aprendem a partir dos dados, ajustando parâmetros internos para minimizar o erro de previsão. Durante o treinamento, o modelo é exposto a exemplos rotulados (em aprendizado supervisionado) ou a padrões nos dados (em aprendizado não supervisionado) para identificar as relações subjacentes. São sub-etapas a serem destacadas?

- » **Divisão dos dados:** Os dados são divididos em conjuntos de treinamento e teste para avaliar a performance do modelo.
- » **Treinamento:** O modelo é ajustado iterativamente para melhorar sua capacidade de prever os resultados corretos.
- » **Validação:** Técnicas como validação cruzada são usadas para garantir que o modelo não se ajuste excessivamente aos dados de treinamento (*overfitting*).

Algumas questões devem ser levadas em consideração para execução dessa etapa, sendo elas:

- » **Um bom treinamento é crucial para que o modelo aprenda os padrões corretos nos dados.**
- » **O modelo deve ser capaz de generalizar bem para novos dados que ele não tenha visto durante o treinamento.**



1.1.1.4 Aplicação e Tomada de Decisão

Depois que o modelo é treinado e validado, ele é aplicado em novos dados para fazer previsões ou tomar decisões. Nesta fase, o modelo é implementado em um ambiente de produção onde ele pode ser usado para automatizar processos, prever resultados futuros, ou tomar decisões baseadas em dados em tempo real. São exemplos de aplicações em ambientes de produção reais:

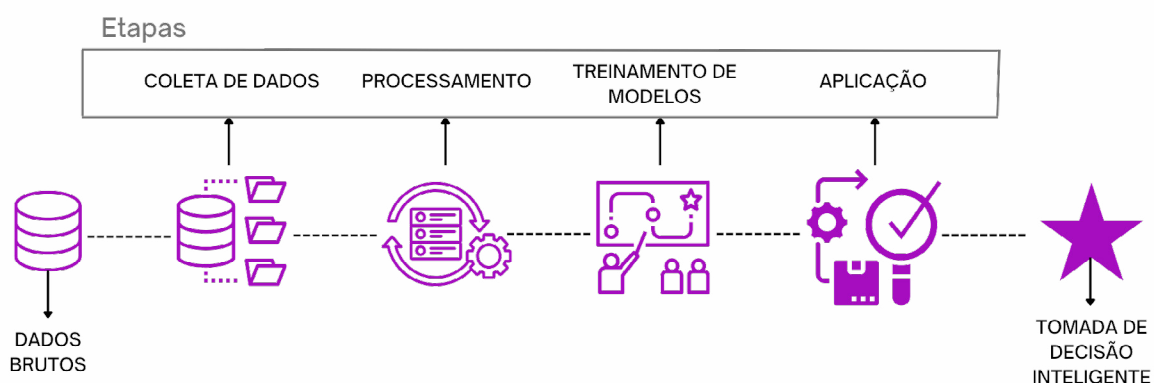
- » **Detecção de fraudes:** Identificação de transações suspeitas em sistemas financeiros.
- » **Recomendação de produtos:** Sugestão de produtos personalizados para usuários em plataformas de e-commerce (comércio digital).
- » **Previsão de demandas:** Estimativa de demanda futura para otimização de estoques.

Mesmo sendo as duas últimas etapas do processo de construção de um modelo de ML, ainda podemos destacar pontos de atenção visando garantir o sucesso do projeto:

- » **A aplicação bem-sucedida do modelo permite que as organizações tomem decisões mais informadas, rápidas e precisas.**
- » **Modelos implementados em produção podem melhorar continuamente à medida que recebem novos dados e feedback.**

Portanto, a evolução de dados brutos para a tomada de decisão inteligente em ML é um processo iterativo e dinâmico, que exige atenção cuidadosa em cada etapa. Desde a coleta de dados até a aplicação do modelo, cada fase contribui para o desenvolvimento de soluções que não só aprendem com os dados, mas também melhoram a eficiência e a precisão na tomada de decisões. Este processo, ilustrado na Figura 1, é fundamental para transformar grandes volumes de dados em insights (percepções) acionáveis, permitindo que as organizações respondam de forma eficaz a desafios e oportunidades em tempo real.

Figura 1 - Etapas de construção de um modelo de *machine learning*



Fonte: adaptada de [Waltrick \(2020\)](#).

Também podemos compreender os algoritmos de ML dividindo-os quanto ao tipo de aprendizado, que discutiremos com mais detalhes adiante:

- » **Algoritmos supervisionados:** nestes algoritmos, o modelo é treinado com um conjunto de dados rotulados, ou seja, os dados de entrada são acompanhados por resultados desejados. Exemplos incluem regressão linear e classificadores de árvore de decisão.
- » **Algoritmos não-supervisionados:** o modelo é treinado com dados que não possuem rótulos. O objetivo é encontrar padrões ou estruturas subjacentes nos dados. Exemplos incluem algoritmos de agrupamentos como *k-means* e métodos de redução de dimensionalidade como Análise de Componentes Principais (PCA).
- » **Algoritmos semi-supervisionados:** estes algoritmos utilizam uma combinação de dados rotulados e não rotulados para o treinamento. Eles são particularmente úteis quando a rotulagem de dados é cara ou demorada.
- » **Algoritmos de aprendizado por reforço:** baseados em um sistema de recompensas e punições, esses algoritmos aprendem a tomar decisões sequenciais para maximizar uma recompensa cumulativa. Um exemplo comum é o treinamento de agentes de IA para jogar jogos de vídeo.

Citamos algumas técnicas como classificação, regressão e agrupamentos para explicar os algoritmos de ML com exemplos. Neste microcurso, você terá a oportunidade de desenvolver, testar e melhorar cada uma dessas técnicas no decorrer das unidades; até lá, abaixo citamos alguns exemplos de aplicações desses algoritmos:

- » **Classificação:** diferenciar emails em “spam” e “não spam”.
- » **Regressão:** prever o preço de uma casa com base em suas características.
- » **Agrupamentos:** agrupar clientes com comportamentos de compra semelhantes para campanhas de marketing direcionadas.
- » **Recomendações:** sugerir filmes ou produtos com base nas preferências do usuário.

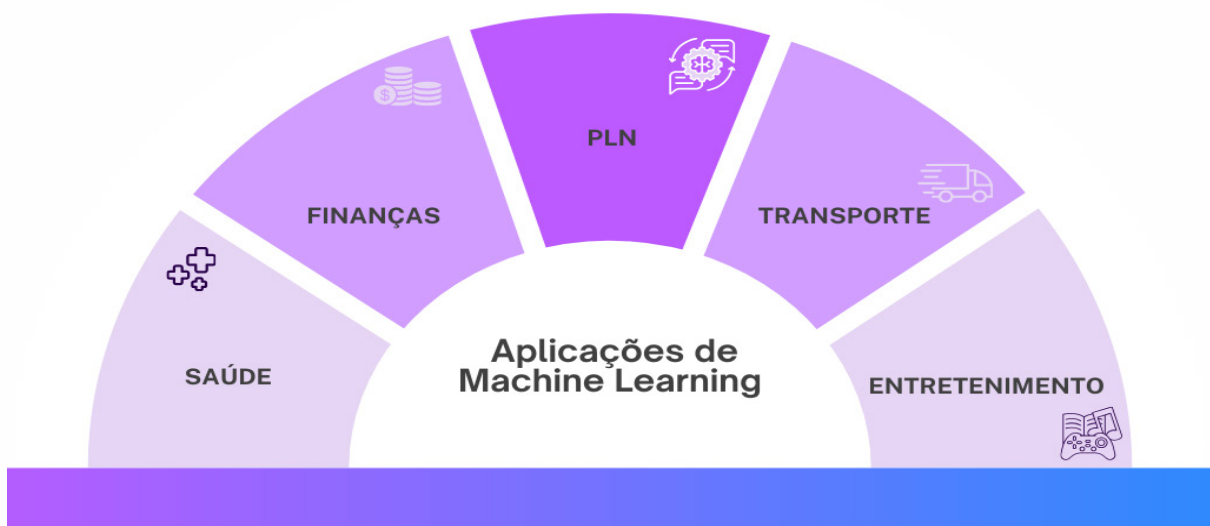
Os algoritmos são a base do aprendizado de máquina, permitindo que sistemas computacionais façam previsões e tomem decisões informadas a partir de grandes volumes de dados. Eles são fundamentais para a criação de sistemas inteligentes que podem evoluir e melhorar continuamente, ampliando as capacidades da IA.

1.2 Aplicações de *Machine Learning*

As aplicações de ML são vastas e se estendem por diversos setores (Figura 2). Alguns exemplos incluem:

- » **Reconhecimento de imagem:** utilizado em sistemas de segurança, diagnósticos médicos (radiologia) e redes sociais para reconhecimento facial;
- » **Processamento de linguagem natural (PLN):** aplicado em assistentes virtuais, tradução automática, análise de sentimentos e *chatbots*;
- » **Sistemas de recomendação:** utilizado por empresas como Netflix, Amazon e Spotify para sugerir produtos, filmes ou músicas com base nas preferências do usuário;
- » **Previsão de mercado:** usado em finanças para prever preços de ações, analisar risco de crédito e detectar fraudes;
- » **Veículos autônomos:** utilizam ML para processar informações sensoriais e tomar decisões em tempo real sobre navegação e segurança;
- » **Saúde:** aplicado em diagnósticos, personalização de tratamentos e predição de surtos de doenças.

Figura 2 - Aplicações de *machine learning* em diferentes setores



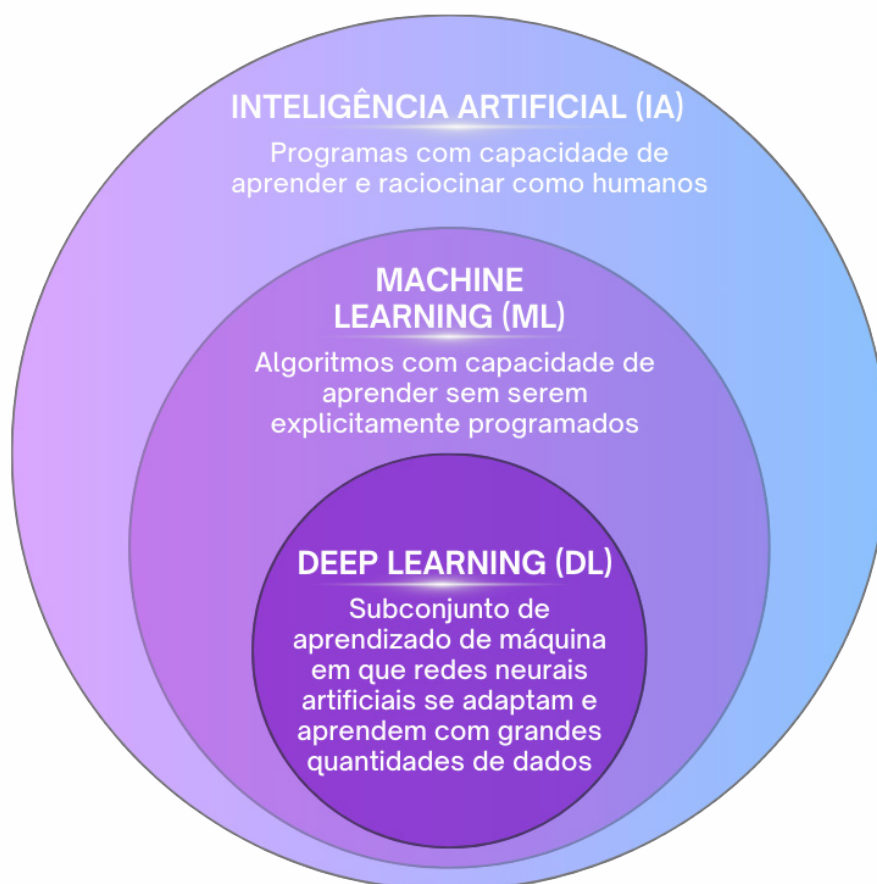
Fonte: autoria própria.

1.3 Inteligência Artificial x *Machine Learning* x *Deep Learning*

É importante entender a relação entre IA, ML e *Deep Learning* (DL) ou Aprendizagem Profunda. Apesar de apresentadas na Figura 3 como áreas independentes com interseções com suas correlatas, não temos um limite claro onde cada uma começa e termina. Para fins didáticos, vamos explicá-las separadamente:

1. **Inteligência artificial:** trata-se de um campo amplo da ciência da computação que visa criar sistemas capazes de realizar tarefas que normalmente requerem inteligência humana. Isso inclui raciocínio, aprendizado, percepção, e interação.
2. **Machine learning:** é um subcampo da IA focado no desenvolvimento de algoritmos que permitem às máquinas aprender a partir de dados e melhorar com a experiência. Em vez de serem explicitamente programadas para cada tarefa, as máquinas usam dados para identificar padrões e tomar decisões.
3. **Deep learning:** é uma subárea de ML que utiliza Redes Neurais Artificiais (RNAs) com muitas camadas (profundas) para modelar e aprender representações de dados complexos. DL tem sido particularmente eficaz em tarefas como reconhecimento de imagem, PLN e jogo autônomo.

Figura 3 - Estreita relação entre inteligência artificial, *machine learning* e *deep learning*



Fonte: adaptada de [Medeiros \(2019\)](#).



PARA RELEMBRAR...

- » ML é uma subárea da IA que desenvolve algoritmos capazes de aprender e melhorar a partir de dados, em vez de serem explicitamente programados.
- » As aplicações de ML são vastas e incluem reconhecimento de imagem, PLN, sistemas de recomendação, previsão de mercado, veículos autônomos e diagnósticos médicos.
- » É crucial diferenciar entre IA, ML e DL: IA é o campo geral que busca criar sistemas inteligentes; ML é uma subárea de IA focada em algoritmos de aprendizado a partir de dados; e DL é uma subárea de ML que utiliza Redes Neurais Profundas (DL) para modelar dados complexos. Juntos, esses conceitos formam a base para a compreensão e aplicação de tecnologias inteligentes em diversas áreas.

SAIBA MAIS...

- » Artigo: “[What is machine learning?](#)” (IBM). Um artigo introdutório que explica o que é ML, como funciona e onde é aplicado.
- » Artigo: “[A tour of machine learning algorithms](#)”, de Jason Brownlee (*Machine Learning Mastery*). Um artigo abrangente que descreve diferentes tipos de algoritmos de ML, suas características e aplicações.

Unidade II
**Detecção,
reconhecimento
e previsão**



Unidade II - Detecção, reconhecimento e previsão



2.1 Detecção de padrões e anomalias

A detecção de padrões e anomalias é uma das principais tarefas no pré-processamento de dados e uma aplicação crucial em ML. Identificar padrões envolve encontrar regularidades ou estruturas repetitivas nos dados. No contexto de reconhecimento de padrões, “regularidades” ou “estruturas repetitivas” referem-se a padrões recorrentes e consistentes que podem ser identificados em conjuntos de dados. Essas regularidades são fundamentais para o aprendizado de máquina e o reconhecimento de padrões porque fornecem a base para a criação de modelos que podem generalizar para novos dados. Esses padrões recorrentes podem se manifestar como correlações entre variáveis, tendências ao longo do tempo, ou agrupamentos de dados com características semelhantes. Uma vez encontrados esses padrões, eles podem ser usados para criar modelos que façam previsões ou classifiquem novos dados. Exemplos de regularidades incluem:

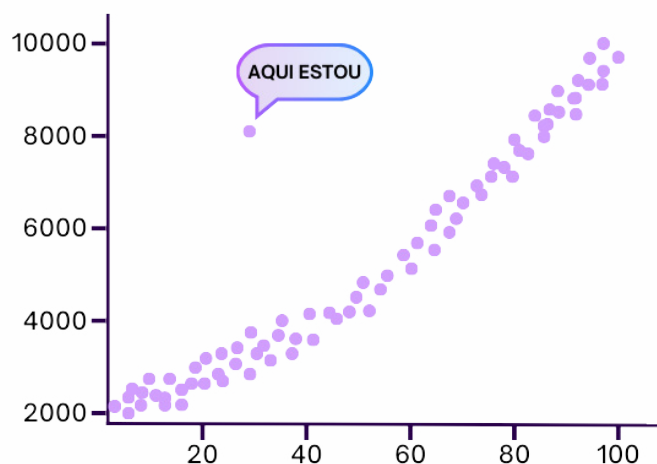
1. **Séries temporais:** em dados de séries temporais, como medições de temperatura ao longo do tempo, uma regularidade pode ser uma tendência de aumento ou diminuição. Padrões sazonais, como aumento de vendas de determinados produtos durante feriados, também são estruturas repetitivas.
2. **Imagens:** no reconhecimento de imagens, estruturas repetitivas podem ser características como bordas, texturas ou formas específicas que aparecem consistentemente em imagens de um determinado objeto ou classe de objetos.
3. **Texto:** em NLP, podem ser frequências de palavras, padrões de coocorrência de palavras, ou estruturas sintáticas que aparecem repetidamente em um corpus de texto.
4. **Dados tabulares:** em dados de clientes, por exemplo, podem ser comportamentos de compra, como a tendência de certos grupos demográficos a comprar produtos específicos. Padrões de uso, como horários de acesso a um serviço, também são estruturas repetitivas.

Como já discutimos, a identificação de regularidades ou estruturas repetitivas nos dados é fundamental para o reconhecimento de padrões. Essas regularidades são a base sobre a qual modelos de aprendizado são construídos, permitindo que os sistemas façam previsões, classifiquem dados e tomem decisões informadas. A seguir, citamos 3 situações em que a identificação de regularidades é aplicada:

1. **Modelagem:** identificar regularidades permite a criação de modelos que capturam relações consistentes. Por exemplo, um modelo de regressão linear pode capturar a relação entre a temperatura e a demanda por eletricidade.
2. **Generalização:** modelos que capturam regularidades nos dados de treinamento são capazes de generalizar para novos dados. Isso significa que o modelo pode fazer previsões precisas ou categorizações baseadas em dados não vistos, pois as regularidades subjacentes continuam presentes.
3. **Redução de ruído:** identificar estruturas repetitivas ajuda a separar os sinais relevantes do ruído nos dados. Isso permite que os modelos de reconhecimento de padrões se concentrem nas informações importantes e ignorem variações aleatórias, frequentemente chamadas de *outliers*.

Anomalias, por outro lado, são dados que não seguem o comportamento esperado ou padrão (veja Figura 4). Detectar anomalias é essencial para identificar erros, fraudes ou eventos raros. Por exemplo, na detecção de fraudes em transações financeiras, um sistema de ML pode aprender os padrões normais de gastos de um usuário e detectar transações que não se encaixam nesses padrões. Em manutenção preditiva, anomalias podem indicar falhas iminentes em máquinas ou equipamentos, permitindo intervenções antes que ocorram falhas catastróficas.

Figura 4 - Série temporal com dados normais e uma anomalia em destaque



Fonte: Adaptada de [Pandya, Jadeja, Degadwala \(2022\)](#).



2.2 Detecção, Previsão e Reconhecimento

Em ML, os termos “detecção”, “reconhecimento” e “previsão” referem-se a diferentes tipos de tarefas que os algoritmos podem executar. Cada um desses conceitos tem suas próprias características e aplicações específicas. Exploramos cada um deles em detalhes no Quadro 1.

Detecção, Previsão e Reconhecimento em Machine Learning

Em Machine Learning, os termos "detecção", "reconhecimento" e "previsão" referem-se a diferentes tipos de tarefas que os algoritmos podem executar. Cada um desses conceitos tem suas próprias características e aplicações específicas. Vamos explorar cada um deles em detalhes



Detecção

Descrição: Detecção é o processo de identificar a presença de um objeto, evento ou padrão específico em um conjunto de dados. O objetivo da detecção é simplesmente determinar se algo está presente ou ausente nos dados.

Características: **Foco na existência** - A detecção se concentra em verificar a existência de um determinado padrão, sem necessariamente identificar ou classificar o objeto.

Aplicações: Pode ser usada para detectar anomalias, presença de objetos em imagens, ou ocorrências de eventos em séries temporais.

Exemplos:

- Identificar se uma imagem contém um carro.
- Detectar transações fraudulentas em dados financeiros.
- Detectar picos de carga em dados de consumo de energia.



Fonte: autoria própria.

Para sintetizar essas diferenças, abaixo, na Tabela 1, estão estruturadas as características de cada abordagem.

Tabela 1 - Comparativo entre problemas de detecção, reconhecimento e previsão

Abordagem	Detecção	Reconhecimento	Previsão
Objetivo	Identificar a presença de algo	Identificar e classificar algo	Estimar valores ou eventos futuros
Foco	Existência	Identidade	Futuro
Exemplo	Detectar a presença de um carro em uma imagem	Identificar a marca e modelo do carro na imagem	Prever o número de carros que passarão por uma rua
Aplicação	Detecção de anomalias, detecção de eventos	Reconhecimento facial, reconhecimento de voz	Previsão de vendas, previsão do tempo

Fonte: autoria própria.



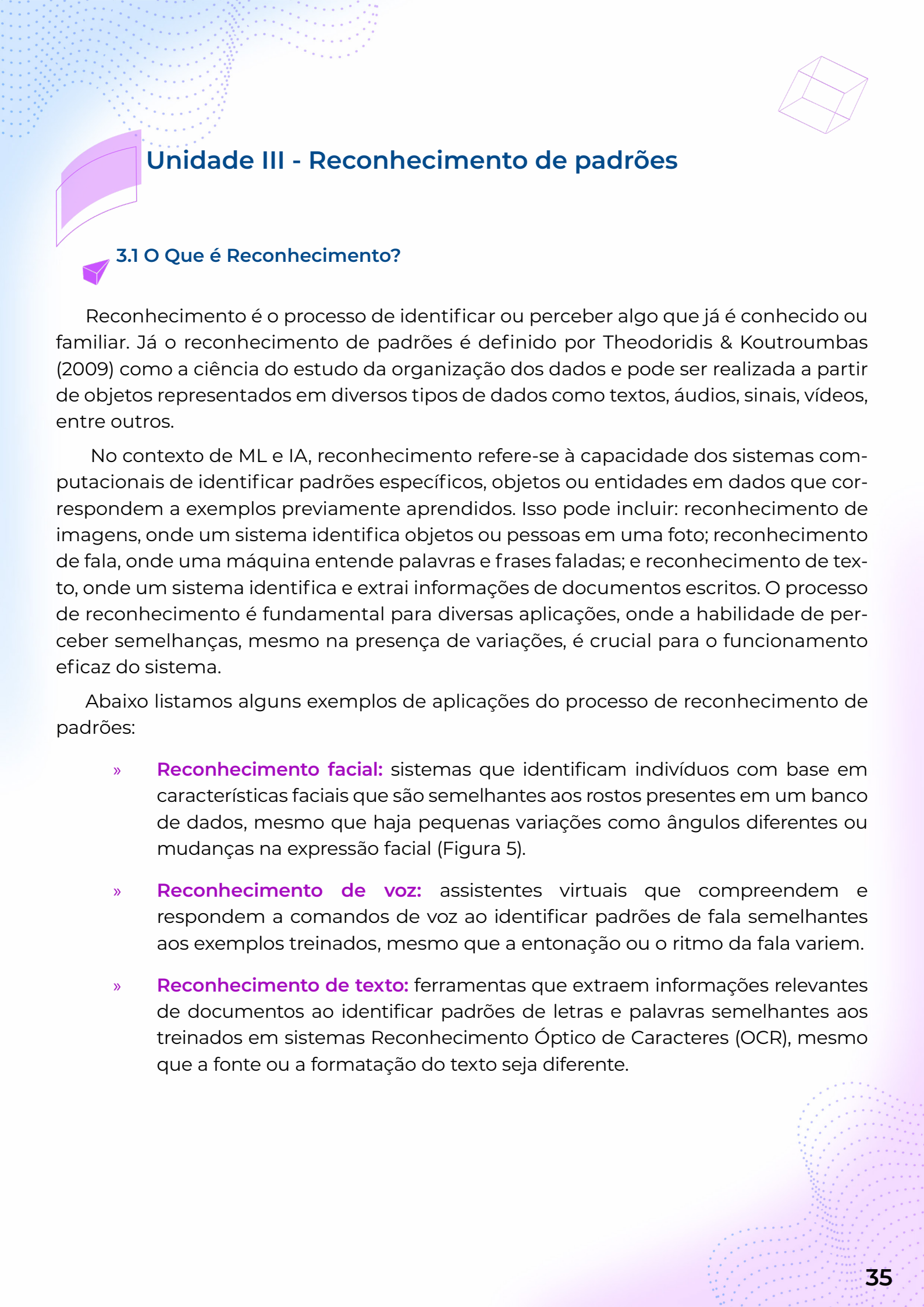
PARA RELEMBRAR...

- » No pré-processamento de dados, a detecção de padrões e anomalias desempenha um papel crucial, permitindo identificar regularidades e *outliers* nos dados.
- » As tarefas de detecção, reconhecimento e previsão são fundamentais em ML, ajudando a identificar a presença de características específicas, reconhecer entidades e prever resultados futuros.
- » Essas tarefas são aplicadas em diversas áreas, desde a detecção de fraudes até a previsão de demanda, e são essenciais para a criação de modelos de ML precisos e confiáveis.

SAIBA MAIS...

- » [Artigos sobre diversos tópicos em ciência de dados e *machine learning*.](#)
- » [Um site que explora conceitos complexos de *machine learning* de forma visual e intuitiva.](#)

Unidade III
**Reconhecimento
de padrões**



Unidade III - Reconhecimento de padrões

3.1 O Que é Reconhecimento?

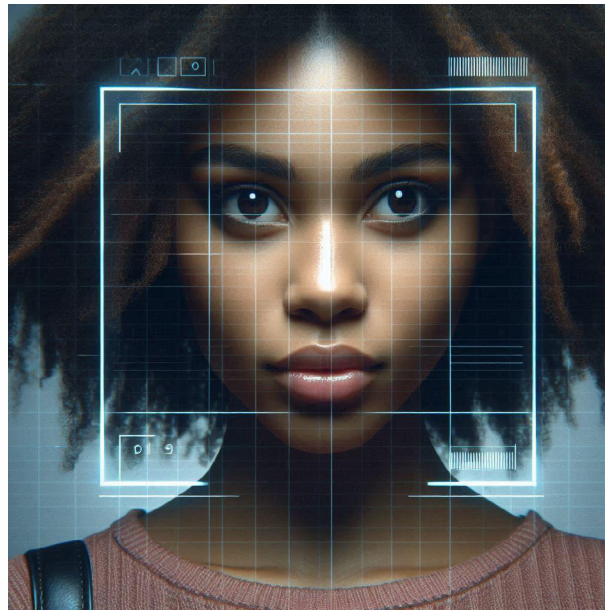
Reconhecimento é o processo de identificar ou perceber algo que já é conhecido ou familiar. Já o reconhecimento de padrões é definido por Theodoridis & Koutroumbas (2009) como a ciência do estudo da organização dos dados e pode ser realizada a partir de objetos representados em diversos tipos de dados como textos, áudios, sinais, vídeos, entre outros.

No contexto de ML e IA, reconhecimento refere-se à capacidade dos sistemas computacionais de identificar padrões específicos, objetos ou entidades em dados que correspondem a exemplos previamente aprendidos. Isso pode incluir: reconhecimento de imagens, onde um sistema identifica objetos ou pessoas em uma foto; reconhecimento de fala, onde uma máquina entende palavras e frases faladas; e reconhecimento de texto, onde um sistema identifica e extrai informações de documentos escritos. O processo de reconhecimento é fundamental para diversas aplicações, onde a habilidade de perceber semelhanças, mesmo na presença de variações, é crucial para o funcionamento eficaz do sistema.

Abaixo listamos alguns exemplos de aplicações do processo de reconhecimento de padrões:

- » **Reconhecimento facial:** sistemas que identificam indivíduos com base em características faciais que são semelhantes aos rostos presentes em um banco de dados, mesmo que haja pequenas variações como ângulos diferentes ou mudanças na expressão facial (Figura 5).
- » **Reconhecimento de voz:** assistentes virtuais que compreendem e respondem a comandos de voz ao identificar padrões de fala semelhantes aos exemplos treinados, mesmo que a entonação ou o ritmo da fala variem.
- » **Reconhecimento de texto:** ferramentas que extraem informações relevantes de documentos ao identificar padrões de letras e palavras semelhantes aos treinados em sistemas Reconhecimento Óptico de Caracteres (OCR), mesmo que a fonte ou a formatação do texto seja diferente.

Figura 5 - Exemplo de reconhecimento de face



Fonte: banco de imagens Canva.



3.1.1 Treinamento de Modelos

Mas você deve estar pensando: na prática, como ocorre o reconhecimento de algo? Vamos detalhar todo esse processo mas, descrevendo de uma forma direta, em ML, o reconhecimento é realizado por meio da aproximação de um modelo previamente treinado.

De forma simples, um modelo em ML é um conjunto de regras ou um sistema matemático que foi produzido durante a etapa de treinamento para fazer previsões ou tomar decisões com base em dados. Pense nele como uma fórmula que aprende a partir de exemplos fornecidos e é capaz de aplicar esse aprendizado a novos dados.

Para reconhecer novos dados, o sistema utiliza o modelo que foi treinado com dados conhecidos. Durante o treinamento, o modelo aprende a identificar padrões e características nos dados de entrada. Quando um novo dado é apresentado, o modelo compara esse dado com os padrões aprendidos e faz uma previsão ou classificação com base na similaridade. E essa similaridade é uma medida numérica de quão parecidos dois itens são entre si, determinando o grau de semelhança entre eles.

Por exemplo, em reconhecimento facial, um modelo é treinado com milhares de imagens de rostos diferentes. O modelo aprende as características faciais, como a distância entre os olhos, a forma do nariz e a linha do queixo. Quando uma nova imagem de rosto é apresentada, o modelo analisa essas características e tenta encontrar uma correspondência com os rostos conhecidos.

Os modelos são treinados com grandes volumes de dados históricos, e é essa quantidade de informação que torna possível fazer previsões e identificar padrões em novos dados.

Agora que você sabe o que é e como é realizado o reconhecimento, vamos introduzir o próximo conceito: padrões.

3.2 Padrões

Padrão, em um primeiro momento, pode ser entendido como uma sequência ou grupo de fenômenos que se repetem em forma, comportamento, frequência, etc. Entretanto, no contexto de ML, o padrão tem outro significado. Vamos entender o padrão e outros conceitos que permearão todo o curso.

Um padrão é uma descrição quantitativa ou estrutural de um objeto que permite identificar e reconhecer características recorrentes dentro de um conjunto de dados. O que antes poderíamos chamar de padrão, na verdade, chamamos de similaridade: explicando melhor, quando observamos algo que se repete de forma parecida, vemos uma similaridade de padrões; isso porque os padrões, no contexto de ML, são agrupamentos de valores de características dos objetos dessa observação; e quando dizemos que há uma similaridade entre esses padrões quer dizer que os valores das características desses objetos tendem a ser próximos.

Abaixo listamos situações em que podemos observar similaridade de valores e/ou comportamentos:

- » **Numérico:** Sequências de números que seguem uma determinada regra, como a sequência de Fibonacci.
- » **Visual:** Formas ou cores recorrentes em imagens, como a estrutura de folhas de uma planta ou características faciais em fotos.
- » **Comportamental:** Ações ou eventos que ocorrem repetidamente, como hábitos de compra de consumidores.

3.2.1 Arranjos de Padrões

Em todas essas situações descritas nos exemplos anteriores, as observações podem ser representadas por meio de um conjunto de valores, organizados em arranjos de padrões (Bishop, 2006), ou seja, estruturas de dados com valores de características (Theodoridis & Koutroumbas, 2009). Uma vez representadas em forma de arranjos de padrões, essas observações podem ser categorizadas em classes de padrões conforme suas propriedades em comum. E esse é um dos objetivos do reconhecimento de padrões: agrupar observações similares, baseado no valor de suas características.

Na prática, os padrões podem ser organizados de várias maneiras, dependendo das características e da natureza dos dados. Dois dos principais arranjos de padrões são vetores e descrições estruturais como cadeias e árvores:

Vetores são descrições quantitativas dos objetos, representando os dados em uma forma numérica e ordem fixa. Possuem 2 características principais:

- » **Dimensões:** cada vetor possui várias dimensões ou atributos, que são as características mensuráveis do objeto.

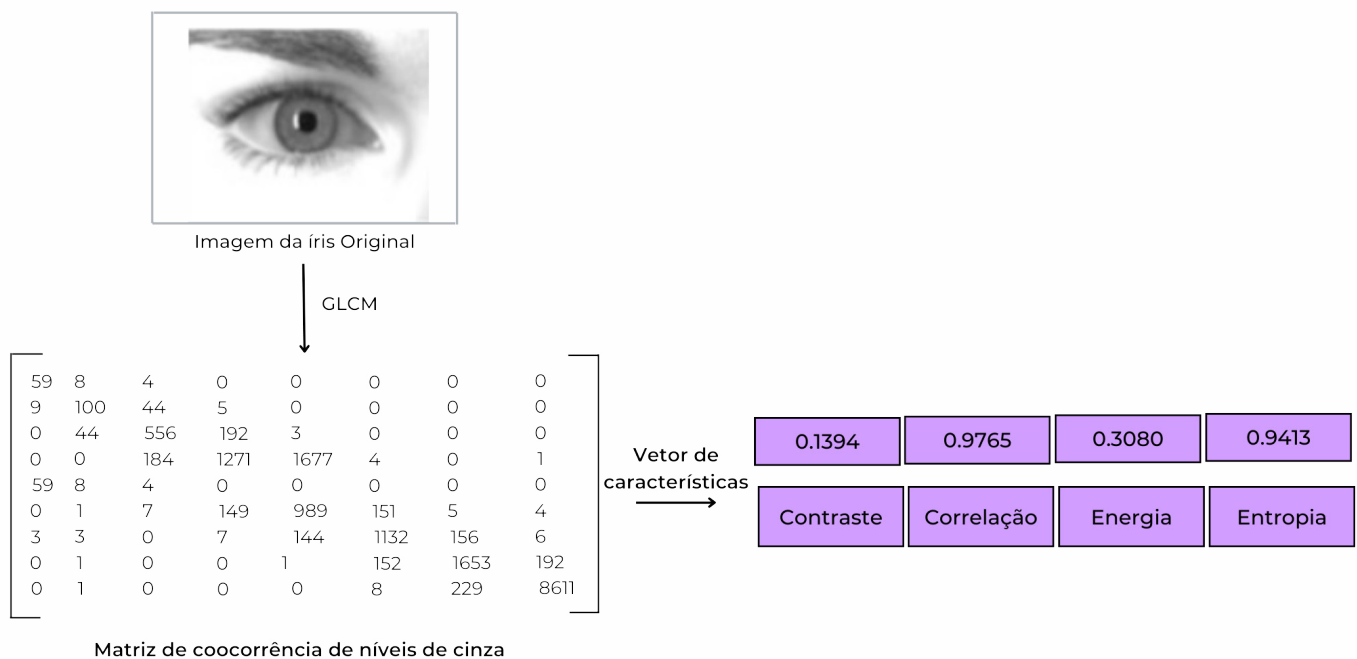
- » **Espaço vetorial:** os vetores são representados em um espaço multidimensional onde cada dimensão corresponde a um atributo.

Abaixo listamos alguns exemplos de tipos de dados e suas respectivas formas de representação em forma de vetor:

- » **Imagem:** um vetor pode representar os valores de intensidade de pixel de uma imagem.
- » **Texto:** um vetor de frequência de palavras, onde cada dimensão representa a contagem de uma palavra específica no texto.
- » **Áudio:** um vetor de características como frequência, amplitude e duração.

A Figura 6 ilustra o processo de criação de um vetor de 4 características a partir de uma imagem de uma íris.

Figura 6 - Extração de características de uma imagem de íris

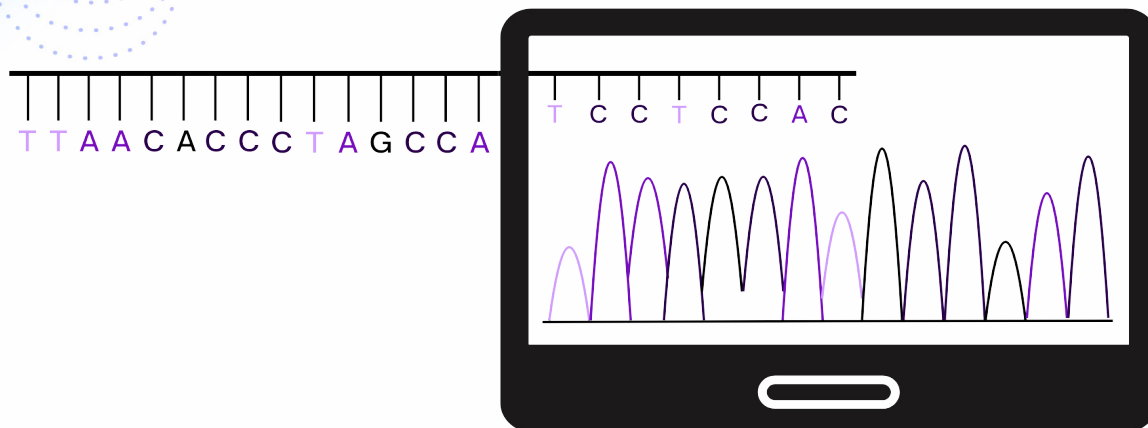


Fonte: adaptada de [Khade et al. \(2021\)](#).

Cadeias e árvores são descrições estruturais que capturam a relação entre diferentes partes dos dados. Cadeias são sequências lineares de elementos conectados de forma ordenada, por exemplo:

- » **Texto:** cadeias de caracteres ou palavras em uma frase.
- » **DNA:** sequências de nucleotídeos (Figura7).

Figura 7 - Sequência de ácido desoxirribonucleico (DNA)



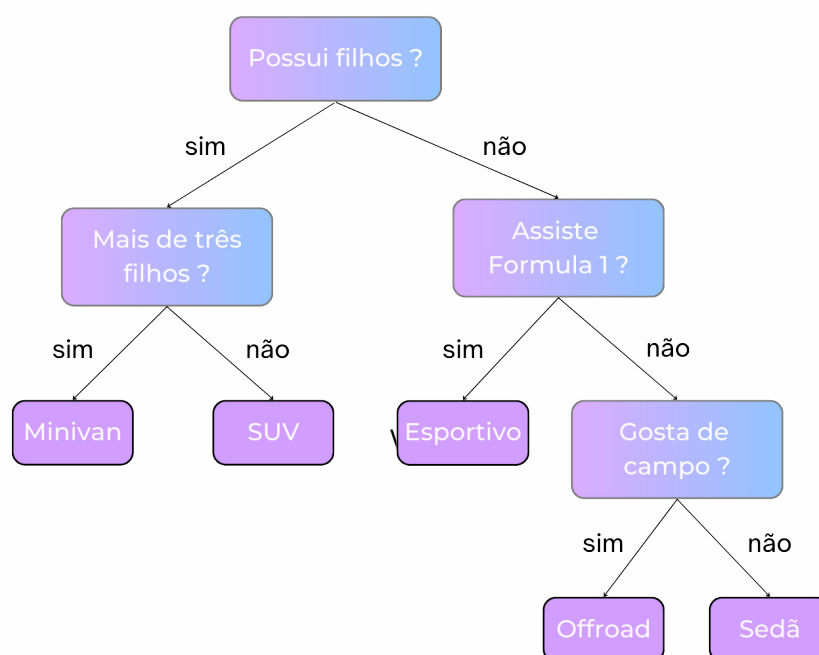
Fonte: adaptada de [Mendelikes \(2023\)](#).

Árvores são estruturas hierárquicas onde os dados são organizados em nós, com um nó raiz e ramificações para outros nós, por exemplo:

- » **Árvores de decisão:** utilizadas em algoritmos de classificação e regressão.
- » **Estruturas de dados:** árvores binárias utilizadas em bancos de dados e algoritmos de busca.

A Figura 8 ilustra um algoritmo de recomendação de tipo de carro baseado na composição familiar e preferências do usuário.

Figura 8 - Árvore de decisão para escolha de um carro



Fonte: adaptada de [Araujo \(2020\)](#).

3.2.2 Objetos e Características

Note que usamos o termo “observações” para se referir aos exemplos dados nas seções. Entretanto, vamos introduzir o termo usual: objeto. Um objeto em reconhecimento de padrões é uma entidade ou unidade de dados que possui características específicas (Mitchell, 1997) que podem ser analisadas para identificar regularidades. Esses objetos podem ser imagens, textos, sons, ou qualquer outra forma de dados estruturados ou não estruturados.

Explicando melhor, as características são propriedades mensuráveis ou categóricas de um objeto (Mitchell, 1997), como cor, forma, intensidade de pixel (em imagens), palavras ou frases (em textos), frequência e amplitude (em sinais de áudio). E quando falamos de classes (de padrões), estamos nos referindo a um grupo de objetos ou dados que possuem características ou atributos semelhantes. Esses padrões são agrupados porque compartilham propriedades que os tornam similares entre si. A seguir listamos alguns exemplos dessas propriedades nos diferentes domínios de imagem, texto e áudio:

» Imagem:

- » **Cor:** representada por valores de cores *Red*, *Green* and *Blue* (RGB), *Hue*, *Saturation* and *Value* (HSV) e etc. que indicam as cores vermelho, verde e azul e a tonalidade, saturação e brilho.
- » **Forma:** contornos e geometria que definem a estrutura de objetos, como círculos, quadrados ou polígonos.
- » **Intensidade de Pixel:** valores de brilho ou intensidade em cada pixel da imagem, que podem ser usados para detectar contrastes e detalhes.
- » **Textura:** padrões e variações na intensidade de pixel que indicam a superfície e a aparência de um objeto.

» Texto:

- » **Palavras:** conjuntos de letras ou caracteres que formam unidades de significado.
- » **Frases:** agrupamentos de palavras que constroem sentenças com significado completo.
- » **Repetições:** palavras ou termos que aparecem repetidamente em um texto, úteis para análise de tópicos ou sentimentos.

» Áudio:

- » **Frequência:** número de ciclos por segundo de uma onda sonora, medida em Hertz (Hz), essencial para identificar notas musicais e timbres.

- » **Amplitude:** intensidade ou volume da onda sonora, que determina a percepção de volume e potência do som.
- » **Tempo:** duração dos sons ou intervalos entre eventos sonoros, relevante para a análise de ritmo e tempo.

3.2.3 Classes

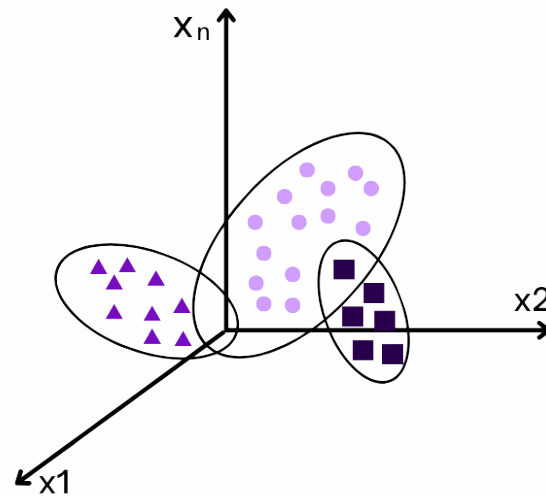
Uma classe é um rótulo ou categoria que define um grupo de objetos ou instâncias que compartilham características ou atributos comuns. As classes são usadas principalmente em tarefas de classificação, onde o objetivo é atribuir um rótulo de classe a uma nova instância com base em suas características.

Quando pretendemos determinar quais objetos pertencem à mesma classe, o compartilhamento dessas propriedades, características ou atributos similares muda conforme contexto que o algoritmo está atuando, por exemplo:

- » **Imagens de gatos:** têm propriedades comuns como orelhas pontudas, olhos grandes e pelos macios.
- » **Textos sobre tecnologia:** compartilham termos e jargões técnicos, como “algoritmo”, IA e ML.
- » **Músicas:** podem ter frequências, ritmos e estruturas semelhantes.

Na Figura 9, utilizamos um exemplo de representação dos objetos em um espaço vetorial e podemos entender como encontramos similaridade entre objetos baseado na proximidade de suas características. É possível identificar que se trata de um plano 3 Dimensões (3D): isso significa que cada objeto é representado por um vetor de características com 3 valores quaisquer. Também temos 3 agrupamentos de objetos e dizemos que estes pertencem a uma das classes: quadrado, triângulo ou círculo. Em cada uma dessas classes, os objetos estão mais próximos entre si do que de objetos de outras classes. Como esses objetos são representados por vetores com 3 valores, cada um deles indica um ponto no plano 3D. A partir desses pontos podemos calcular a distância entre eles e determinar quais estão mais próximos e, portanto, pertencem à mesma classe. Essa é uma das muitas formas de se reconhecer padrões e veremos algumas delas durante o curso. O objetivo de mostrar esse cenário é apresentar conceitos importantes que são aplicados em quaisquer algoritmos de ML. Vamos lembrá-los:

Figura 9 - Disposição de objetos em um espaço vetorial de três dimensões

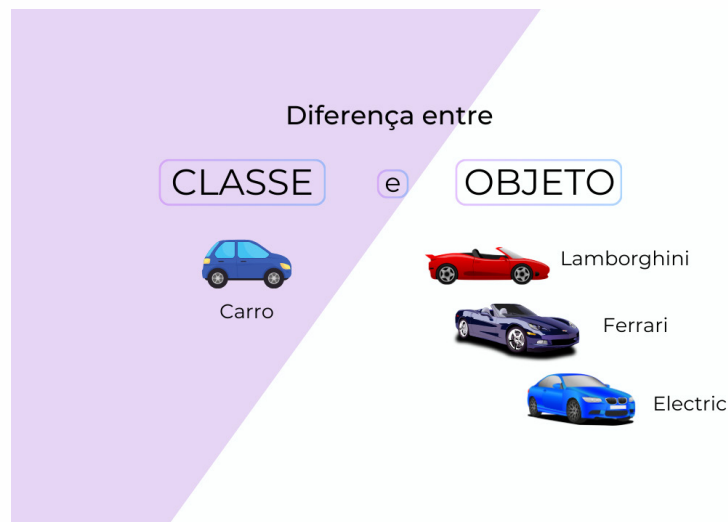


Fonte: adaptada de [Lu, Wu e Kita \(2014\)](#).

- » **Objetos:** entidades ou instâncias de dados que possuem propriedades específicas e podem ser definidos em diferentes contextos, como imagens, textos, áudios, entre outros.
- » **Características:** propriedades ou atributos mensuráveis que descrevem os objetos e permitem diferenciá-los uns dos outros.
- » **Classes:** grupos ou conjuntos de objetos que compartilham um conjunto comum de características ou atributos. Representam categorias ou tipos específicos de objetos.

A Figura 10 exemplifica o caso de vários objetos de uma mesma classe Carro. Apesar de possuírem diferenças entre si, compartilham de um mesmo conjunto de características como quantidade de rodas, presença de farol, capacidade de passageiros, velocidade máxima, entre outras, mesmo com valores diferentes para cada uma dessas características.

Figura 10 - Imagem de classe, objeto e característica



Fonte: adaptada de [Scaler \(2024\)](#).

Abaixo, na Tabela 2, listamos alguns exemplos de classes e objetos. Esses exemplos ilustram como objetos diferentes podem compartilhar características semelhantes que os agrupam na mesma classe.

Tabela 2 - Exemplos de classes e objetos pertencentes a elas

Tema	Objetos	Características similares	Classe
Imagens de veículos	Um carro compacto, um sedã, e um SUV	Rodas, portas, janelas, faróis.	Veículos
Textos de notícias	Artigos sobre uma conferência internacional, uma reunião da ONU e um discurso de um chefe de estado.	Uso de termos como "diplomacia", "política internacional", "acordos".	Política internacional
Sons de instrumentos musicais	Som de um piano, um teclado eletrônico e um cravo	Frequências harmônicas, timbre percussivo	Instrumentos de tecla
Produtos eletrônicos	Um smartphone, um tablet e um e-reader	Tela sensível ao toque, conectividade à internet, bateria recarregável	Dispositivos móveis
Animais domésticos	Um gato persa, um gato siamês e um gato de rua	Bigodes, patas, comportamento felino	Gatos
Vegetais de folhas verdes	Espinafre, alface e couve	Folhas verdes, ricos em nutrientes	Vegetais de folhas verdes
Livros de ficção científica	"Duna" de Frank Herbert, "Fundação" de Isaac Asimov, e "Neuromancer" de William Gibson	Ambientação futurista, elementos de tecnologia avançada, exploração de temas científicos	Ficção científica
Aplicativos de redes sociais	Facebook, Instagram e Twitter	Perfis de usuários, compartilhamento de conteúdo, interações sociais	Redes sociais
Fotografias de paisagens naturais	Uma foto de uma montanha, uma floresta e uma praia	Elementos naturais, beleza cênica, ausência de construções humanas	Paisagens naturais
Documentos acadêmicos	Uma tese de mestrado, uma dissertação de doutorado e um artigo de conferência	Estrutura formal, revisão por pares, contribuições científicas	Publicações acadêmicas

Fonte: autoria própria.

Para que ocorra o reconhecimento de padrões com o objetivo de agrupar ou categorizar esses objetos, são necessárias algumas etapas a serem seguidas descritas na Figura 11:

- » **Pré-processamento:** preparação dos objetos para análise, como normalização e remoção de ruído (conceitos que discutiremos adiante), e extração de características relevantes.
- » **Seleção de características:** Identificação das propriedades mais significativas dos objetos que ajudarão na análise e reconhecimento.
- » **Classificação e agrupamento:** Uso de algoritmos para classificar ou agrupar os objetos com base nas características extraídas.

Figura 11 - Processo de reconhecimento de padrões em *machine learning*



Fonte: adaptada de [Marquês de Sá, 2000](#).

Então, resumindo, até agora aprendemos que, em ML, trabalhamos técnicas de reconhecimento de padrões, ou seja, podemos categorizar ou agrupar objetos em classes por meio da similaridade de suas características representadas em vetores de dados (padrões). Falaremos com mais detalhes sobre os tipos de algoritmos de aprendizagem de máquina e sobre o espaço vetorial, também conhecido como espaço de características nas próximas Unidades.



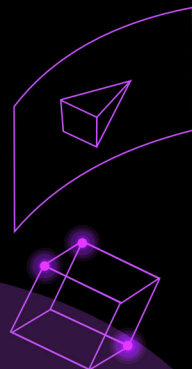
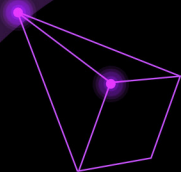
PARA RELEMBRAR...

- » O reconhecimento em ML refere-se à habilidade de identificar e categorizar objetos ou padrões com base em características previamente aprendidas. Este processo envolve a percepção de semelhanças entre os novos dados e os modelos já treinados, utilizando inferência baseada em conceitos aprendidos no passado. Em termos práticos, isso significa que o sistema de reconhecimento pode identificar rostos em imagens, detectar palavras em áudios ou classificar textos em categorias específicas, dependendo dos dados de treinamento e das características extraídas.
- » Além disso, discutimos a importância das propriedades ou características mensuráveis dos objetos, como cor, forma e intensidade de pixel em imagens, palavras ou frases em textos, e frequência e amplitude em sinais de áudio. Esses atributos são essenciais para a análise e classificação de dados. Também abordamos os principais arranjos de padrões utilizados na prática, como vetores para descrições quantitativas e cadeias e árvores para descrições estruturais.

SAIBA MAIS...

- » [Uma referência clássica para fundamentos em reconhecimento de padrões e aprendizado de máquinas.](#)
- » Site da Google DeepMind, que publica pesquisas inovadoras e artigos sobre avanços em IA, incluindo reconhecimento de padrões e previsão.

Unidade IV
**Tipos de
aprendizagem
de máquina**





Unidade IV - Tipos de Aprendizagem de Máquina

Nesta Unidade, vamos explorar os principais tipos de aprendizagem de máquina: supervisionada, não supervisionada, semi-supervisionada e por reforço. Vamos explicar cada um deles, utilizando metáforas para facilitar a compreensão, e apresentaremos exemplos para cada tipo.

No vasto campo da aprendizagem de máquina, conhecer os diferentes tipos de aprendizagem é crucial para aplicar a abordagem mais adequada a cada situação. Compreender as nuances entre a aprendizagem supervisionada, não supervisionada, semi-supervisionada e por reforço permite que você escolha a técnica que melhor se adapta aos dados disponíveis e ao problema que deseja resolver. Cada método possui suas próprias vantagens e limitações, e a escolha correta pode significar a diferença entre o sucesso e o fracasso de um projeto. Portanto, dominar esses conceitos não apenas aprimora sua capacidade de desenvolver soluções eficazes, mas também otimiza o uso dos recursos disponíveis, tornando suas análises e previsões mais precisas e eficientes.

Os diferentes tipos de aprendizagem de máquina possuem processos de treinamento distintos, refletindo suas abordagens únicas para lidar com dados. Vamos antecipar a diferença entre eles e discutiremos com mais detalhes cada um adiante.

Na aprendizagem supervisionada, o treinamento envolve dados rotulados que orientam o modelo para fazer previsões precisas. A aprendizagem não supervisionada, por outro lado, busca descobrir padrões ocultos em dados não rotulados sem orientação explícita. A aprendizagem semi-supervisionada combina ambos os métodos, utilizando um pequeno conjunto de dados rotulados e um grande conjunto de dados não rotulados para melhorar a precisão. Já a aprendizagem por reforço envolve a interação contínua com um ambiente, onde o modelo aprende através de recompensas e punições. Cada tipo de aprendizagem adapta-se a diferentes cenários e desafios, destacando a importância de entender suas características e aplicações específicas. No mapa mental apresentado na Figura 12, esses tipos de aprendizado são sintetizados.

Figura 12 - Tipos de aprendizagem de máquina



Fonte: autoria própria.

4.1 Aprendizagem Supervisionada

Na aprendizagem supervisionada, o modelo é treinado com um conjunto de dados rotulados, ou seja, cada entrada possui uma saída correspondente. Em outras palavras, cada objeto utilizado no treinamento possui a informação sobre qual classe ele pertence. O objetivo é fazer com que o modelo aprenda a mapear entradas (arranjos de padrões) para saídas corretas (classes), permitindo que ele faça previsões sobre novos dados.

Para entender melhor, imagine que você está ensinando uma criança a reconhecer frutas. Você mostra a ela várias frutas (entradas) e diz o nome de cada uma (saídas). Com o tempo, a criança aprende a associar a aparência da fruta ao seu nome. Se você mostrar uma maçã vermelha ou uma verde, a criança dirá “maçã” pois ambas compartilham de características que as tornam similares entre si, mesmo sendo de variedades distintas, e as diferencia de outras frutas.

De forma simplificada, o processo de treinamento de um modelo utilizando aprendizagem supervisionada pode ser descrito em 4 passos na Figura 13.

Figura 13 - Treinamento de um algoritmo de aprendizagem supervisionada



Fonte: autoria própria.

A classificação de emails como spam ou não-spam é um outro exemplo clássico de um problema de aprendizagem supervisionada. O objetivo é criar um modelo de ML que possa classificar automaticamente emails recebidos em duas classes ou categorias: **spam** (e-mail indesejado) e **não-spam** (e-mail legítimo):

- » **Dados de entrada:** texto dos emails.
- » **Dados de saída:** rótulos indicando se o email é spam ou não-spam.
- » **Modelo:** algoritmo de classificação ex.: *Naive Bayes* e *Support Vector Machines* (SVM).

4.2 Aprendizagem Não Supervisionada

Na aprendizagem não supervisionada não há uma medida de resultado (Hastie *et al.*, 2009), o modelo trabalha com dados não rotulados. O objetivo é encontrar padrões ou agrupamentos nos dados sem orientação explícita sobre o que procurar.

Na aprendizagem não supervisionada, não há uma medida de resultado, e o objetivo é descrever as associações e padrões entre um conjunto de medidas de entrada.

Pense em um biólogo em trabalho de campo para identificar espécies nativas. Ele não tem um mapa (rótulos), mas pode observar as árvores e plantas ao seu redor. Com o tempo, ele percebe que certas áreas da floresta têm árvores semelhantes e as agrupa mentalmente.

Com exceção dos dados não rotulados, as etapas para treinar um modelo utilizando um algoritmo de aprendizagem não supervisionada, em linhas gerais, são as mesmas do aprendizado supervisionado (Figura 14).

Figura 14 - Treinamento de um algoritmo de aprendizagem não supervisionada



Fonte: autoria própria.

Imagine que uma empresa deseja entender melhor seus clientes para oferecer produtos e serviços mais direcionados. Para isso, quer agrupar os clientes em segmentos de mercado distintos com base em suas características e comportamentos. Não se sabe quantos ou quais serão os segmentos de mercado. Esse tipo de problema é conhecido como agrupamento (ou *clustering*). Essa abordagem permite à empresa entender melhor seus clientes, resultando em estratégias mais eficazes e aumento da satisfação e lealdade do cliente:

- » **Dados de entrada:** informações sobre clientes (ex.: idade, renda, histórico de compras).
- » **Dados de saída:** agrupamentos de clientes com características semelhantes.
- » **Modelo:** algoritmo de *clustering* (ex.: *K-means* e *Density-Based Spatial Clustering of Applications with Noise* [DBSCAN]).

4.3 Aprendizagem Semi-supervisionada

A aprendizagem semi-supervisionada é um meio-termo entre a aprendizagem supervisionada e a não supervisionada (Chapelle *et al.*, 2006). Ela utiliza uma pequena quantidade de dados rotulados e uma grande quantidade de dados não rotulados. O modelo aproveita os dados rotulados para aprender e, em seguida, refinar seu conhecimento com os dados não rotulados.

Imagine um estudante que recebe algumas respostas corretas de um professor (dados rotulados) e, em seguida, tenta completar o restante da tarefa por conta própria usando essas respostas como referência.

Nas etapas de treinamento, incluímos o treinamento inicial e rodadas de refinamento para melhorar a acurácia do modelo (Figura 15).

Figura 15 - Treinamento de um algoritmo de aprendizagem semi-supervisionada



Fonte: adaptada de [Vecteezy \(2024\)](#).

Agora, veja a seguinte situação: imagine que temos uma coleção de documentos de texto de um fórum *online* onde os usuários discutem uma ampla variedade de tópicos. Queremos classificar esses documentos em categorias específicas, como “Tecnologia”, “Saúde”, “Educação”, etc. No entanto, só temos rótulos para um pequeno subconjunto dos documentos (talvez apenas 10% do total), enquanto a maioria dos documentos não está rotulada. A quantidade de documentos rotulados é pequena, o que torna difícil treinar um modelo de classificação tradicional com bom desempenho. Temos uma grande quantidade de documentos não rotulados que, se utilizados corretamente, podem melhorar a performance do modelo. Entretanto, a variação nos tópicos discutidos nos documentos pode ser ampla, tornando a classificação mais complexa.

- » **Dados de entrada:** texto dos documentos.
- » **Dados de saída:** etiquetas indicando a categoria dos documentos (rotulados).
- » **Modelo:** algoritmo semi-supervisionado (ex.: *Semi-Supervised SVM*).

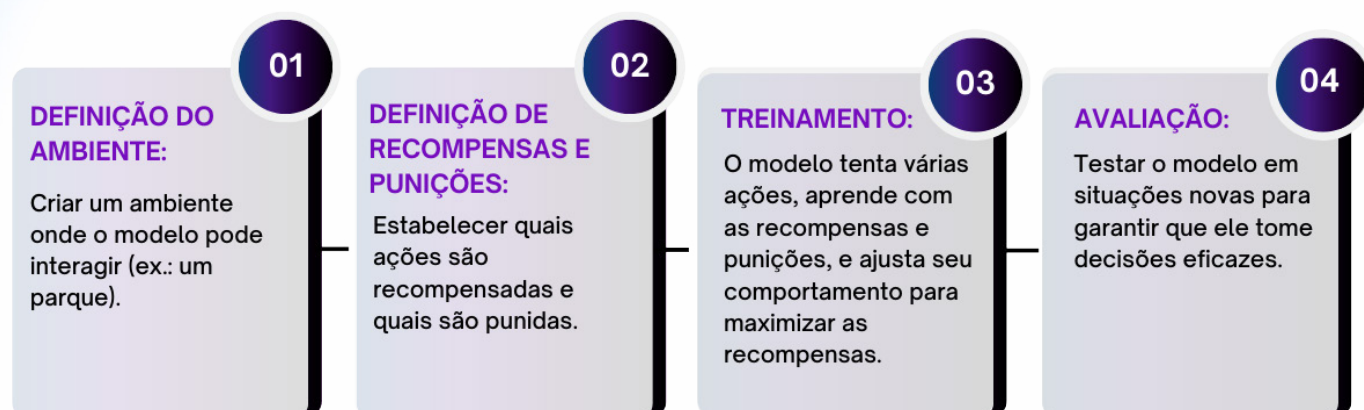
4.4 Aprendizagem por Reforço

Na aprendizagem por reforço, o modelo aprende a tomar decisões ao interagir com um ambiente. Ele recebe recompensas ou punições com base nas ações que realiza, e o objetivo é maximizar a recompensa total ao longo do tempo. Em outras palavras, problemas de aprendizagem por reforço envolvem aprender o que fazer - como mapear situações para ações - de modo a maximizar um sinal de recompensa numérica (Hastie *et al.*, 2009). É muito utilizado em jogos, robótica e sistemas autônomos.

Pense que você deseja treinar um cão a buscar uma bola. Quando o cão traz a bola de volta, você o recompensa com um petisco (recompensa). Se ele corre na direção errada, você não entrega o petisco (punição). Com o tempo, o cão aprende a buscar a bola corretamente para ganhar mais petiscos.

As etapas do processo de treinamento no caso de aprendizagem por reforço seguem outra lógica em relação às demais (Figura 16):

Figura 16 - Treinamento de um algoritmo de aprendizagem por reforço



Fonte: autoria própria.

Em um problema de aprendizagem por reforço aplicado a um jogo eletrônico de xadrez, o agente (um programa de IA) aprende a jogar xadrez interagindo com o ambiente (o tabuleiro de xadrez). O objetivo do agente é maximizar uma recompensa ao longo do tempo, que neste caso é ganhar a partida de xadrez. O agente é o jogador de xadrez controlado pelo algoritmo de aprendizagem por reforço. Esse agente deve aprender a tomar decisões sobre qual movimento fazer em cada estado do jogo. O ambiente é o próprio tabuleiro de xadrez e o estado atual do jogo, incluindo a posição de todas as peças e o histórico de movimentos. Cada estado representa uma configuração específica do tabuleiro de xadrez. Por exemplo, a posição inicial das peças no início do jogo é um estado, e qualquer configuração resultante de uma série de movimentos subsequentes também é um estado. As ações são todos os movimentos válidos que o agente pode fazer a partir de um determinado estado. No xadrez, isso inclui movimentar qualquer peça para qualquer posição permitida pelas regras do jogo. Por fim, a recompensa é um valor numérico que o agente recebe após realizar uma ação. No xadrez, uma recompensa positiva pode ser atribuída para capturar uma peça adversária, colocar o rei adversário em xeque-mate, ou ganhar a partida. Recompensas negativas podem ser atribuídas para perder uma peça, colocar o próprio rei em xeque, ou perder a partida:

- » **Dados de entrada:** estado atual do tabuleiro.
- » **Dados de saída:** ação do agente (ex.: movimento de uma peça).
- » **Modelo:** algoritmo de aprendizagem por reforço (ex.: *Q-Learning*, *Deep Q-Network*).



SAIBA MAIS...

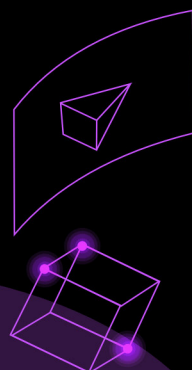
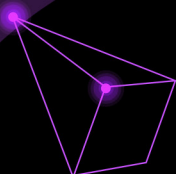
- » [Artigo que revisa várias técnicas de aprendizagem de máquina aplicadas à classificação de documentos de texto.](#)
- » [Livro-texto fundamental para entender os conceitos e aplicações da aprendizagem por reforço.](#)

PARA RELEMBRAR...

- » Nesta Unidade, abordamos os diferentes tipos de aprendizagem: supervisionada, não supervisionada, semi-supervisionada e por reforço. Para cada tipo, fornecemos uma explicação detalhada, metáforas para facilitar a compreensão e exemplos práticos. Discutimos também como os dados são manipulados e processados durante o treinamento de cada tipo de aprendizagem.
- » Também destacamos a importância de conhecer as diferentes abordagens de aprendizagem de máquina para escolher a melhor técnica para cada situação específica. Enfatizamos que cada tipo de aprendizagem possui um processo de treinamento único, que varia desde o uso de dados rotulados na aprendizagem supervisionada até a interação contínua com um ambiente na aprendizagem por reforço. Compreender essas nuances é essencial para desenvolver soluções eficazes e precisas em ML, otimizando assim os recursos disponíveis e garantindo melhores resultados nos projetos.

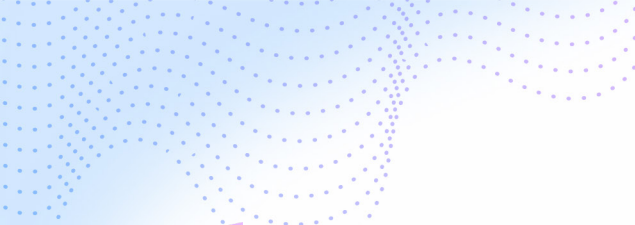
Unidade V

Espaço de características





Unidade V - Espaço de Características



5.1 Extração de Características

A extração de características é uma etapa fundamental no processo de aprendizado de máquina, onde informações relevantes são derivadas dos dados brutos. Este processo visa transformar dados complexos e volumosos em um conjunto de características mais manejável e significativo para a modelagem. Por exemplo, em imagens, características como bordas, texturas e formas podem ser extraídas para facilitar a detecção ou o reconhecimento.

As características extraídas devem capturar a essência dos dados, preservando as informações críticas necessárias para as tarefas de análise subsequentes. Técnicas como Histogramas de Gradientes Orientados (HOG) para imagens, *Term Frequency-Inverse Document Frequency* (TF-IDF) para textos, e *Mel-Frequency Cepstral Coefficients* (MFCC) para áudios são exemplos comuns de métodos de extração de características. Cada técnica é escolhida com base na natureza dos dados e nos requisitos específicos da aplicação.

A extração manual de características é um processo no qual um especialista analisa os dados brutos para identificar e definir características relevantes que serão utilizadas em modelos de ML. Este processo exige um conhecimento profundo do domínio específico e das propriedades dos dados, permitindo ao especialista selecionar atributos significativos que capturam as informações mais importantes. Essas características, uma vez extraídas, são organizadas em um formato estruturado, facilitando a análise e o treinamento de algoritmos de aprendizado. A extração manual de características é essencial em muitos campos, pois garante que os modelos sejam treinados com base em dados bem definidos e altamente relevantes, aumentando assim a eficácia e a precisão das previsões. Falaremos mais sobre conjuntos de dados adiante.

Além disso, a extração de características pode ser automatizada utilizando DL. Em aplicações de DL, camadas convolucionais de Rede Neural Convolucional (CNN) extraem automaticamente características hierárquicas de imagens, enquanto Rede Neural Recorrente (RNN) ou *Transformers* (transformadores) podem capturar dependências temporais em dados sequenciais.

Não existe receita pronta: cada contexto deve ser criteriosamente estudado para se definir quais características serão extraídas manualmente pelo especialista ou qual método será empregado para a seleção automática, pois essas escolhas são cruciais para o desempenho do modelo de ML.

5.2 Seleção de características

A seleção de características é o processo realizado após a extração das características e tem como objetivo identificar e escolher um subconjunto das características mais relevantes para a construção de um modelo de ML (Guyon & Elisseeff, 2003). Esse processo é essencial para melhorar a eficiência do modelo, reduzir a complexidade computacional e prevenir o (sobreajuste) (falaremos sobre isso adiante). De forma direta, podemos entender que técnicas de seleção de características ajudam a eliminar dados redundantes ou irrelevantes que não contribuem para a precisão do modelo.

Existem vários métodos de seleção de características, incluindo métodos de filtro, *wrapper*¹ e baseados em modelos. Métodos de filtro utilizam estatísticas para avaliar a relevância de cada característica de forma independente do modelo. Exemplos incluem correlação, qui-quadrado e testes de informação mútua. Métodos de *wrapper*, como *Recursive Feature Elimination* (RFE), avaliam combinações de características utilizando o desempenho do modelo. Já os métodos baseados em modelos, como *Least Absolute Shrinkage and Selection Operator* (LASSO)², incorporam a seleção de características diretamente no processo de treinamento do modelo.

Ao eliminar características irrelevantes ou redundantes, a seleção de características simplifica o modelo, reduzindo o número de parâmetros que precisam ser ajustados durante o treinamento. Isso resulta em algoritmos mais rápidos, pois há menos dados a serem processados. Modelos simplificados necessitam de menos iterações para convergir durante o treinamento, acelerando o processo de aprendizado. Isso é particularmente vantajoso em algoritmos de otimização que dependem de múltiplas iterações, como gradiente descendente. Além disso, com menos características (mantendo, é claro, as mais relevantes), o tempo necessário para a realização dos cálculos durante o treinamento é significativamente reduzido. Isso é crucial em aplicações práticas onde o tempo de processamento é uma limitação.

Outra vantagem que determina a importância do processo de seleção de características é a redução da demanda de recursos computacionais. Menos características significam que menos dados precisam ser armazenados na memória durante o processamento. Isso é particularmente importante em sistemas com recursos limitados, como dispositivos embarcados ou aplicações móveis. A quantidade de operações matemáticas necessárias para processar os dados é reduzida, diminuindo a carga computacional e permitindo que modelos sejam treinados e executados em hardwares menos potentes. Por fim, a seleção de características facilita a escalabilidade dos modelos para grandes conjuntos de dados. Ao trabalhar com um subconjunto mais gerenciável de características, torna-se mais fácil lidar com volumes massivos de dados sem sobrecarregar os recursos computacionais.

¹ *Wrapper* é uma abordagem para a seleção de características em *machine learning* onde a eficácia de diferentes subconjuntos de características é avaliada diretamente pelo desempenho do modelo de aprendizado. Em vez de utilizar medidas estatísticas simples para selecionar características, o método *wrapper* usa o próprio algoritmo de aprendizado para determinar quais características são mais relevantes. Isso é feito testando iterativamente diferentes combinações de características e observando como a inclusão ou exclusão de cada uma afeta o desempenho do modelo.

² *LASSO* é uma técnica de regularização e seleção de características que adiciona uma penalidade baseada na soma dos valores absolutos dos coeficientes de regressão. Essa penalidade força a redução de alguns coeficientes a zero, efetivamente selecionando um subconjunto de características durante o treinamento do modelo.

A seleção adequada de características pode levar a modelos mais interpretáveis e eficientes, facilitando a compreensão dos dados e a tomada de decisões. Também pode ajudar a reduzir a dimensionalidade dos dados, abordando desafios associados a conjuntos de dados de alta dimensão.

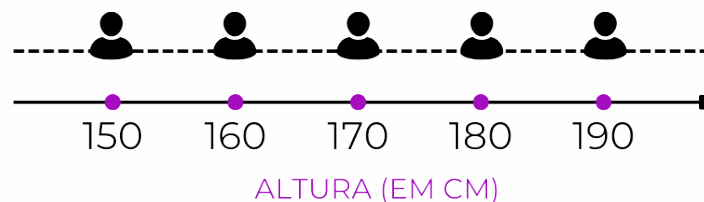
5.3 Características vs. Dimensionalidade

A relação entre características e dimensionalidade é um aspecto crucial em ML. Cada característica adicionada a um conjunto de dados aumenta sua dimensionalidade, o que pode ter implicações significativas no desempenho e na complexidade do modelo. Embora a inclusão de mais características possa fornecer mais informações ao modelo, também pode levar a desafios como a maldição da dimensionalidade (abordaremos essa questão adiante). Vamos utilizar alguns exemplos para entendermos melhor como é a relação característica vs. dimensionalidade.

a) Uma dimensão (1D ou unidimensional):

- » Imagine que temos uma característica, como a altura de uma pessoa.
- » Cada pessoa pode ser representada como um ponto em uma linha, onde a posição na linha indica a altura.

Figura 17 - Dados unidimensionais sobre altura (em cm)

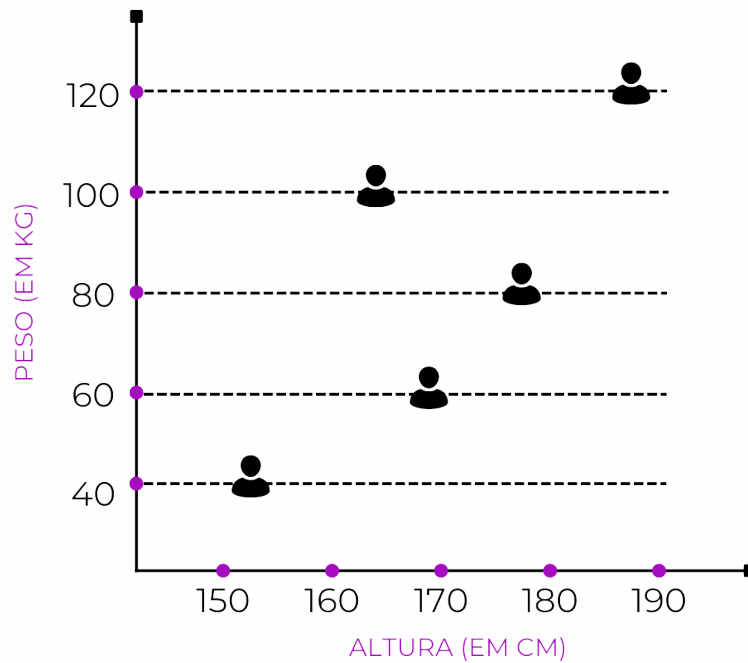


Fonte: autoria própria.

b) Duas dimensões (2D ou bidimensional):

- » Agora, adicionamos uma segunda característica, como o peso.
- » Cada pessoa agora é representada como um ponto em um plano bidimensional, onde a posição é determinada pela altura e pelo peso.

Figura 18 - Dados bidimensionais que relacionam peso (em kg) e altura (em cm) de cinco pessoas



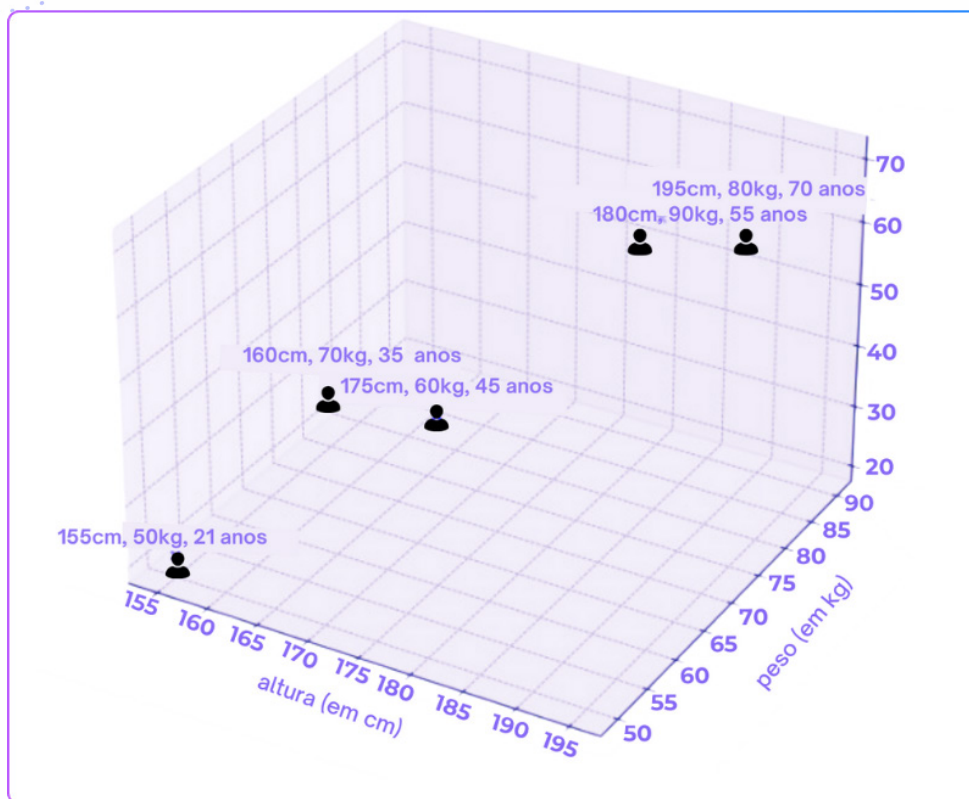
Fonte: autoria própria.

Cada ponto no gráfico da Figura 18 representa uma pessoa com uma combinação respectiva de altura e peso

c) Três dimensões (3D ou tridimensional):

- » Adicionando uma terceira característica, como a idade, movemos para um espaço tridimensional.
- » Cada pessoa agora é representada como um ponto em um espaço 3D, com altura, peso e idade determinando sua posição. Observe o gráfico da Figura 19 a seguir.

Figura 19 - Dados tridimensionais que relacionam peso (em kg), altura (em cm) e idade (em anos) de cinco pessoas



Fonte: autoria própria.

Cada nova característica adicionada representa uma nova dimensão no espaço dos dados. Assim:

- » 1 característica = 1 dimensão (linha);
- » 2 características = 2 dimensões (plano);
- » 3 características = 3 dimensões (espaço 3D);
- » N características = N dimensões (hiperespaço).

Em espaços de alta dimensão (hiperespaços), visualizar os dados se torna impossível, mas a matemática e os algoritmos de ML ainda podem operar nesses espaços. Cada ponto de dado em um espaço N-dimensional representa uma instância com N características, e os algoritmos de ML utilizam essas coordenadas para encontrar padrões, realizar classificações e fazer previsões.

Por exemplo, se adicionarmos uma nova característica como a “pressão arterial” ao nosso exemplo de pessoas, estaríamos aumentando a dimensionalidade do nosso espaço de dados para quatro dimensões (4D) (altura, peso, idade, pressão arterial). Embora não possamos visualizar um espaço 4D, a adição dessa característica fornece mais informações que podem ser úteis para melhorar a precisão do modelo de ML.

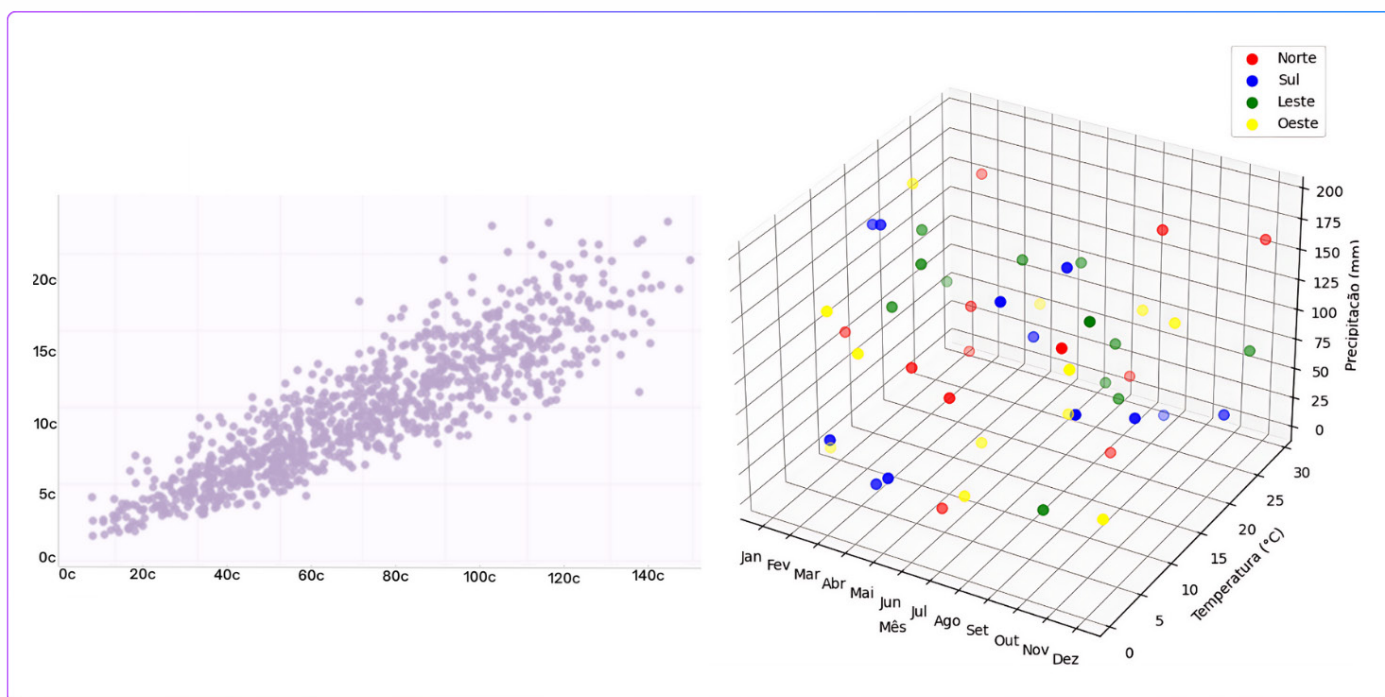
Em um espaço de alta dimensão, a densidade dos dados tende a diminuir, tornando mais difícil encontrar padrões significativos. Isso ocorre porque, à medida que a dimensionalidade aumenta, o volume do espaço de características cresce exponencialmente, e os dados se tornam esparsos. Consequentemente, os algoritmos de ML podem ter dificuldades para generalizar a partir dos dados de treinamento, resultando em *overfitting* (sobreajuste).

Portanto, equilibrar o número de características é fundamental. A seleção cuidadosa de características relevantes e a redução da dimensionalidade são práticas importantes para manter um conjunto de dados manejável e para garantir que o modelo seja capaz de aprender e generalizar de forma eficaz.

5.4 Maldição da Dimensionalidade

A maldição da dimensionalidade (Bellman, 1961) refere-se aos vários fenômenos que surgem quando se trabalha com dados em espaços de alta dimensão. À medida que a dimensionalidade dos dados aumenta, o volume do espaço aumenta de tal forma que os pontos de dados se tornam esparsos. Em consequência, muitas das intuições e técnicas que funcionam bem em espaços de baixa dimensão deixam de ser aplicáveis. Na Figura 20, há 2 gráficos representando um plano bidimensional denso e um plano tridimensional esparsos. Veja que, no gráfico da Figura 22, os objetos estão distantes entre si de forma generalizada, o que dificulta encontrar similaridades.

Figura 20 - Espaços de características bidimensional denso (à esquerda) e tridimensional esparsos (à direita)



Fonte: adaptada de [Pragmatiq \(2023\)](#).

Uma das principais dificuldades associadas à maldição da dimensionalidade é que a distância entre os pontos de dados tende a se tornar uniforme, dificultando a distinção entre eles. Isso pode prejudicar a eficácia de algoritmos de aprendizado de máquina baseados em distância, como kNN (*k-Nearest Neighbors*) e de *clustering* (agrupamento).

Para mitigar os efeitos da maldição da dimensionalidade, técnicas de redução de dimensionalidade (aprofundaremos neste assunto adiante), como PCA e t-Distributed Stochastic Neighbor Embedding (t-SNE), são frequentemente utilizadas. Essas técnicas ajudam a projetar os dados em um espaço de menor dimensão, preservando as características mais importantes e facilitando a análise e a modelagem.

5.5 Normalização do Espaço de Características

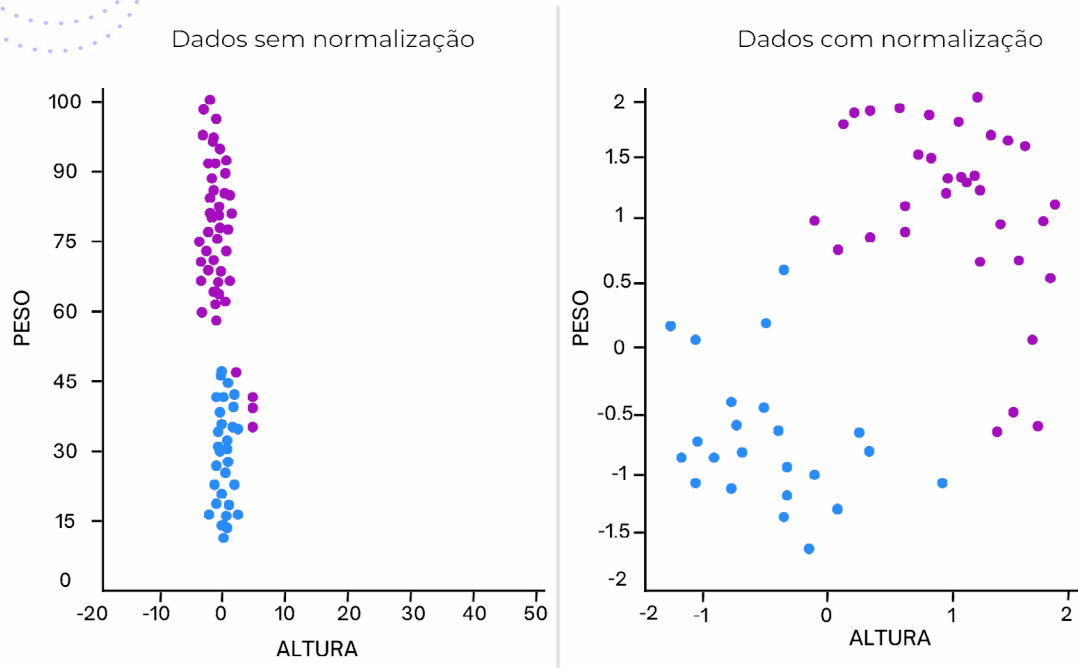
A normalização do espaço de características é uma etapa crucial no pré-processamento de dados em ML. Consiste em ajustar as escalas das características para que todas contribuam igualmente para o modelo, evitando que características com valores maiores dominem o processo de aprendizado. Isso é particularmente importante em algoritmos baseados em distância, como kNN e SVM.

Existem várias técnicas de normalização, como a padronização (*standardization*), que transforma as características para terem média zero e desvio padrão um, e a normalização min-max, que reescala os valores para um intervalo específico, geralmente entre 0 e 1. A escolha da técnica de normalização depende da natureza dos dados e do algoritmo utilizado.

Retomando o exemplo anterior, em que apresentamos um espaço de características bidimensionais com os eixos de altura (em cm) e peso (em kg), se os valores de altura estivessem em metros, a amplitude dessa característica em relação ao peso das pessoas seria bem menor. Isso quer dizer que, sem normalização, as diferenças de escala entre essas características poderiam causar problemas nos algoritmos de aprendizado de máquina. Por exemplo, os valores de peso podem variar entre 50 e 150 kg, enquanto os valores de altura variam entre 1,5 e 2 metros. Esta diferença de escala pode fazer com que a característica de peso domine os cálculos de distância, levando a modelos que não consideram a altura de forma adequada. A normalização transforma os dados para que todas as características tenham a mesma escala, tipicamente ajustando os valores para um intervalo padrão, como 0 a 1, por exemplo. Isso garante que cada característica contribua igualmente para o modelo, melhorando a precisão, a eficiência do treinamento e a capacidade de generalização do modelo para novos dados.

A Figura 21 apresenta um mesmo espaço de características com os objetos representados por círculos. Na Figura com dados sem normalização percebemos que a escala dos dados do eixo de altura (x_1) é bem menor em relação ao peso (x_2). Enquanto na outra, após a normalização dos valores entre -2 e 2, os objetos aparecem mais distribuídos, com a classe 0 (cor azul) e classe 1 (cor vermelha) mais distantes entre si, ao passo que respectivos elementos mantêm a proximidade dentro da mesma classe.

Figura 21 - Espaço de características de peso e altura não normalizado (à esquerda) e normalizado (à direita)



Fonte: adaptada de [Kedarps \(2017\)](#).

Portanto, a normalização melhora a convergência de algoritmos de otimização e pode levar a um melhor desempenho geral do modelo. Além disso, facilita a comparação de diferentes características e garante que os modelos treinados sejam mais robustos e interpretáveis. Falaremos mais sobre normalização de dados mais adiante.

5.6 Transformação do Espaço de Características

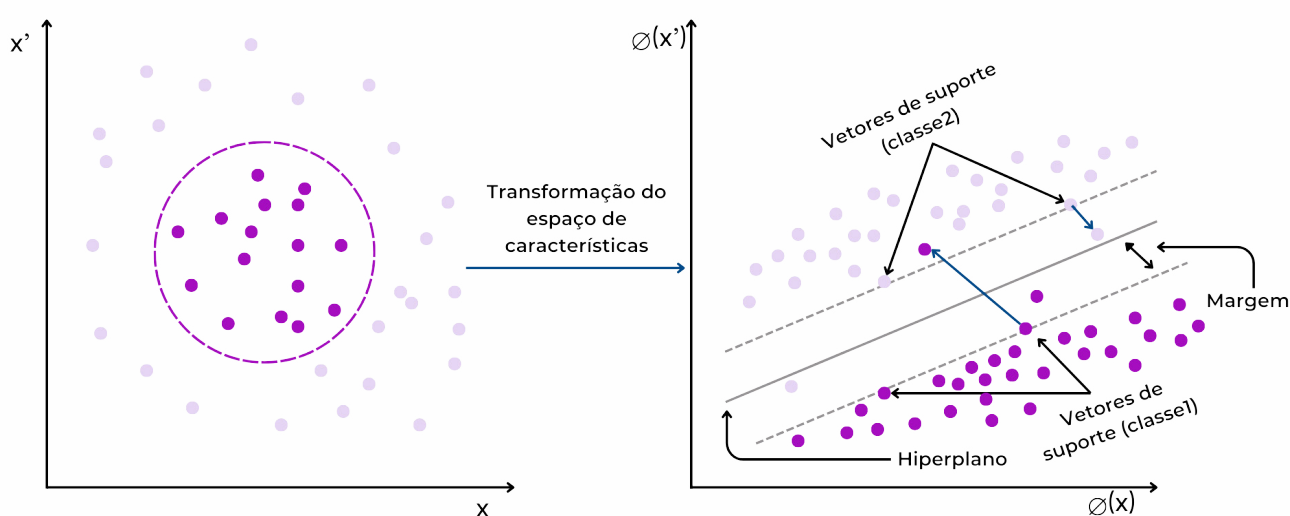
A transformação do espaço de características envolve a aplicação de técnicas para alterar a representação das características, de forma a destacar padrões ocultos nos dados ou tornar as características mais apropriadas para o modelo de ML. Apesar das técnicas de transformação do espaço de características alterarem a escala dos valores das características, não confunda com normalização. O processo de normalizar reescala as características para mitigar possíveis desbalanceamentos nos valores que podem priorizar algumas características em detrimento de outras. Já as transformações re-escalam os valores das características para destacar padrões mais significativos e melhorar o desempenho do modelo. Transformações podem incluir a aplicação de funções matemáticas, como logaritmos, exponenciais, ou polinômios, bem como a utilização de técnicas avançadas como PCA.

PCA é uma técnica amplamente utilizada que transforma um conjunto de características originais em um novo conjunto de características ortogonais (componentes principais), que capturam a maior parte da variação presente nos dados originais. Essa transformação pode reduzir a dimensionalidade dos dados, ao mesmo tempo que preserva as informações essenciais, facilitando a visualização e a modelagem.

Outra técnica importante é o uso de *embeddings*, particularmente em PLN. *Embeddings*, como *Word2Vec* ou BERT, transformam palavras em vetores de características densos que capturam contextos semânticos, permitindo que os modelos de PLN lidem com a semântica de maneira mais eficaz. A transformação correta do espaço de características pode melhorar significativamente o desempenho dos modelos de ML.

Na Figura 22, é exemplificado o processo de transformação do espaço de características usando o *kernel* do SVM. De forma simplificada, o *kernel* transforma os dados de entrada para um espaço de características de maior dimensão. Esta transformação pode tornar dados que são não linearmente separáveis em seu espaço original (de baixa dimensão) linearmente separáveis no novo espaço (de alta dimensão). Nesse caso, aumentar a dimensionalidade, efeito colateral da transformação, favorece a separação das classes.

Figura 22 - Transformação do espaço de características



Fonte: adaptada de [Niyogisubizo et al. \(2023\)](#).

5.7 Redução da Dimensionalidade

A redução da dimensionalidade é o processo de diminuir o número de características em um conjunto de dados, mantendo a maior quantidade possível de informação relevante. Essa técnica é crucial para lidar com a maldição da dimensionalidade e melhorar a eficiência dos algoritmos de ML. Métodos como PCA e *Linear Discriminant Analysis* (LDA) são frequentemente utilizados para esse fim.

PCA reduz a dimensionalidade projetando os dados em um espaço de menor dimensão que captura a maior variância possível. LDA, por outro lado, busca maximizar a separabilidade entre diferentes classes ao projetar os dados em um espaço que preserva as informações discriminantes. Ambas as técnicas ajudam a simplificar o modelo, reduzindo o ruído e a complexidade computacional.

A redução da dimensionalidade não apenas melhora a eficiência do processamento, mas também pode levar a modelos mais robustos e generalizáveis. Ao focar nas características mais importantes, os modelos são menos propensos a *overfitting* (sobreajuste) e mais capazes de capturar os padrões subjacentes nos dados.

5.8 Seleção de Características vs. Redução da Dimensionalidade

A seleção de características e a redução da dimensionalidade são técnicas relacionadas, mas distintas, em ML. A seleção de características envolve a escolha de um subconjunto das características originais que são mais relevantes para a tarefa de aprendizado. Como vimos anteriormente, isso pode ser feito utilizando métodos de filtro, *wrapper* (empacotagem) ou baseados em modelos, que avaliam a importância de cada característica de forma individual ou em combinação.

Por outro lado, a redução da dimensionalidade cria novas características a partir das originais, projetando os dados em um espaço de menor dimensão. Técnicas como PCA e LDA transformam as características existentes em novas combinações que capturam a maior parte da variabilidade ou informação discriminante dos dados. Essas novas características não são um simples subconjunto das originais, mas uma reconfiguração que pode melhorar a análise.

Ambas as abordagens são valiosas para lidar com conjuntos de dados de alta dimensão, mas a seleção de características mantém a interpretabilidade das características originais, enquanto a redução da dimensionalidade pode revelar estruturas e padrões mais profundos. A escolha entre essas técnicas depende do objetivo específico e das características do conjunto de dados.



PARA RELEMBRAR...

- » Nessa Unidade, discutimos o processamento e transformação de características em ML, abordando temas como extração e seleção de características, relação entre características e dimensionalidade, maldição da dimensionalidade, normalização e transformação do espaço de características, além da distinção entre seleção de características e redução de dimensionalidade. A extração de características foi apresentada como o processo de derivar informações relevantes dos dados brutos, com exemplos de técnicas como HOG para imagens e TF-IDF para textos. A seleção de características, por sua vez, envolve identificar e escolher o subconjunto mais relevante de características, utilizando métodos como filtros, *wrappers* e técnicas baseadas em modelos.
- » Exploramos a relação entre características e dimensionalidade, destacando como cada nova característica adiciona uma nova dimensão ao espaço dos dados. Isso pode aumentar a complexidade do modelo e levar à maldição da dimensionalidade, onde a alta dimensionalidade faz com que os dados se tornem esparsos e os algoritmos de ML tenham dificuldades para generalizar. A

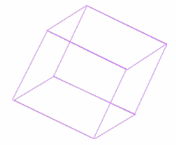
normalização do espaço de características foi enfatizada como uma prática essencial para ajustar as escalas das características, garantindo que todas contribuam igualmente para o modelo, o que é crucial para algoritmos baseados em distância, como kNN e SVM.

- » Além disso, discutimos a transformação do espaço de características, que pode incluir técnicas como PCA para redução da dimensionalidade, ajudando a simplificar os dados e melhorar a eficiência computacional. Foi feita uma distinção clara entre seleção de características e redução de dimensionalidade, onde a primeira mantém a interpretabilidade das características originais e a segunda transforma os dados em um novo espaço de menor dimensão. Abordamos também a importância de evitar o *overfitting* (sobreajuste), onde modelos complexos se ajustam excessivamente aos dados de treinamento, e como técnicas como validação cruzada e regularização podem ajudar a mitigar esse problema.

SAIBA MAIS...

- » [Artigo de revisão abrangente que revisa técnicas de seleção e extração de características.](#)
- » Revisão comparativa de diversas técnicas de redução de dimensionalidade, como PCA, t-SNE, e LDA.

Unidade VI
**Funções
discriminantes**



Unidade VI - Funções Discriminantes

6.1 Introdução

As funções discriminantes desempenham um papel fundamental no campo da aprendizagem de máquina e reconhecimento de padrões. Estas funções matemáticas são utilizadas para classificar ou categorizar objetos em diferentes classes com base em suas características ou atributos observáveis. O conceito de funções discriminantes tem suas raízes na estatística e na teoria da decisão, remontando ao início do século XX.

Um dos pioneiros no desenvolvimento de funções discriminantes foi o estatístico britânico Ronald Fisher, que introduziu a LDA em 1936. Fisher aplicou esta técnica para classificar espécies de íris com base em medidas de suas pétalas e sépalas. Seu trabalho estabeleceu as bases para muitas das abordagens modernas de classificação em aprendizagem de máquina.

Ao longo das décadas, outros pesquisadores contribuíram significativamente para o desenvolvimento das funções discriminantes e suas aplicações em ML. Entre esses estão Vladimir Vapnik, co-criador do SVM e Cedric Smith que introduziu uma variação do LDA com a Análise Discriminante Quadrática (QDA).

Formalmente, uma função discriminante é uma função matemática que mapeia um vetor de características de entrada para um valor escalar, que é então usado para tomar uma decisão de classificação. Para um problema de classificação com K classes, geralmente são definidas K funções discriminantes, uma para cada classe. A classe predita para um novo exemplo é determinada pela função discriminante que produz o maior valor para esse exemplo.

Esta abordagem fornece uma base teórica sólida para muitos algoritmos de classificação em aprendizagem de máquina, permitindo a criação de modelos que podem aprender a distinguir entre diferentes classes de objetos ou eventos com base em suas características observáveis.

6.2 Aplicações

As funções discriminantes são amplamente utilizadas em diversas aplicações de aprendizagem de máquina para solucionar problemas do cotidiano. Alguns exemplos notáveis incluem:

- » **Reconhecimento de caracteres:** em sistemas de OCR, funções discriminantes são empregadas para classificar caracteres individuais em letras ou números. Isso é crucial para a digitalização de documentos, leitura automática de placas de veículos e processamento de formulários escritos à mão.
- » **Diagnóstico médico:** em sistemas de diagnóstico, como a detecção de câncer, as funções discriminantes podem ser utilizadas para classificar se uma célula é cancerosa ou não com base em características extraídas de imagens médicas. O LDA, por exemplo, pode ser aplicado para distinguir entre células benignas e malignas, analisando parâmetros como a textura e a forma das células.
- » **Detecção de spam:** sistemas de filtragem de e-mails frequentemente utilizam funções discriminantes para classificar mensagens como spam ou não-spam. O modelo aprende a distinguir entre as duas classes com base em características do e-mail, como palavras-chave, estrutura do texto e informações do remetente.
- » **Reconhecimento de fala:** em sistemas de reconhecimento de voz, funções discriminantes são aplicadas para classificar segmentos de áudio em fonemas ou palavras específicas. Isso é fundamental para a criação de assistentes virtuais e sistemas de transcrição automática.
- » **Análise de sentimentos:** na análise de mídias sociais e avaliações de produtos, funções discriminantes são usadas para classificar o sentimento expresso em textos como positivo, negativo ou neutro. Isso auxilia empresas a entender a percepção do público sobre seus produtos ou serviços.
- » **Visão computacional:** em sistemas de reconhecimento facial, as funções discriminantes ajudam a identificar indivíduos com base em características faciais. Utilizando técnicas como o SVM, o sistema pode aprender a distinguir entre diferentes rostos, mesmo em condições de iluminação variada e ângulos diferentes.
- » **Previsão de risco de crédito:** instituições financeiras utilizam funções discriminantes para classificar clientes em diferentes níveis de risco de crédito, baseando-se em seu histórico financeiro, renda e outros fatores relevantes. Modelos como o QDA podem ser aplicados para identificar padrões em dados financeiros que indicam um risco elevado de inadimplência ou atividades fraudulentas.



6.3 Conceitos Importantes

Para compreender como as funções discriminantes podem ser utilizadas em aplicações é necessário conhecer um pouco mais. As subseções seguintes tratarão de conceitos importantes para esse entendimento.

6.3.1 Espaço de Características

O espaço de características é um conceito fundamental em ML. Ele representa um espaço multidimensional onde cada dimensão corresponde a uma característica ou atributo dos dados. Refere-se ao conjunto de todas as características ou variáveis independentes que são usadas para descrever as amostras de dados. Cada ponto nesse espaço representa uma amostra específica, e suas coordenadas correspondem aos valores das características dessa amostra.

Para ilustrar, considere um problema de classificação de flores, onde cada flor é descrita por quatro características: comprimento da sépala, largura da sépala, comprimento da pétala e largura da pétala. Aqui, o espaço de características é $\mathbb{R}^4$, e cada flor é um ponto nesse espaço.

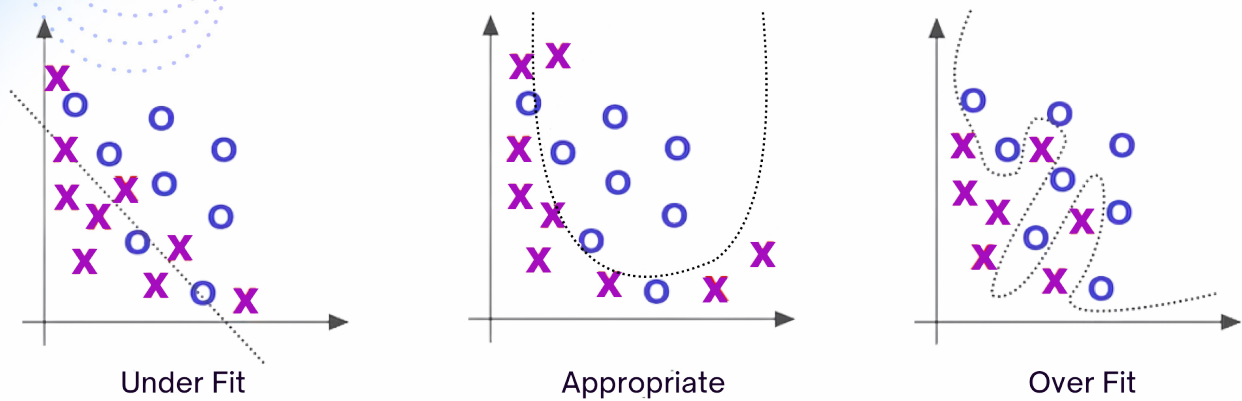
6.3.2 Fronteiras de Decisão

As fronteiras de decisão referem-se às regiões que separam o espaço de características em áreas associadas a diferentes classes. Na figura abaixo temos alguns exemplos de fronteiras de decisão para classificar X e O. O X e O podem ser, por exemplo, pessoas que foram diagnosticadas com uma doença ou não, respectivamente. Os eixos x, pode ser o valor dos resultados da contagem de leucócitos e o eixo y pode ser a quantidade de dias que a pessoa faz exercícios físicos.

As fronteiras de decisão podem ser:

- » **Fronteiras lineares:** hiperplanos que separam as classes de forma linear no espaço de características, como ilustrado na figura abaixo e a esquerda.
- » **Fronteiras quadráticas:** superfícies quadráticas que permitem separação não linear entre classes, sendo a mais simples fronteira não linear. Ela segue uma equação do segundo grau, como por exemplo . Um exemplo dessa curva aplicada à fronteira de decisão é ilustrada na figura abaixo e ao centro.
- » **Fronteiras não lineares:** superfícies de decisão complexas que podem assumir formas arbitrárias para melhor separar as classes. A figura abaixo ilustra uma complexa fronteira de decisão que separa perfeitamente X de O, contudo veremos na unidade Conjunto de Dados que nem sempre esse tipo de “perfeição” é vantajosa.

Figura 23 - Exemplo tipo de fronteiras



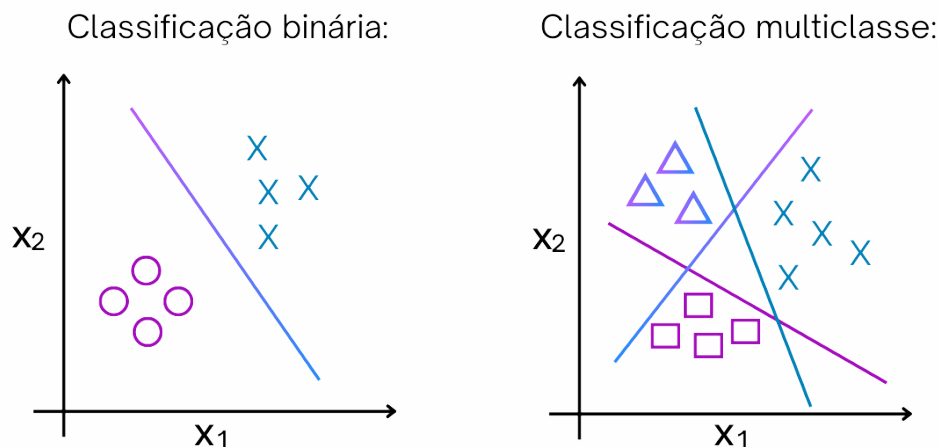
Fonte: adaptada de [Towards Data Science, 2018](#).

6.3.3 Classificação Binária vs. Multiclasse

Existem dois grandes grupos de classificação: binária e multiclasse. A classificação binária, como o próprio nome diz, refere-se apenas a duas classes, como por exemplo os casos ilustrados na figura abaixo e a direita. Nesse caso se resumem a uma única função (reta em verde) que separa as duas classes (O de X). Exemplos de métodos de classificação binária incluem Regressão Logística e SVM.

Em problemas de classificação com mais de duas classes, como classificação de imagens em diferentes categorias, são requeridas múltiplas funções discriminantes, uma para cada par de classes, como o ilustrado abaixo (Figura 24) e a direita. A reta vermelha separa o X dos quadrados e triângulos. A reta em verde separa os triângulos dos X e quadrados, por fim, a reta em azul separa os quadrados dos triângulos e X. Alguns exemplos de métodos multiclasse são a LDA e QDA.

Figura 24 - Na classificação binária utilizou-se uma função discriminante, na multi-classes, múltiplas funções discriminantes são utilizadas



Fonte: adaptada de [Medium, 2021](#).

6.4 Principais Algoritmos

Os algoritmos de aprendizagem de máquina que utilizam funções discriminantes variam em complexidade e capacidade de modelagem, os principais são:

- » **LDA:** é o algoritmo mais simples e amplamente utilizado. Ele assume que os dados seguem uma distribuição normal em cada classe e que as matrizes de covariância são iguais. É ideal para situações com baixo número de variáveis e alta dimensionalidade.
- » **QDA:** essa variante da LDA relaxa a suposição de igualdade das matrizes de covariância, permitindo que elas sejam diferentes para cada classe. É mais robusto que a LDA, mas pode ser computacionalmente mais caro.
- » **Análise discriminante regularizada (RDA):** essa técnica introduz regularização para lidar com problemas de alta dimensionalidade, onde o número de características é maior do que o número de observações.

6.4.1 Conceitos Matemáticos Importantes

Para entendimento dos algoritmos é bom termos uma noção de alguns conceitos matemáticos que são utilizados.

6.4.1.1 Matriz de Covariância ou de Dispersão

Suponha que você esteja conduzindo um estudo sobre o crescimento de diferentes espécies de plantas em um jardim. Para cada planta, você coleta dados sobre três características: altura, número de folhas e largura das folhas. Essas características podem variar de acordo com fatores como exposição ao sol, quantidade de água e tipo de solo.

A matriz de dispersão é uma ferramenta matemática que permite compreender como essas características variam em relação umas às outras. Por exemplo:

- » **Altura x Número de Folhas:** Plantas mais altas podem ter um maior número de folhas, ou essa correlação pode ser inexistente.
- » **Altura x Largura das Folhas:** Pode haver uma relação entre a altura da planta e a largura das folhas, indicando que plantas mais altas têm folhas mais largas.
- » **Número de Folhas x Largura das Folhas:** O número de folhas pode influenciar ou não a largura média das folhas.

A matriz de dispersão organiza essas relações de forma sistemática, permitindo uma análise clara de como cada par de características está relacionado. Se as características (altura, número de folhas e largura das folhas) tendem a aumentar ou diminuir juntas,

isso sugere uma correlação positiva entre elas. Caso contrário, a matriz indicará uma falta de correlação significativa.

A matriz de dispersão é composta por elementos que representam a variância e a covariância entre as diferentes características:

- » **Variância:** Refere-se à medida de quanto uma única característica (por exemplo, a altura) varia em relação à média.
- » **Covariância:** Refere-se à medida de como duas características (por exemplo, altura e número de folhas) variam juntas.

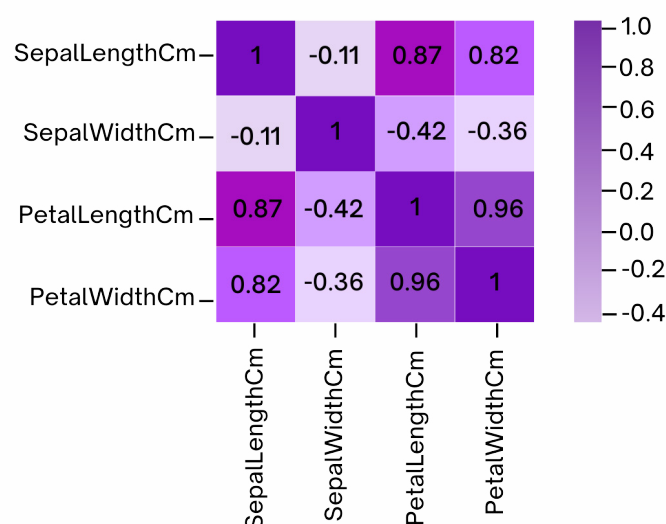
Por meio da matriz de dispersão, é possível identificar padrões nos dados que podem fornecer tendências valiosas sobre os fatores que mais influenciam o crescimento das plantas.

Se uma covariância entre duas variáveis for **positiva**, isso indica que, à medida que uma das variáveis aumenta, a outra também tende a aumentar. Por exemplo, na figura abaixo, o valor positivo na interseção de “*Sepal.Length*” e “*Petal.Length*” indica que, conforme o comprimento da sépala aumenta, o comprimento da pétala também tende a aumentar. Isso sugere uma relação linear direta entre essas duas variáveis.

Por outro lado, se uma covariância entre duas variáveis for **negativa**, isso indica que, à medida que uma das variáveis aumenta, a outra tende a diminuir. Por exemplo, o valor negativo na interseção de “*Sepal.Length*” e “*Sepal.Width*”, sugere que à medida que o comprimento da sépala aumenta, a largura da sépala tende a diminuir, indicando uma relação inversa entre essas duas variáveis.

A matriz de covariância (veja Figura 25 abaixo) permite entender as relações de co-dependência entre as variáveis, e a presença de valores positivos ou negativos ajuda a identificar o tipo de relação linear entre elas.

Figura 25 - Matriz de covariância e as relações de co-dependência entre as variáveis



Fonte: adaptada de [Hackers Realm \(2022\)](#).

6.4.1.2 Autovetores e Autovalores

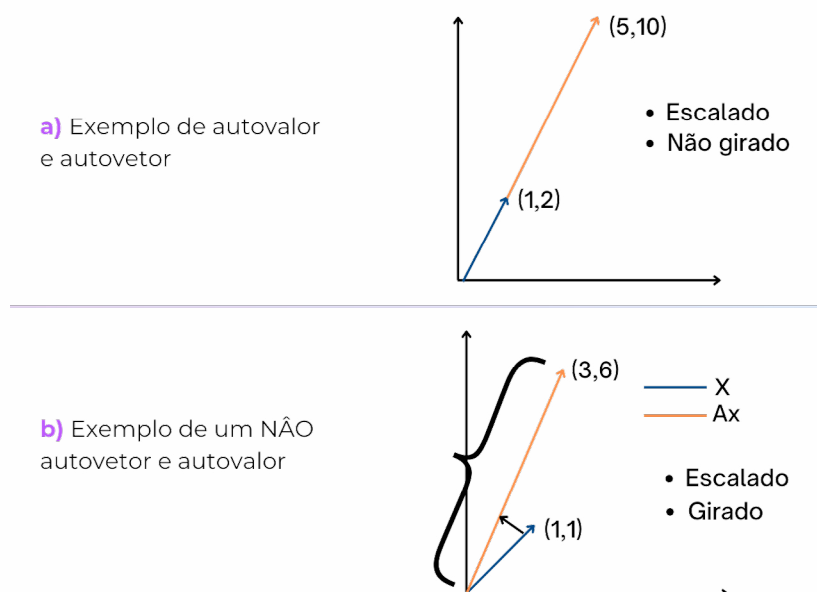
Os conceitos de autovalores e autovetores são fundamentais em diversas áreas da matemática aplicada, incluindo a análise de dados e o aprendizado de máquina. Apesar de serem conceitos avançados, eles podem ser compreendidos de maneira mais intuitiva por meio de uma analogia visual.

Imagine que você tem um vetor representado por uma flecha em um plano. Esse vetor aponta em uma determinada direção e possui um certo comprimento. Agora, suponha que aplicamos uma transformação matemática a esse vetor, como uma rotação, esticamento ou compressão. Após essa transformação, o vetor pode mudar de direção, de comprimento, ou ambos. Entretanto, em alguns casos especiais, o vetor mantém sua direção original, embora seu comprimento possa ter sido alterado. Quando isso ocorre, esse vetor é denominado **autovetor**.

O fator pelo qual o comprimento do vetor é alterado durante a transformação é chamado de **autovalor**. Ou seja, o autovalor representa o quanto o autovetor foi esticado ou comprimido, enquanto o autovetor é a direção que permanece inalterada.

Na figura abaixo, imagine que o vetor em azul passa por uma transformação matemática que resulta nos vetores em laranja. A transformação matemática aplicada a ambos os vetores em azul é a mesma (tanto no primeiro quanto no segundo gráfico da figura), porém os resultados são diferentes. No primeiro gráfico da figura 26, temos um vetor $(1,1)$ que após a transformação matemática teve rotação e alteração de tamanho, portanto pode-se dizer que o vetor azul **não é** um autovetor. Já no segundo gráfico, temos o vetor $(1,2)$ que após a transformação matemática **não** teve rotação porém teve alteração de tamanho, portanto pode-se dizer que o vetor azul **é** um autovetor e possui um autovalor de 5, que é o fator de escala quando comparados os vetores azul e laranja.

Figura 26 - Aplicação de transformações em vetor, (a) exemplo de autovalor e autovetor, (b) exemplo de um não autovetor e autovalor



Fonte: adaptada de [Solid Mechanics Classroom \(2017\)](#).

Em termos mais formais:

- » **Autovetor:** Um vetor que, após uma transformação linear, mantém sua direção original.
- » **Autovalor:** O escalar pelo qual o autovetor é multiplicado após a transformação.

Esses conceitos são amplamente utilizados na decomposição de matrizes, uma técnica que permite simplificar problemas complexos.

6.4.2 Análise Discriminante Linear (LDA)

A LDA é um método de classificação que busca encontrar uma combinação linear das características que melhor separa duas ou mais classes. O LDA assume que as classes têm a mesma matriz de covariância, mas médias diferentes.

Funcionamento do algoritmo LDA:

1. Calcula-se a média de cada classe;
2. Calcula-se a matriz de dispersão entre classes e dentro das classes;
3. Calcula-se os autovetores e autovalores da matriz de dispersão;
4. Seleciona-se os k melhores autovetores para projeção;
5. Projeta-se os dados no novo espaço e realiza a classificação.

Para ilustrar na prática uma aplicação do LDA usando o famoso conjunto de dados Iris, o código abaixo faz todo o processo do algoritmo descrito acima, cujo objetivo é classificar o tipo da flor com base nas suas características (largura e comprimento) de pétalas e sépalas.

Código 1 - Processamento do algoritmo de análise discriminante linear (LDA), usando o conjunto de dados Íris

```
# Importação das bibliotecas necessárias
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

# Carrega o conjunto de dados Iris
iris = load_iris()
X, y = iris.data, iris.target

# Divide os dados em conjuntos de treino e teste
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Cria e treina o modelo LDA
lda = LinearDiscriminantAnalysis()
lda.fit(X_train, y_train)

# Realiza previsões e avalia a acurácia
accuracy = lda.score(X_test, y_test)
print(f"Acurácia do LDA: {accuracy:.2f}")
```



6.4.2.1 Detalhamento do Código

Código 2 - Importa as bibliotecas necessárias

```
# Importação das bibliotecas necessárias
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
```

- **from sklearn.discriminant_analysis import LinearDiscriminantAnalysis:** Este comando importa a classe *Linear Discriminant Analysis* (LDA) da biblioteca *scikit-learn* (comumente abreviada como **sklearn**).

- **from sklearn.datasets import load_iris:** Aqui, importamos a função **load_iris**, que carrega o conjunto de dados Iris, um dos *datasets* mais conhecidos e frequentemente utilizados em exemplos de aprendizado de máquina.

- **from sklearn.model_selection import train_test_split:** Este comando importa a função **train_test_split**, que facilita a divisão do conjunto de dados em subconjuntos de treinamento e de teste.

Código 3 - Carrega o conjunto de dados Íris

```
iris = load_iris()  
X, y = iris.data, iris.target
```

- **iris = load_iris()**: Nesta linha, o conjunto de dados Iris é carregado e armazenado na variável **iris**. Este conjunto de dados contém informações sobre três espécies de flores: setosa, versicolor e virginica; com quatro características mensuradas para cada amostra (comprimento e largura das pétalas e sépalas).

- **X, y = iris.data, iris.target**: Aqui, o conjunto de dados é dividido em duas partes:

- **X**: Representa as características das amostras: comprimento e largura das pétalas e sépalas (as variáveis independentes).

- **y**: Corresponde às classes ou rótulos das amostras (a variável dependente), indicando a qual espécie de flor cada amostra pertence (setosa, versicolor ou virginica).

Código 4 - Divide o conjunto de dados em dois subconjuntos

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

- **train_test_split(X, y, test_size=0.3, random_state=42)**: Esta função divide o conjunto de dados em dois subconjuntos, seu objetivo mais claro será abordado na unidade 8:

- **X_train e y_train**: Contêm as amostras que serão utilizadas para treinar o modelo.

- **X_test e y_test**: Contêm as amostras que serão utilizadas para testar o modelo, avaliando seu desempenho.

- **test_size=0.3**: Especifica que 30% das amostras serão reservadas para o teste, enquanto os 70% restantes serão utilizados para o treinamento.

- **random_state=42**: Define um valor fixo para o gerador de números aleatórios, garantindo que a divisão dos dados seja reproduzível em diferentes execuções do código.

Código 5 - Cria o modelo de análise discriminante linear (LDA)

```
lda = LinearDiscriminantAnalysis()  
lda.fit(X_train, y_train)
```

- **lda = LinearDiscriminantAnalysis()**: Esta linha cria um modelo LDA que será utilizado para a classificação das amostras.

- **lda.fit(X_train, y_train)**: O método fit treina o modelo LDA utilizando os dados de treinamento **X_train** e **y_train**. Durante este processo, o modelo aprende as relações en-

tre as características das amostras e suas respectivas classes, ajustando os parâmetros internos para maximizar a separação entre as diferentes classes. É nessa fase que o algoritmo apresentado anteriormente é de fato aplicado.

Código 6 - Realiza previsões e avalia a acurácia do modelo

```
# Realiza previsões e avalia a acurácia
accuracy = lda.score(X_test, y_test)
print(f"Acurácia do LDA: {accuracy:.2f}")
```

- `accuracy = lda.score(X_test, y_test)`: Este método, chamado *score*, é aplicado ao modelo *lda* previamente treinado. Ele calcula a acurácia do modelo, ou seja, a proporção de previsões corretas que o modelo fez ao ser testado com o conjunto de dados *X_test* em comparação com os rótulos verdadeiros *y_test*. O resultado é guardado na variável *accuracy*, que representará a porcentagem de amostras corretamente classificadas pelo modelo.

- `print(f"Acurácia do LDA: {accuracy:.2f}")`: Esta função `print` imprime uma mensagem na tela. `f"Acurácia do LDA: {accuracy:.2f}"` é uma string formatada que inclui o valor da acurácia calculada. A expressão `{accuracy:.2f}` formata o valor da acurácia para que seja exibido com duas casas decimais, facilitando a leitura e a interpretação do resultado. Isso só é possível com o uso do `f` antes da string, que permite inserir o valor da variável *accuracy* diretamente no texto.

6.4.3 Análise Discriminante Quadrática (QDA)

A QDA é uma extensão do LDA que não assume que as classes têm a mesma matriz de covariância. Isso permite que o QDA capture relações não lineares entre as características.

Funcionamento do Algoritmo QDA:

1. Calcula a média e a matriz de covariância para cada classe;
2. Calcula a função discriminante quadrática para cada classe;
3. Classifica novos pontos de dados com base na função discriminante que produz o maior valor.

Para ilustrar na prática uma aplicação do QDA abaixo é proposto um código muito similar ao utilizado no LDA, com pequenas diferenças que serão detalhadas a seguir.

Código 7 - Processamento do algoritmo de análise discriminante quadrática (QDA)

```
# Importação das bibliotecas necessárias
from sklearn.discriminant_analysis import QuadraticDiscriminantAnalysis
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

# Carrega o conjunto de dados Iris
iris = load_iris()
X, y = iris.data, iris.target

# Divide os dados em conjuntos de treino e teste
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_
state=42)

# Cria e treina o modelo QDA
qda = QuadraticDiscriminantAnalysis()
qda.fit(X_train, y_train)

# Realiza previsões e avalia a acurácia
accuracy = qda.score(X_test, y_test)
print(f"Acurácia do QDA: {accuracy:.2f}")
```

Nessa primeira parte do código, somente a primeira linha possui alguma diferença.

Código 8 - Importa as bibliotecas necessárias

```
# Importação das bibliotecas necessárias
from sklearn.discriminant_analysis import QuadraticDiscriminantAnalysis
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
```

- `from sklearn.discriminant_analysis import QuadraticDiscriminantAnalysis`: Este comando importa a classe referente ao QDA da biblioteca *scikit-learn*.

Outra parte que foi necessária adaptação é o emprego dessa classe mais a frente no código.

Código 9 - Cria e treina o modelo de análise discriminante quadrática (QDA)

```
# Cria e treina o modelo QDA
qda = QuadraticDiscriminantAnalysis()
qda.fit(X_train, y_train)
```

De forma similar ao visto no QDA:

- `qda = QuadraticDiscriminantAnalysis()`: Esta linha cria um modelo QDA que será utilizado para a classificação das amostras.

- `qda.fit(X_train, y_train)`: O método `fit` treina o modelo LDA utilizando os dados de treinamento `X_train` e `y_train`.

Por fim, a apresentação dos resultados de acurácia também precisam ser adaptados.

Código 10 - Realiza previsões e avalia a acurácia

```
# Realiza previsões e avalia a acurácia
accuracy = qda.score(X_test, y_test)
print(f"Acurácia do QDA: {accuracy:.2f}")
```

Em comparação com o LDA em `accuracy = qda.score(X_test, y_test)`, substitui-se `lda` por `qda` para que sejam capturados os `score` do modelo `qda`. De forma análoga, em `print(f"Acurácia do QDA: {accuracy:.2f}")` é alterado o texto da impressão de LDA para QDA.

6.4.4 Análise Discriminante Regularizada (RDA)

A RDA é uma técnica que combina LDA e QDA, permitindo um equilíbrio entre os dois métodos através de um parâmetro de regularização.

O parâmetro de regularização é uma variável crucial que controla a complexidade do modelo e a forma como a regularização é aplicada. A regularização é uma técnica utilizada para evitar o sobreajuste (*overfitting*), que ocorre quando um modelo se ajusta excessivamente aos dados de treinamento e perde capacidade de generalização para novos dados.

Ele equilibra entre a matriz de covariância empírica (calculada diretamente a partir dos dados) e uma matriz diagonal (uma matriz onde apenas as variâncias das variáveis são consideradas, e as covariâncias são ignoradas) com valores variando entre 0 e 1. Com valor 0, não é aplicada nenhuma regularização, portanto a matriz de covariância regularizada é igual à matriz de covariância empírica. Com valor 1, temos o máximo de regularização, ou seja, a matriz de covariância regularizada é igual à matriz de covariância diagonal. Valores intermediário de regularização a matriz de covariância regularizada é uma combinação ponderada entre a matriz de covariância empírica e a matriz de covariância diagonal

Funcionamento do algoritmo RDA:

1. Calcula a média de cada classe;
2. Estima as matrizes de covariância de cada classe;
3. Aplica regularização para ajustar as matrizes de covariância;
4. Calcula a função discriminante regularizada;
5. Classifica novos pontos de dados com base na função discriminante que produz o maior valor.

Código 11 - Processamento do algoritmo de análise discriminante regularizada (RDA)

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

# Carrega o conjunto de dados Iris
iris = load_iris()
X, y = iris.data, iris.target

# Divide os dados em conjuntos de treino e teste
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_
state=42)

# Cria e treina o modelo RDA (usando LDA com shrinkage)
rda = LinearDiscriminantAnalysis(solver='lsqr', shrinkage='auto')
rda.fit(X_train, y_train)

# Realiza previsões e avalia a acurácia
accuracy = rda.score(X_test, y_test)
print(f"Acurácia do RDA: {accuracy:.2f}")
```

Se compararmos o código proposto ao apresentado no LDA notam-se pequenas diferenças. Não foi necessário a importação de um nova classe, sendo utilizada a mesma **LinearDiscriminantAnalysis** aplicada no LDA. A principal mudança está no uso de alguns argumentos no momento da criação do modelo **rda**.

Código 12 - Cria e treina o modelo RDA (usando LDA com shrinkage)

```
# Cria e treina o modelo RDA (usando LDA com shrinkage)
rda = LinearDiscriminantAnalysis(solver='lsqr', shrinkage='auto')
rda.fit(X_train, y_train)
```

- **rda = LinearDiscriminantAnalysis(solver='lsqr', shrinkage='auto')**: cria o modelo **rda**, utilizando a regularização automática através do emprego do argumento **shrinkage='auto'**.

- **shrinkage** é o parâmetro de regularização, visto anteriormente. O valor **'auto'** encontrará o melhor valor de regularização, porém pode ser alterado manualmente para valores entre 0 e 1.

- **solver** é um parâmetro para escolher o algoritmo para encontrar a solução. Para o uso com o RDA é necessário ajustá-lo para **'lsqr'**, para utilizar o método dos mínimos quadrados; ou **'eigen'**, quando quer utilizar decomposição de autovalores. Somente com o uso desses 2 valores é calculada a matriz de covariância e o pode-se utilizar em conjunto o parâmetro **shrinkage**.



6.5 Notebook Colab

Nessa seção nós vamos utilizar o [Notebook Colab](#) para a implementação dos algoritmos LDA, QDA e RDA explicados anteriormente, bem como alguns exercícios.

Objetivos de Aprendizagem

- » Fazer uma análise exploratória inicial no conjunto de dados Iris
- » Entender o funcionamento da Análise Discriminante Linear (LDA)
- » Entender o funcionamento da Análise Discriminante Quadrática (QDA)
- » Entender o funcionamento da Análise Discriminante Regularizada (RDA)

Análise Exploratória do dataset Iris

O conjunto de dados **Iris** é extremamente reconhecido e utilizado em estatística e aprendizagem de máquina, especialmente como exemplo introdutório para a aplicação de técnicas de classificação. Este conjunto de dados consiste em 150 amostras de flores pertencentes à espécie Iris, distribuídas igualmente entre três subespécies: **Iris setosa**, **Iris versicolor** e **Iris virginica**.

Cada amostra no conjunto de dados é caracterizada por quatro variáveis numéricas que descrevem atributos morfológicos das flores:

- » **Comprimento da sépala** (em centímetros);
- » **Largura da sépala** (em centímetros);
- » **Comprimento da pétala** (em centímetros);
- » **Largura da pétala** (em centímetros).

Devido à sua simplicidade, equilíbrio e bem-definidas classes, o conjunto de dados Iris é frequentemente empregado para ilustrar algoritmos de classificação e técnicas de análise exploratória de dados.

```
from sklearn.datasets import load_iris
from sklearn.metrics import accuracy_score
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Carrega o conjunto de dados Iris, incluindo um dataframe
# Essa função retorna um dicionário, com vários itens
iris = load_iris(as_frame=True)

# Guarda o item 'frame' do dicionário como df
df = iris.frame
```

continua

```
# Mostra os primeiros dados do dataset
df.head()
```



	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

```
# Note que a coluna target está em função de valores numéricos
# Para melhor entender, será adicionado uma coluna que converte esse número para a
espécie da flor
```

```
# Acessando os nomes das espécies
# Segundo a documentação do sklearn, os valores dos rótulos estão contidos no
dicionário do load_iris()
# Guarda o item 'target_names' do dicionario como df
target_names = iris.target_names
```

```
# Adicionando uma nova coluna com os nomes das espécies
df['species'] = target_names[df['target']]
```

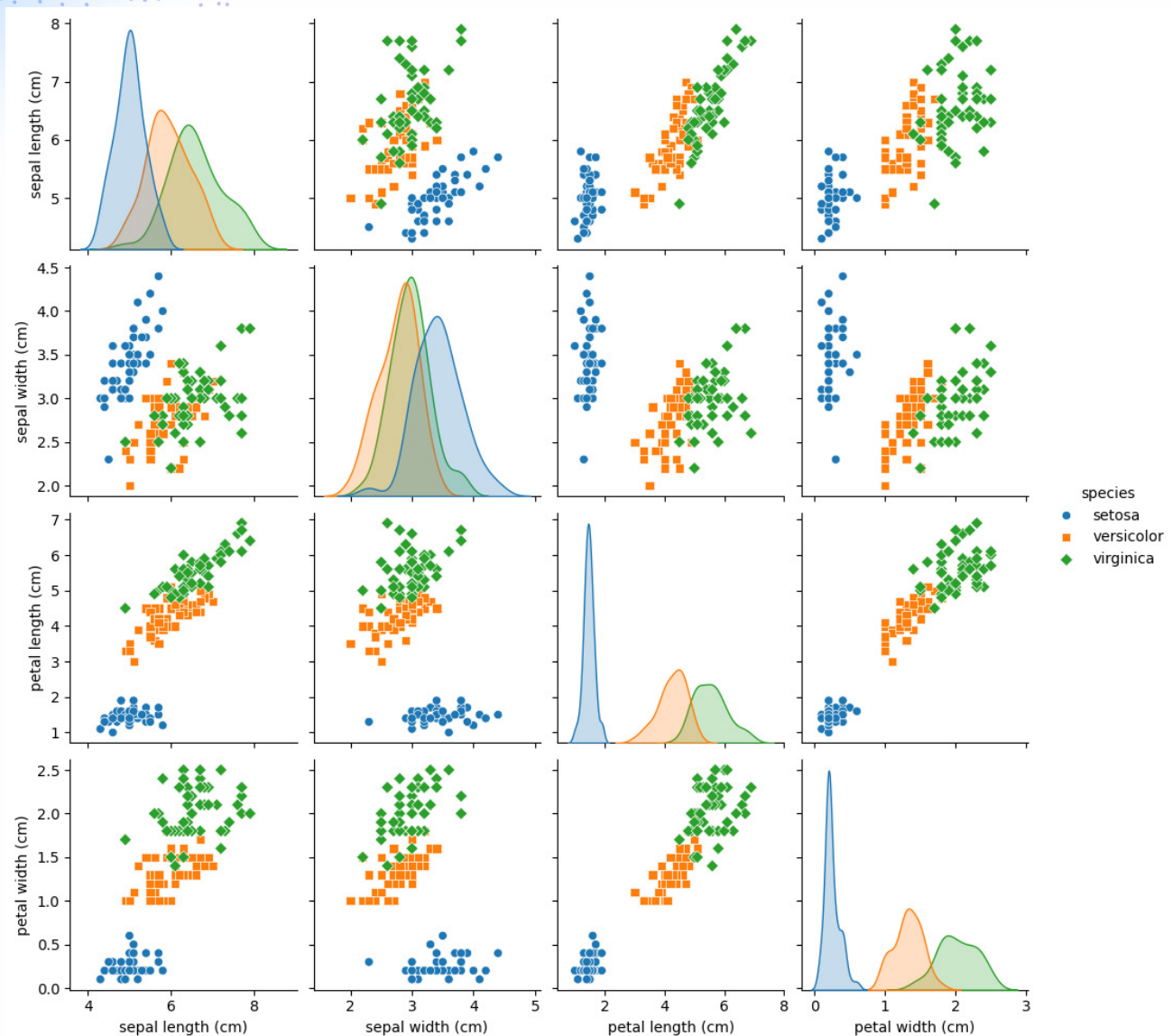
```
# Visualizando o DataFrame
# Observe a última coluna com o nome das espécies
df.head()
```



	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target	species
0	5.1	3.5	1.4	0.2	0	setosa
1	4.9	3.0	1.4	0.2	0	setosa
2	4.7	3.2	1.3	0.2	0	setosa
3	4.6	3.1	1.5	0.2	0	setosa
4	5.0	3.6	1.4	0.2	0	setosa

```
# Apresenta um gráfico com os dados do dataset iris
# Cada gráfico é composto de 16 minigráficos
# Cada minigráfico possui uma característica no eixo x e outra no eixo y
# Essa apresentação facilita o entendimento de todo o dataset e dá um panorama geral
dos dados
ax = sns.pairplot(df.drop(columns='target'), hue='species', markers=["o", "s", "D"])
```

Figura 27 - Gráfico com os dados do dataset iris



Fonte: autoria própria.

Análise Discriminante Linear (LDA)

Análise Discriminante Linear (LDA) é um poderoso método de classificação que opera projetando um conjunto de dados em um espaço de menor dimensionalidade com boa separabilidade de classes. É utilizada em estatística, reconhecimento de padrões e aprendizado de máquina para encontrar uma combinação linear de características que caracterizam ou separam duas ou mais classes de objetos ou eventos. A combinação resultante pode ser usada como um classificador linear ou, mais comumente, para redução de dimensionalidade antes da classificação posterior. A LDA é particularmente eficaz para problemas de classificação multiclasse.

Vantagens :

Algumas vantagens do LDA residem no fato de que é um método linear, ou que o torna computacionalmente eficiente e fácil de implementar, especialmente em grandes conjuntos de dados.

Ele é capaz de reduzir a dimensionalidade dos dados, projetando-os em um espaço de menor dimensão, onde as classes são mais separáveis. Isso é particularmente útil para visualizar e melhorar o desempenho de outros algoritmos de aprendizado.

Ao maximizar a separação entre as classes, o LDA é eficaz em situações onde as classes são linearmente separáveis.

O LDA funciona bem em cenários onde as classes são aproximadamente balanceadas e com distribuições gaussianas, oferecendo bom desempenho.

Objetivo :

Nesse exemplo o objetivo é criar 2 novas características que representem as 4 já existentes de forma que melhor separem as 3 espécies.

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

# Separar em X os dados com as características
X = df.iloc[:, :-2].values
# Separar em y os dados com as espécies
y = df.iloc[:, -1].values

# Divide os dados em conjuntos de treino e teste
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Cria o modelo LDA para redução em 2 componentes
lda = LinearDiscriminantAnalysis(n_components=2)

# Treina com os valores de X_train e y_train
# obtem um novo X (X_train_lda) de dimensionalidade 2 com base no X_train
X_train_lda = lda.fit_transform(X_train, y_train)
# Com o modelo treinado co X_train e y_train
# obtem um novo X (X_test_lda) de dimensionalidade 2, com base no X_test
X_test_lda = lda.transform(X_test)
```

```
# Observe como o novo X (X_train_lda) agora possui apenas 2 características
print(f"Tamanho do X_train: {X_train.shape[1]}")
print(f"Tamanho do X_train_lda: {X_train_lda.shape[1]}")

# O mesmo ocorre com o conjunto X_test
print(f"Tamanho do X_test: {X_test.shape[1]}")
print(f"Tamanho do X_test_lda: {X_test_lda.shape[1]}")
```

```
➡ Tamanho do X_train: 4
   Tamanho do X_train_lda: 2
   Tamanho do X_test: 4
   Tamanho do X_test_lda: 2
```

O LDA criou essas duas características visando a máxima separação entre as classes, mas resta saber se foi efetivo, ou seja, será que com apenas essas duas classes é possível identificar as espécies?

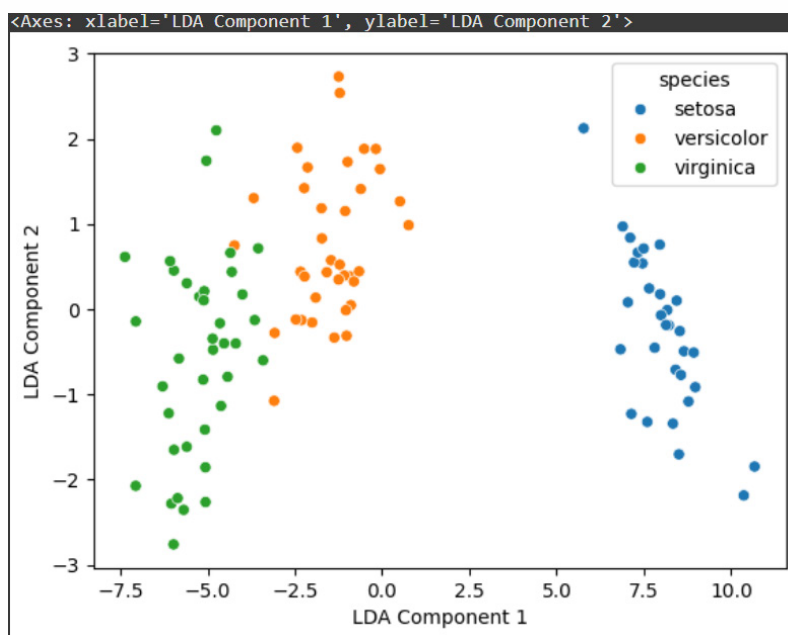
Abaixo veremos se isso ocorre.

```
# Coloca os dados do X_train_lda e y_train em um dataframe para melhor visualização
df_lda_train = pd.DataFrame(X_train_lda, columns=['LDA Component 1', 'LDA Component 2'])
df_lda_train['species'] = y_train
df_lda_train.head()
```

	LDA Component 1	LDA Component 2	species
0	-0.514612	1.882729	versicolor
1	-3.549638	0.715341	virginica
2	-4.527751	-0.400175	virginica
3	-1.036480	-0.011181	versicolor
4	-5.975968	-2.759810	virginica

```
# Plota os valores do X_train_lda
# o sort_values é apenas para que as espécies apareçam em uma determinada ordem
sns.scatterplot(df_lda_train.sort_values(by='species'), x='LDA Component 1', y='LDA Component 2', hue='species')
```

Figura 28 - Gráfico dos valores do X_train_lda



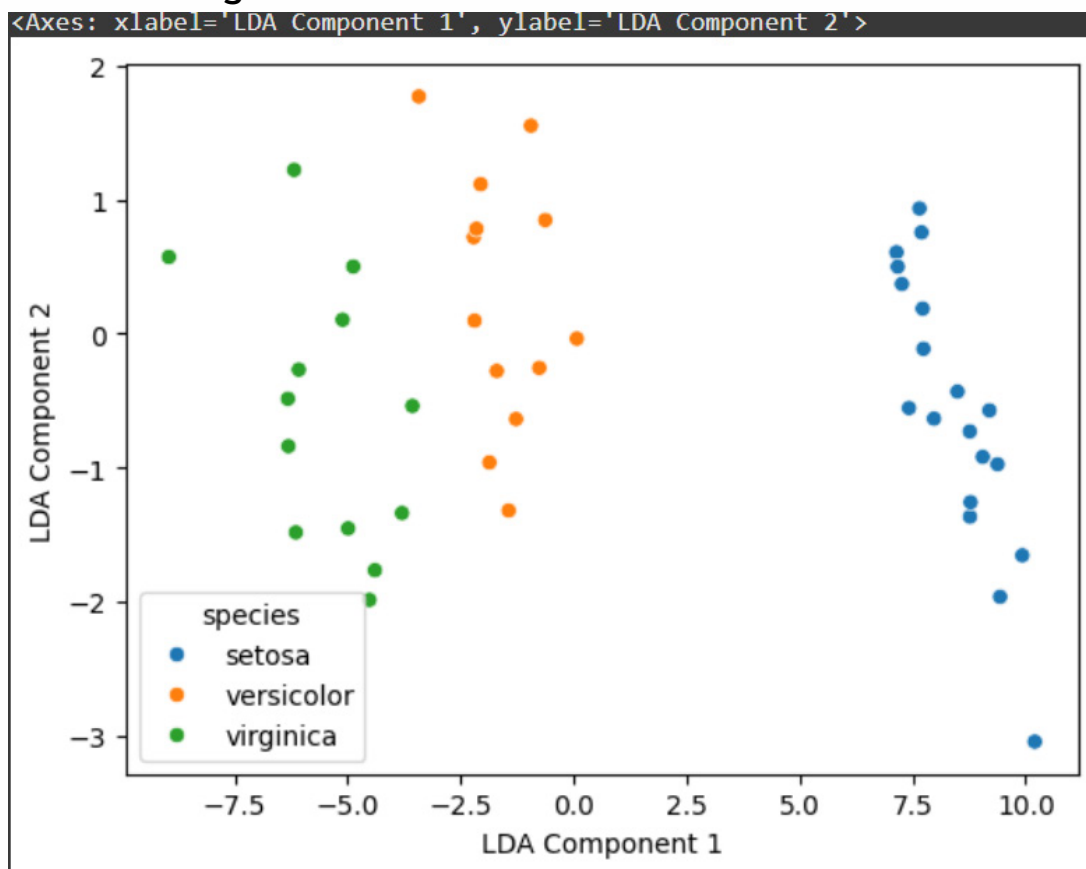
Fonte: autoria própria.

```
# Repetindo para o X_test

# Coloca os dados do X_test_lda e y_test em um dataframe para melhor visualização
df_lda_test = pd.DataFrame(X_test_lda, columns=['LDA Component 1', 'LDA Component 2'])
df_lda_test['species'] = y_test

# Plota os valores do X_train_lda
# o sort_values é apenas para que as espécies apareçam em uma determinada ordem
sns.scatterplot(df_lda_test.sort_values(by='species'), x='LDA Component 1', y='LDA Component 2', hue='species')
```

Figura 29 - Gráfico dos valores do X_train_lda



Fonte: autoria própria.

Análise dos gráficos

Compare ambos gráficos (Treino e Test).

Note como é praticamente possível separar as 3 espécies apenas com linhas verticais a partir do LDA Component 1. Isso é mais visível na classe setosa, porém a fronteira entre as classes virginica e versicolor está bem definida, apesar de, no caso do uso de uma regressão linear, alguns poucos pontos do conjunto de treino podem ser classificados equivocadamente. Já há caso do teste há clara divisão.

Os gráficos apresentados são o resultado da LDA, no entanto não é possível dizer se ele foi efetivo em classificação corretamente. É necessário comparar com o "gabarito".

```
# Obtem as predições dos dados com base nos modelos treinados
y_pred_lda_train = lda.predict(X_train)
y_pred_lda_test = lda.predict(X_test)

# Adicionar aos dataframes
df_lda_train['species_predicted'] = y_pred_lda_train
df_lda_test['species_predicted'] = y_pred_lda_test

df_lda_train.head()
```

	LDA Component 1	LDA Component 2	species	species_predicted
0	-0.514612	1.882729	versicolor	versicolor
1	-3.549638	0.715341	virginica	versicolor
2	-4.527751	-0.400175	virginica	virginica
3	-1.036480	-0.011181	versicolor	versicolor
4	-5.975968	-2.759810	virginica	virginica

Notem como o LDA fez um excelente trabalho reduzindo a dimensionalidade do dataset e ainda mantendo um excelente performance (elevada acurácia).

- » Acertou 97% dos dados de TREINO
- » Acertou 100% dos dados de TESTE

```
# Calculando a acurácia do modelo, comparando o gabarito (y_train) com os valores do
modelo y_pred_lda_train
lda_train_acc = accuracy_score(y_train, y_pred_lda_train)
print(f"Acurácia do LDA no dataset de Treino: {lda_train_acc}")

# Calculando a acurácia do modelo, comparando o gabarito (y_test) com os valores do
modelo y_pred_lda_test
lda_test_acc = accuracy_score(y_test, y_pred_lda_test)
print(f"Acurácia do LDA no dataset de Teste: {lda_test_acc}")
```

```
➡ Acurácia do LDA no dataset de Treino: 0.9714285714285714
Acurácia do LDA no dataset de Teste: 1.0
```

```
# Identificando quais foram os casos de erros
df_lda_train_erros = df_lda_train.query("species != species_predicted")

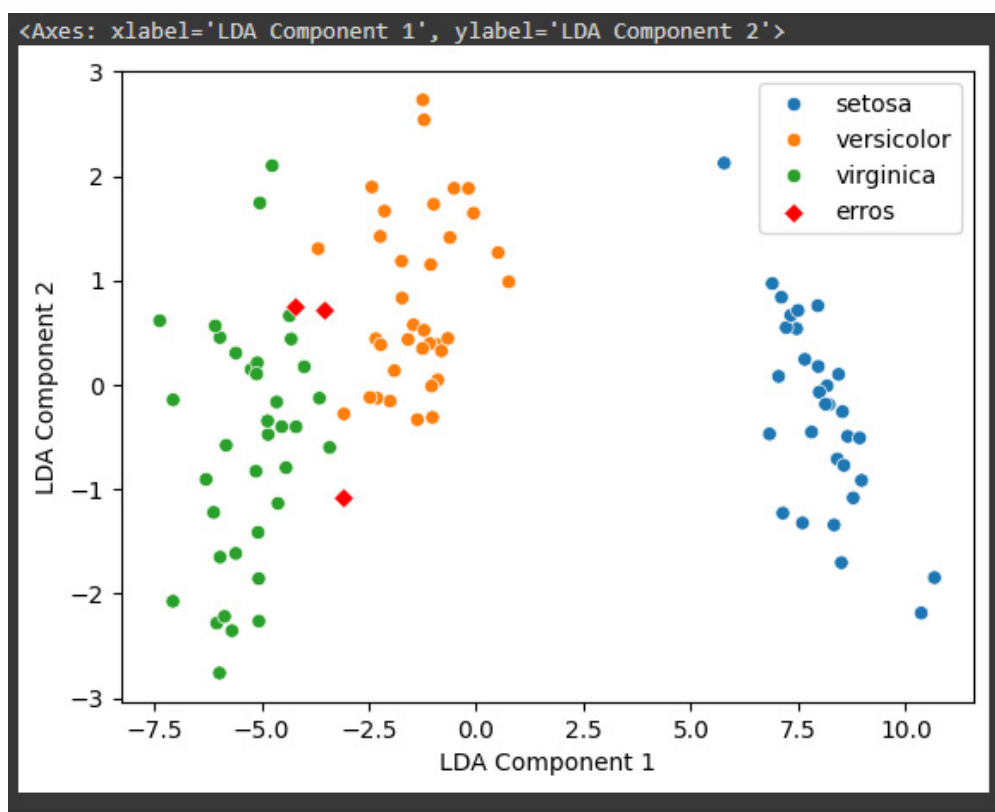
# Foram apenas 3 erros e estão listados abaixo
df_lda_train_erros.head()
```

	LDA Component 1	LDA Component 2	species	species_predicted
1	-3.549638	0.715341	virginica	versicolor
44	-3.094060	-1.075211	versicolor	virginica
47	-4.228972	0.748766	versicolor	virginica

Observando o gráfico abaixo, fica claro que os erros ocorreram mesmo na fronteira entre virginica e versicolor.

```
# Gráficos mostrando onde foram os erros
sns.scatterplot(df_lda_train.sort_values(by='species'), x='LDA Component 1', y='LDA Component 2', hue='species')
sns.scatterplot(df_lda_train_erros.sort_values(by='species'), x='LDA Component 1', y='LDA Component 2', color='red', marker='D', label='erros')
```

Figura 30 - Gráfico dos valores do X_train_lda



Análise Discriminante Quadrática (QDA)

A Análise Discriminante Quadrática (QDA) é um algoritmo de classificação que, assim como o LDA, busca separar classes de dados, mas difere na suposição de que as variâncias das classes não são iguais. Enquanto o LDA assume que todas as classes compartilham a mesma matriz de covariância e, portanto, traça limites de decisão lineares, o QDA permite que cada classe tenha sua própria matriz de covariância, resultando em limites de decisão quadráticos. Essa flexibilidade torna o QDA mais adequado para problemas onde as fronteiras de decisão entre as classes são não-lineares e as classes têm distribuições com diferentes variâncias.

Vantagens :

As principais vantagens do QDA incluem a sua capacidade de modelar limites de decisão mais complexos, o que é especialmente útil em situações onde os dados não seguem distribuições gaussianas idênticas entre as classes. O QDA pode capturar padrões mais complexos, o que pode levar a um melhor desempenho em classificações onde as classes são não-linearmente separáveis. Além disso, o QDA é mais flexível do que o LDA, o que permite uma adaptação mais precisa aos dados, embora essa maior flexibilidade também possa aumentar o risco de overfitting, especialmente em conjuntos de dados pequenos ou com muitas dimensões.

Objetivo :

Nesse exemplo o objetivo é identificar corretamente através das 4 características as 3 espécies.

Nesse exemplo será utilizado o mesmo dataset Iris importado anteriormente.

```
from sklearn.discriminant_analysis import QuadraticDiscriminantAnalysis
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

# Separar em X os dados com as características
X = df.iloc[:, :-2].values
# Separar em y os dados com as espécies
y = df.iloc[:, -1].values

# Divide os dados em conjuntos de treino e teste
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Cria e treina o modelo QDA
qda = QuadraticDiscriminantAnalysis()
qda.fit(X_train, y_train)

# Obtem as predições dos dados com base nos modelos treinados
y_pred_qda_train = qda.predict(X_train)
y_pred_qda_test = qda.predict(X_test)
```

Exercícios :

Exercício 1

Utilizando os dados gerados anteriormente, verifique a acurácia do QDA nos conjuntos de Treino e Validação. Quais são esses valores?

Resposta: os valores são 98% no Treino e 100% no teste. O código abaixo traz a solução:

```
# Calculando a acurácia do modelo, comparando o gabarito (y_train) com os valores do modelo y_pred_lda_train
qda_train_acc = accuracy_score(y_train, y_pred_qda_train)
print(f"Acurácia do QDA no dataset de Treino: {qda_train_acc}")

# Calculando a acurácia do modelo, comparando o gabarito (y_test) com os valores do modelo y_pred_lda_test
qda_test_acc = accuracy_score(y_test, y_pred_qda_test)
print(f"Acurácia do QDA no dataset de Teste: {qda_test_acc}")
```

```
⇒ Acurácia do QDA no dataset de Treino: 0.9809523809523809
   Acurácia do QDA no dataset de Teste: 1.0
```

Exercício 2

Com base no `df_qda_train` e nos dados criados nessa seção, liste os index do `df_qda_train` em que o QDA errou na classificação.

Resposta: Os index são 1 e 47. Os códigos abaixo esclarecem esse valor:

```
# Para verificarmos onde foi o erro, é criado um dataframe para melhor visualização
# df.iloc[:, :-2].columns utiliza os rótulos do df eliminando as últimas 2 colunas
df_qda_train = pd.DataFrame(X_train, columns=df.iloc[:, :-2].columns)
df_qda_train['species'] = y_train
```

```
# Adicionar aos dataframes
df_qda_train['species_predicted'] = y_pred_qda_train
```

```
df_qda_train.head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species	species_predicted
0	5.5	2.4	3.7	1.0	versicolor	versicolor
1	6.3	2.8	5.1	1.5	virginica	versicolor
2	6.4	3.1	5.5	1.8	virginica	virginica
3	6.6	3.0	4.4	1.4	versicolor	versicolor
4	7.2	3.6	6.1	2.5	virginica	virginica

continua

```
# Identificando quais foram os casos de erros
df_qda_train_erros = df_qda_train.query("species != species_predicted")

# Foram apenas 2 erros e estão listados abaixo
# Note pelo número do index, que eles também tinham sido classificados erroneamente no LDA
df_qda_train_erros.head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species	species_predicted
1	6.3	2.8	5.1	1.5	virginica	versicolor
47	6.0	2.7	5.1	1.6	versicolor	virginica

Análise Discriminante Regularizada (RDA)

A Análise Discriminante Regularizada (RDA) é uma extensão tanto do Linear Discriminant Analysis (LDA) quanto do Quadratic Discriminant Analysis (QDA). Enquanto o LDA assume que todas as classes têm a mesma matriz de covariância e o QDA permite diferentes matrizes de covariância para cada classe, o RDA introduz um parâmetro de regularização que ajusta o grau de suavização entre essas duas abordagens. Essencialmente, o RDA permite um trade-off controlável entre a simplicidade do LDA e a flexibilidade do QDA, tornando-o adequado para situações em que se deseja evitar o overfitting (comum no QDA) ou underfitting (potencial no LDA).

Vantagens:

As principais vantagens do RDA incluem sua flexibilidade e capacidade de adaptação. O uso do parâmetro de regularização permite que o RDA se ajuste melhor aos dados, especialmente em cenários onde as suposições estritas do LDA (matrizes de covariância iguais) são demasiado simplistas, e o QDA, com sua abordagem mais flexível, pode ser propenso ao overfitting. Essa regularização permite que o RDA ofereça um desempenho mais robusto, equilibrando a capacidade de generalização e a modelagem precisa dos dados. Isso é particularmente útil em contextos com conjuntos de dados limitados ou com muitas variáveis, onde escolher o modelo correto entre LDA e QDA pode ser desafiador.

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

# Separar em X os dados com as características
X = df.iloc[:, :-2].values
# Separar em y os dados com as espécies
y = df.iloc[:, -1].values

# Divide os dados em conjuntos de treino e teste
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Cria e treina o modelo RDA (usando LDA com shrinkage)
rda = LinearDiscriminantAnalysis(solver='lsqr', shrinkage='auto')
```

continua

```
rda.fit(X_train, y_train)
```

```
# Obtem as predições dos dados com base nos modelos treinados
```

```
y_pred_rda_train = rda.predict(X_train)
```

```
y_pred_rda_test = rda.predict(X_test)
```

```
# Calculando a acurácia do modelo, comparando o gabarito (y_train) com os valores do modelo y_pred_lda_train
```

```
rda_train_acc = accuracy_score(y_train, y_pred_rda_train)
```

```
print(f"Acurácia do RDA no dataset de Treino: {rda_train_acc}")
```

```
# Calculando a acurácia do modelo, comparando o gabarito (y_test) com os valores do modelo y_pred_lda_test
```

```
rda_test_acc = accuracy_score(y_test, y_pred_rda_test)
```

```
print(f"Acurácia do RDA no dataset de Teste: {rda_test_acc}")
```



Acurácia do RDA no dataset de Treino: 0.9714285714285714

Acurácia do RDA no dataset de Teste: 1.0

```
# Para verificarmos onde foi o erro, é criado um dataframe para melhor visualização
```

```
# df.iloc[:, :-2].columns utiliza os rótulos do df eliminando as últimas 2 colunas
```

```
df_rda_train = pd.DataFrame(X_train, columns=df.iloc[:, :-2].columns)
```

```
df_rda_train['species']=y_train
```

```
# Adicionar aos dataframes
```

```
df_rda_train['species_predicted'] = y_pred_rda_train
```

```
df_rda_train.head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species	species_predicted
0	5.5	2.4	3.7	1.0	versicolor	versicolor
1	6.3	2.8	5.1	1.5	virginica	versicolor
2	6.4	3.1	5.5	1.8	virginica	virginica
3	6.6	3.0	4.4	1.4	versicolor	versicolor
4	7.2	3.6	6.1	2.5	virginica	virginica

```
# Identificando quais foram os casos de erros
```

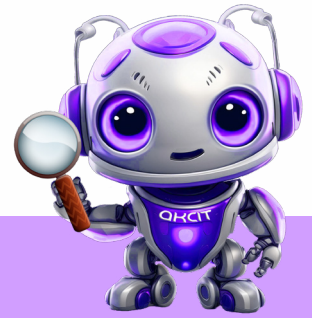
```
df_rda_train_erros = df_rda_train.query("species != species_predicted")
```

```
# Foram apenas 3 erros e estão listados abaixo
```

```
# Note pelo número do index, que eles também tinham sido classificados erroneamente no LDA
```

```
df_rda_train_erros.head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species	species_predicted
1	6.3	2.8	5.1	1.5	virginica	versicolor
44	5.9	3.2	4.8	1.8	versicolor	virginica
47	6.0	2.7	5.1	1.6	versicolor	virginica



SAIBA MAIS...

- » LDA:
 - » Fisher, R. A. (1936). The use of multiple measurements in taxonomic problems. Annals of Eugenics, 7(2), 179-188.
 - » <https://www.geeksforgeeks.org/ml-linear-discriminant-analysis/>
 - » <https://www.youtube.com/watch?v=azXCzI57Yfc>
 - » <https://www.youtube.com/watch?v=julEqA2ozcA>
- » QDA: Smith, C. A. B. (1947). Some examples of discrimination. Annals of Eugenics, 13(1), 272-282.
- » RDA: Friedman, J. H. (1989). Regularized discriminant analysis. Journal of the American Statistical Association, 84(405), 165-175.

PARA RELEMBRAR...

- » Cada algoritmo possui suas próprias características e é mais adequado para determinados tipos de problemas. A escolha do algoritmo depende das características dos dados, do objetivo da tarefa de classificação e dos requisitos de desempenho e interpretabilidade do modelo.

Tabela 3 - Resumo das vantagens e desvantagens dos algoritmos que usam funções discriminantes

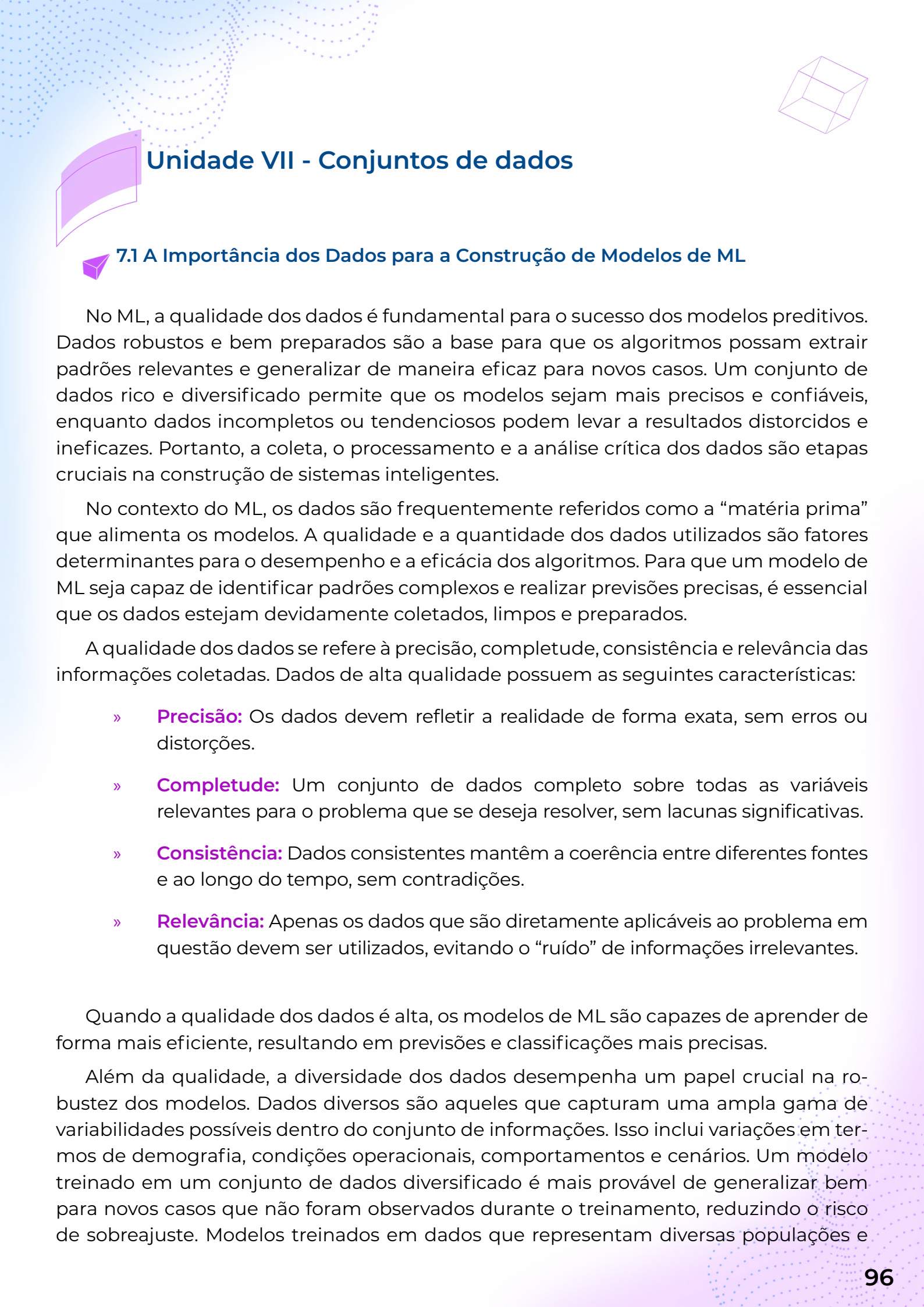
ALGORITMO	VANTAGENS	DESVANTAGENS
LDA	• Simples e rápido • Funciona bem com dados lineares separáveis • Robusto a ruído	• Assume distribuição normal • Não funciona bem com dados não lineares • Sensível a outliers
QDA	• Pode modelar fronteiras de decisão não lineares • Não assume igualdade das matrizes de covariância	• Requer mais dados para estimar as matrizes de covariância • Mais suscetível a overfitting
RDA	• Lida bem com alta dimensionalidade • Mais robusto a overfitting	• Requer escolha do parâmetro de regularização • Pode perder informações importantes com regularização excessiva

Fonte: autoria própria.

Unidade VII

Conjuntos de dados





Unidade VII - Conjuntos de dados

7.1 A Importância dos Dados para a Construção de Modelos de ML

No ML, a qualidade dos dados é fundamental para o sucesso dos modelos preditivos. Dados robustos e bem preparados são a base para que os algoritmos possam extrair padrões relevantes e generalizar de maneira eficaz para novos casos. Um conjunto de dados rico e diversificado permite que os modelos sejam mais precisos e confiáveis, enquanto dados incompletos ou tendenciosos podem levar a resultados distorcidos e ineficazes. Portanto, a coleta, o processamento e a análise crítica dos dados são etapas cruciais na construção de sistemas inteligentes.

No contexto do ML, os dados são frequentemente referidos como a “matéria prima” que alimenta os modelos. A qualidade e a quantidade dos dados utilizados são fatores determinantes para o desempenho e a eficácia dos algoritmos. Para que um modelo de ML seja capaz de identificar padrões complexos e realizar previsões precisas, é essencial que os dados estejam devidamente coletados, limpos e preparados.

A qualidade dos dados se refere à precisão, completude, consistência e relevância das informações coletadas. Dados de alta qualidade possuem as seguintes características:

- » **Precisão:** Os dados devem refletir a realidade de forma exata, sem erros ou distorções.
- » **Completude:** Um conjunto de dados completo sobre todas as variáveis relevantes para o problema que se deseja resolver, sem lacunas significativas.
- » **Consistência:** Dados consistentes mantêm a coerência entre diferentes fontes e ao longo do tempo, sem contradições.
- » **Relevância:** Apenas os dados que são diretamente aplicáveis ao problema em questão devem ser utilizados, evitando o “ruído” de informações irrelevantes.

Quando a qualidade dos dados é alta, os modelos de ML são capazes de aprender de forma mais eficiente, resultando em previsões e classificações mais precisas.

Além da qualidade, a diversidade dos dados desempenha um papel crucial na robustez dos modelos. Dados diversos são aqueles que capturam uma ampla gama de variabilidades possíveis dentro do conjunto de informações. Isso inclui variações em termos de demografia, condições operacionais, comportamentos e cenários. Um modelo treinado em um conjunto de dados diversificado é mais provável de generalizar bem para novos casos que não foram observados durante o treinamento, reduzindo o risco de sobreajuste. Modelos treinados em dados que representam diversas populações e

situações têm maior probabilidade de serem justos e precisos para todos os grupos, evitando vieses indesejados.

Por outro lado, dados incompletos ou tendenciosos podem comprometer significativamente a performance dos modelos de ML. Dados incompletos, com valores ausentes ou insuficientes para determinadas variáveis, podem levar a falhas na aprendizagem do modelo. Da mesma forma, dados tendenciosos, que não representam adequadamente a realidade ou que refletem vieses implícitos, podem resultar em modelos que perpetuam essas distorções.

Por exemplo, se um conjunto de dados para um modelo de classificação de crédito é composto majoritariamente por exemplos de uma única classe social ou região geográfica, o modelo pode aprender a associar características irrelevantes com o resultado desejado, prejudicando sua capacidade de generalização.

Dada a importância dos dados no ML, o processamento e a análise crítica desses dados são etapas que demandam atenção e rigor. O processamento envolve tarefas como limpeza dos dados, normalização, tratamento de valores ausentes e redução de dimensionalidade, todas elas voltadas a melhorar a qualidade dos dados antes de serem utilizados para treinar um modelo.

Além disso, a análise crítica dos dados requer uma compreensão profunda do domínio em questão para identificar possíveis vieses, *outliers* e inconsistências. Esta análise permite a correção de problemas antes que os dados sejam usados na construção do modelo, garantindo que o modelo seja treinado em dados que verdadeiramente representem o problema que se deseja resolver.



7.2 Dados Estruturados vs. Dados Não Estruturados

Dados estruturados são aqueles organizados em uma estrutura predefinida, como em tabelas com linhas e colunas, facilitando a análise por meio de ferramentas de processamento de dados tradicionais. Já os dados não estruturados abrangem informações que não seguem um padrão de organização, como textos, imagens e vídeos, exigindo técnicas mais avançadas para serem analisados, como PLN e visão computacional. A capacidade de trabalhar com ambos os tipos de dados é essencial para aproveitar ao máximo o potencial da aprendizagem de máquina.

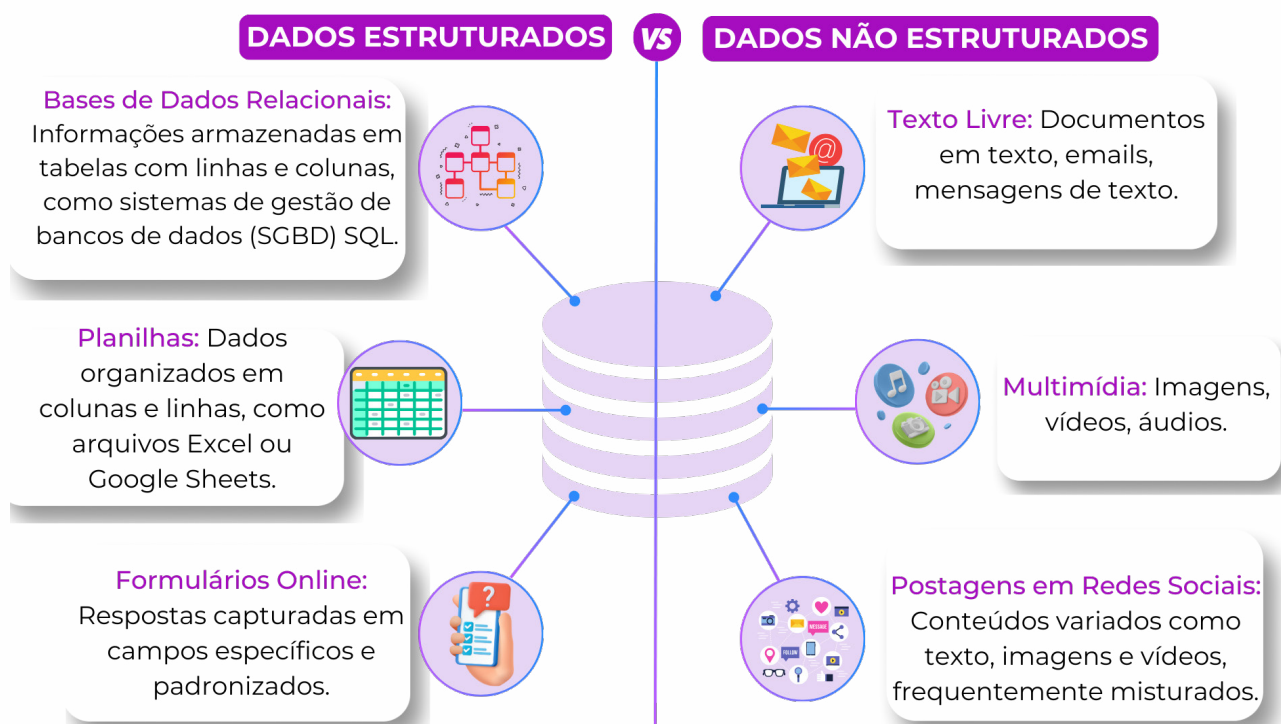
Os **dados estruturados** são organizados em um formato específico, como tabelas, facilitando a sua análise e processamento. Exemplos incluem:

- » **Bases de Dados Relacionais:** Informações armazenadas em tabelas com linhas e colunas, como Sistemas de Gestão de Bancos de Dados (SGBD).
- » **Planilhas:** Dados organizados em colunas e linhas, como arquivos Excel ou Google Sheets.
- » **Formulários Online:** Respostas capturadas em campos específicos e padronizados.

Os dados não estruturados não possuem um formato específico e são mais difíceis de organizar e analisar. Exemplos incluem:

- » **Texto Livre:** Documentos em texto, e-mails, mensagens de texto.
- » **Multimídia:** Imagens, vídeos, áudios.
- » **Postagens em Redes Sociais:** Conteúdos variados como texto, imagens e vídeos, frequentemente misturados.

Figura 31 - Exemplo de dados estruturados e não estruturados



Fonte: Adaptada de [Alam, 2024](#).

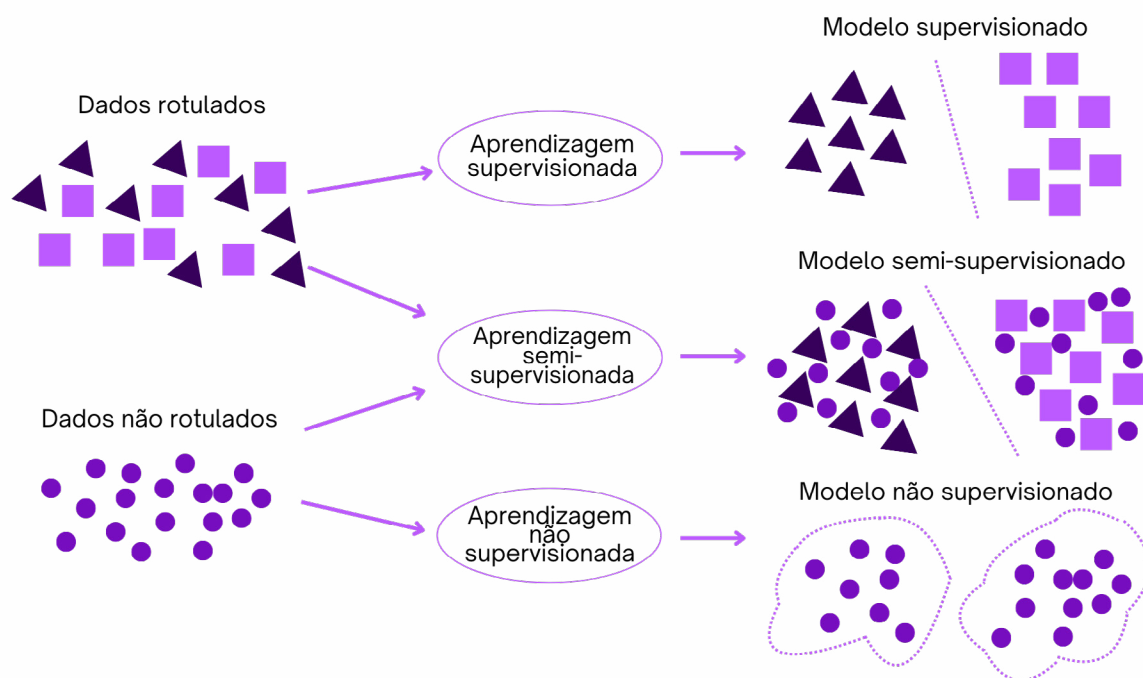
7.3 Conjuntos de Dados Supervisionados, Não Supervisionados e Semi-supervisionados

Nos conjuntos de dados supervisionados, cada exemplo de treinamento inclui uma entrada e uma saída desejada, permitindo ao modelo aprender a relação entre elas. Em contraste, os conjuntos não supervisionados contêm apenas entradas, cabendo ao algoritmo descobrir padrões sem uma resposta direta para guiar o aprendizado. Já os conjuntos semi-supervisionados combinam uma pequena quantidade de dados rotulados com uma grande quantidade de dados não rotulados, aproveitando o melhor de ambas as abordagens para melhorar a precisão dos modelos.

Na figura abaixo (Figura 28) é ilustrado o mesmo conjunto de dados apresentado de duas maneiras: rotulados (em triângulos e quadrados) e não rotulados (em círculos). Os métodos supervisionados necessitam que todos os dados possuam rótulos, como em Modelo Supervisionado da Figura 28. Já os métodos não supervisionados não necessitam de rótulo e tentarão identificar apenas pelas características, como apresentado da

porção Modelo Não Supervisionado. Por último, quando há parte dos dados com rótulo e parte sem rótulos, os métodos semi-supervisionados podem ser a melhor opção, como ilustrado na porção Modelo Semi-supervisionado da figura.

Figura 32 - Exemplo de tipos de dados para diferentes modelos



Fonte: adaptada de [Trambaiolli, Biazoli, Sato \(2022\)](#).

É possível encontrar publicamente uma série de conjuntos de dados, sejam eles supervisionados, semi-supervisionados ou não supervisionados para uso em conjunto com modelos de classificação, regressão, clusterização, etc. Alguns conjuntos de dados se tornaram icônicos no campo da aprendizagem de máquina pela sua relevância e frequente uso em estudos e aplicações. Conjuntos de dados padronizados são fundamentais para o desenvolvimento e a avaliação de algoritmos de aprendizado de máquina, pois permitem comparar o desempenho de diferentes abordagens em condições controladas. Dentre os conjuntos de dados mais amplamente utilizados estão o **Iris**, o **MNIST** e o **CIFAR-10**.

O conjunto **Iris**, por exemplo, é composto por 150 amostras de flores pertencentes à espécie *Iris*, distribuídas igualmente entre três subespécies. Cada amostra é caracterizada por quatro atributos numéricos que descrevem propriedades morfológicas das flores, e é utilizado principalmente em tarefas de classificação para prever a subespécie com base nessas características.

O **MNIST** é um extenso conjunto de 70,000 imagens de dígitos manuscritos, cada uma rotulada com um valor correspondente entre 0 e 9. Este conjunto é amplamente empregado em tarefas de reconhecimento de caracteres e serve como referência para avaliar o desempenho de algoritmos de classificação de imagens.

Por fim, o **CIFAR-10** consiste em 60,000 imagens coloridas de 32x32 pixels, distribuídas igualmente entre 10 categorias de objetos, como aviões, automóveis e animais. Este

conjunto de dados é utilizado para testar e comparar algoritmos de classificação de imagens, especialmente em tarefas que envolvem reconhecimento de objetos em imagens coloridas.

A Tabela abaixo a seguir resume as principais características dos conjuntos de dados mencionados, fornecendo uma visão geral sobre seu tamanho, quantidade de dados rotulados e aplicação principal.

Tabela 4 - Resumo dos conjuntos de dados Iris, MNIST e CIFAR-10

CONJUNTO DE DADOS	ATRIBUTOS	DESCRIÇÃO	LINK
Iris	• Tipo de Dados: Atributos de flores (numéricos) • Tamanho da Base: 150 amostras • Quantidade de Dados Rotulados: 150 • Quantidade de Rótulos: 3	Conjunto de dados contendo medidas de flores Iris, utilizado para classificação de espécies com base em características morfológicas.	Iris
MNIST	• Tipo de Dados: Imagens de dígitos manuscritos • Tamanho da Base: 70,000 imagens • Quantidade de Dados Rotulados: 70,000 • Quantidade de Rótulos: 10	Conjunto de imagens de dígitos manuscritos (0-9) utilizado para tarefas de reconhecimento de caracteres.	MNIST
CIFAR-10	• Tipo de Dados: Imagens coloridas de objetos • Tamanho da Base: 60,000 imagens • Quantidade de Dados Rotulados: 60,000 • Quantidade de Rótulos: 10	Conjunto de imagens coloridas 32x32 pixels, categorizadas em 10 classes, usado para tarefas de classificação de objetos.	CIFAR-10

Fonte: Autoria própria

7.4 Limpeza de Dados (Tratamento de Valores Ausentes, *Outliers*, Erros de Digitação)

A limpeza de dados é uma etapa essencial no processamento e análise de dados, visando a correção e aprimoramento da qualidade dos dados para garantir a eficácia das análises e a confiabilidade dos resultados. Abaixo, são abordados os principais aspectos da importância da limpeza de dados, bem como os conceitos fundamentais envolvidos nesse processo:

- » **Confiabilidade:** Dados limpos e bem preparados são cruciais para garantir a confiabilidade das análises. Quando os dados estão isentos de erros e inconsistências, é possível obter percepções mais precisas e confiáveis, que refletem com maior exatidão a realidade que se pretende analisar. A presença de valores ausentes, duplicados ou errôneos pode comprometer a validade dos resultados e a tomada de decisões.
- » **Qualidade:** A qualidade dos dados é um fator determinante para a obtenção de percepções precisas. Dados bem limpos e estruturados fornecem uma base sólida para a realização de análises avançadas e a construção de modelos preditivos. A qualidade dos dados afeta diretamente a capacidade do modelo de aprender padrões significativos e generalizar para novos dados.
- » **Eficiência:** Dados bem estruturados e limpos facilitam a análise, economizando tempo e recursos. Ao resolver problemas como valores ausentes, duplicações e *outliers* de forma sistemática, a análise pode ser conduzida de maneira mais eficiente, permitindo que os analistas e cientistas de dados concentrem seus esforços na interpretação dos resultados em vez de corrigir problemas de dados.
- » **Valores Ausentes:** Valores ausentes referem-se a dados faltantes ou nulos em um conjunto de dados. Esses valores podem surgir por diversos motivos, como falhas na coleta de dados ou erros de entrada. A abordagem para tratar valores ausentes inclui a imputação, onde valores ausentes são substituídos por estimativas baseadas em outros dados, ou a interpolação, que utiliza os dados existentes para prever os valores faltantes.
- » **Valores Duplicados:** Valores duplicados são registros idênticos dentro do conjunto de dados que podem distorcer a análise e levar a resultados enviesados. A identificação e remoção de duplicatas são importantes para garantir que cada entrada seja considerada apenas uma vez, evitando redundâncias que podem afetar a precisão dos resultados analíticos.
- » **Outliers:** São valores que se desviam significativamente da maioria dos dados. Esses valores extremos podem influenciar desproporcionalmente os resultados das análises e dos modelos estatísticos. A detecção e tratamento de *outliers* são essenciais para evitar que eles distorçam a interpretação dos dados e a construção de modelos preditivos.

- » **Formatação:** A uniformidade na formatação dos dados refere-se à consistência nos formatos e na estrutura dos dados. Dados com formatos inconsistentes, como diferentes padrões de data ou variações na representação de números, podem causar erros e dificultar a análise. A padronização da formatação garante que todos os dados estejam no mesmo formato, facilitando a análise e a integração dos dados.

A limpeza de dados é uma etapa fundamental que assegura a precisão e a confiabilidade das análises. Ao tratar valores ausentes, duplicados e *outliers* e garantir a uniformidade na formatação, é possível obter dados de alta qualidade que são essenciais para a construção de modelos eficazes e a realização de análises precisas.

7.5 Pré-processamento (Normalização, Codificação de Variáveis Categóricas)

O pré-processamento de dados é uma etapa crucial que prepara os dados para o treinamento do modelo. A normalização, como visto na unidade 5, ajusta a escala dos dados para garantir que todas as características contribuam igualmente para o aprendizado. Já a codificação de variáveis categóricas transforma essas variáveis em um formato numérico que os algoritmos podem processar, como a codificação *one-hot* ou a utilização de variáveis *dummy*.

A normalização ajusta a escala dos dados para garantir que todas as características tenham igual importância no processo de aprendizado. As duas técnicas comuns de normalização são: *Min-Max Scaling* e *Z-score Standardization*.

O *Min-Max Scaling*, também conhecido como normalização de intervalo, transforma os dados para um intervalo específico, geralmente $[0, 1]$. A fórmula utilizada é:

$$X_{\text{normalizado}} = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$$

Onde (X) é o valor original da característica, $(X_{\min})$ é o valor mínimo da característica, e $(X_{\max})$ é o valor máximo. Essa técnica é particularmente útil quando é necessário que os dados estejam em um intervalo fixo. No entanto, pode ser sensível a *outliers*, pois os valores extremos podem distorcer o intervalo de normalização.

A padronização pelo Z-score, também conhecida como normalização padrão, transforma os dados para que tenham média zero e desvio padrão um. A fórmula é:

$$X_{\text{padronizado}} = \frac{X - \mu}{\sigma}$$

Onde (X) é o valor original, (μ) é a média dos valores da característica, e (σ) é o desvio padrão. Este método é útil para dados que não possuem uma distribuição uniforme ou quando se deseja que as características tenham a mesma escala, independentemente das unidades originais. É menos sensível a *outliers* em comparação ao *Min-Max Scaling*.

Algoritmos de aprendizado de máquina geralmente exigem que os dados estejam em formato numérico. Portanto, variáveis categóricas, que representam categorias ou

rótulos, precisam ser convertidas em um formato que os algoritmos possam processar. As técnicas comuns para codificação de variáveis categóricas incluem:

- » **One-Hot Encoding** que transforma uma variável categórica em uma série de colunas binárias, onde cada coluna representa uma categoria específica. Se uma variável categórica tem (k) categorias, será criada uma nova coluna para cada categoria, preenchida com 0s e 1s indicando a presença ou ausência daquela categoria. Esta técnica é adequada para variáveis categóricas sem ordem intrínseca e evita a introdução de uma ordem implícita. No entanto, pode aumentar significativamente a dimensionalidade dos dados, especialmente se o número de categorias for grande.
- » **Label Encoding** que atribui um valor numérico a cada categoria de uma variável categórica. Cada categoria é substituída por um número inteiro único. *Label Encoding* é útil para variáveis categóricas ordinais, onde a ordem das categorias é importante. No entanto, pode introduzir uma ordem artificial para variáveis categóricas nominais, o que pode ser inadequado para alguns algoritmos de aprendizado de máquina.

Na figura abaixo é possível compreender melhor como funciona cada uma das técnicas apresentadas. *Label Encoding* simplesmente atribui um número único a cada categoria, como 1 para “Apple”, 2 para “Chicken” e 3 para “Broccoli”. *One-Hot Encoding*, por outro lado, cria uma nova coluna para cada categoria e atribui 1 se a observação pertence àquela categoria e 0 caso contrário.

Figura 33 - Exemplo de técnicas para criação de variáveis categóricas

Label Encoding			One Hot Encodin			
Nome do alimento	Categórico #	Calorias	Maça	Frango	Brócolis	Calorias
Maça	1	95	1	0	0	95
Frango	2	231	0	1	0	231
Brócolis	3	50	0	0	1	50

Fonte: Adaptada de [Medium, 2018](#).

O pré-processamento de dados é fundamental para garantir que as características estejam na mesma escala e em um formato que os algoritmos de aprendizado de máquina possam processar efetivamente. A normalização, por meio do *Min-Max Scaling* e da *Z-score Standardization*, ajusta a escala dos dados para melhorar a convergência e o desempenho do modelo. A codificação de variáveis categóricas, utilizando técnicas como *One-Hot Encoding* e *Label Encoding*, converte categorias em formatos numéricos, preparando os dados para análise e modelagem. Estas práticas são essenciais para a construção de modelos precisos e robustos em aprendizado de máquina.

7.6 Seleção e Engenharia de Características (Feature Selection e Feature Engineering)

A seleção de características envolve escolher as variáveis mais relevantes para o modelo, o que pode aumentar sua eficácia e reduzir a complexidade. Por outro lado, a engenharia de características é a arte de criar novas variáveis a partir das existentes para capturar relações mais complexas entre os dados, que possam influenciar a predição e melhorar a capacidade preditiva do modelo. Ambos os processos são essenciais para otimizar o desempenho do modelo.

A seleção de características visa identificar e escolher o subconjunto mais relevante de atributos que melhor explicam a variabilidade da variável alvo. Ao reduzir a dimensionalidade dos dados, a seleção de características contribui para:

- » **Melhora na performance:** A remoção de características irrelevantes ou redundantes pode reduzir o *overfitting* (sobreajuste) e aumentar a generalização do modelo.
- » **Redução da complexidade:** Modelos com menos características são mais simples de treinar, interpretar e manter.
- » **Diminuição do tempo de treinamento:** Menos características implicam em menor tempo de computação durante o treinamento do modelo.

A engenharia de características é uma etapa que exige um profundo conhecimento do domínio do problema e criatividade. Abaixo algumas técnicas de engenharia de características:

- » **Transformações:** Aplicação de funções matemáticas como logaritmo, exponencial ou normalização para ajustar a distribuição dos dados.
- » **Combinações:** Criação de novas características a partir da combinação de duas ou mais características existentes, como a razão entre duas variáveis.
- » **Interações:** Consideração de interações entre características, como a multiplicação de duas variáveis.
- » **Criação de *features* binárias:** Transformação de variáveis categóricas em variáveis binárias (*one-hot encoding*).
- » **Extração de *features*:** Aplicação de técnicas como PCA para reduzir a dimensionalidade e extrair as características mais relevantes.

7.7 Divisão em Conjuntos de Treino, Validação e Teste

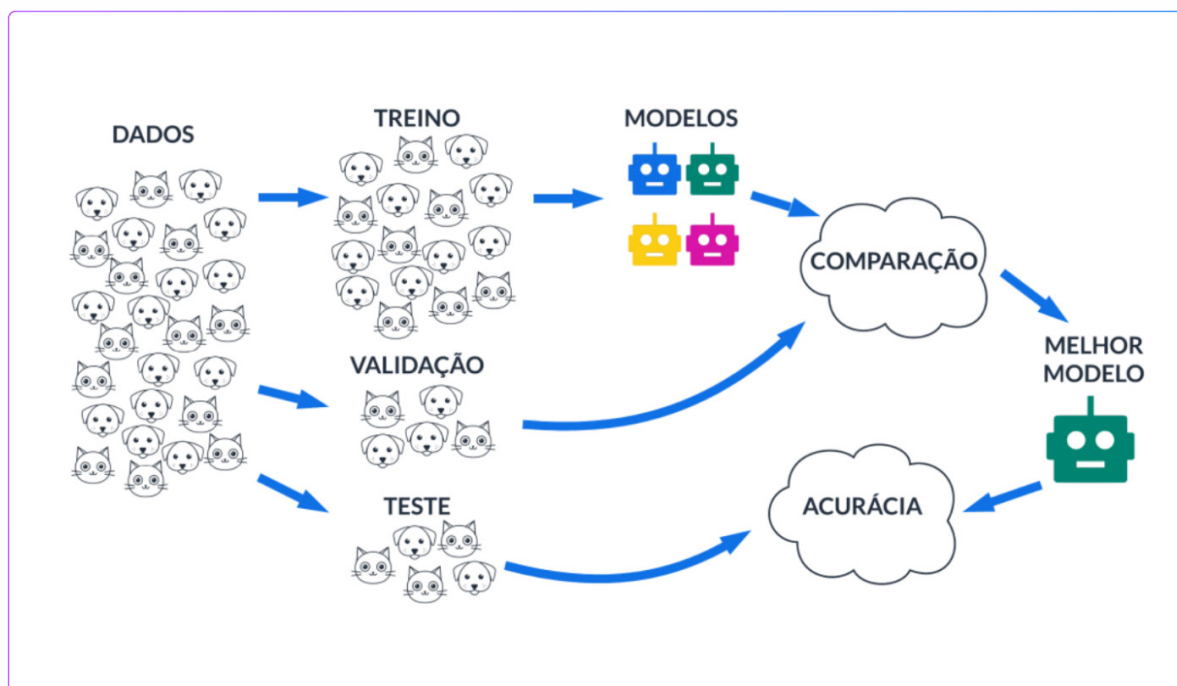
A divisão dos dados em conjuntos de treino, validação e teste é fundamental para a avaliação objetiva dos modelos de ML. O conjunto de treino é utilizado para ajustar os

parâmetros do modelo, o de validação é usado para ajustar hiperparâmetros e evitar *overfitting* (um ajuste excessivo do modelo), e o conjunto de teste fornece uma estimativa final da performance do modelo em dados não vistos, garantindo a generalização do modelo. Normalmente as proporções dos conjuntos são 60% para treino, 20% para validação e 20% para teste.

Para ilustrar, na Figura 32 abaixo temos um conjunto de dados de fotos de cachorros e gatos, cujo objetivo é treinar modelos de classificação para diferenciar fotos de gatos das de cachorros. Uma parte dos dados originais é alocada para o conjunto de treino. O modelo de aprendizado de máquina “aprende” a partir desses dados, ajustando seus parâmetros internos para realizar a tarefa desejada (por exemplo, classificar imagens como gatos ou cachorros). Outra parte dos dados é reservada para o conjunto de validação. Esse conjunto é utilizado para ajustar os hiperparâmetros do modelo, como a taxa de aprendizado ou a complexidade do modelo. A ideia é encontrar a configuração que leva ao melhor desempenho do modelo no conjunto de validação, sem que ele “veja” esses dados durante o treinamento.

Os modelos treinados são avaliados no conjunto de validação. A avaliação permite comparar o desempenho de cada modelo e escolher o que apresenta os melhores resultados. O modelo que obtém o melhor desempenho no conjunto de validação é selecionado como o melhor modelo. Esse melhor modelo é finalmente avaliado em um conjunto de dados totalmente independente, chamado conjunto de teste. Esse conjunto nunca foi utilizado durante o treinamento ou a validação. A avaliação no conjunto de teste fornece uma estimativa mais realista do desempenho do modelo em dados nunca vistos antes. A acurácia é uma métrica que mede o desempenho do modelo no conjunto de teste e indica a porcentagem de exemplos que o modelo classificou corretamente.

Figura 34 - Exemplo divisão do conjunto de dados

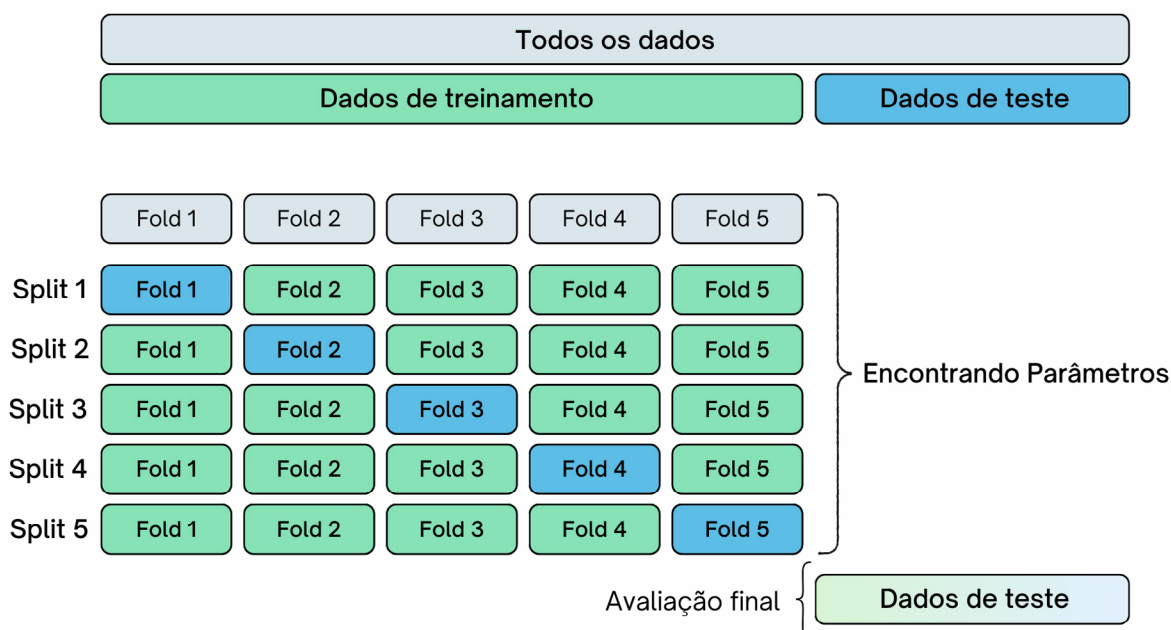


Fonte: adaptada de [Silva \(2023\)](#).

Frequentemente utilizado na fase de divisão dos conjuntos de dados, o Cross-validation (validação cruzada *k-fold*) é uma técnica estatística que permite avaliar a robustez do modelo ao dividir o conjunto de treino em várias partições e realizar treinamentos e validações múltiplas. Isso ajuda a garantir que o modelo não esteja sobreajustado (sofra *overfitting*) a um conjunto de dados específico favorecendo melhor desempenho do modelo para dados desconhecidos. Ao contrário da divisão tradicional em conjuntos de treino, validação e teste, a validação cruzada *k-fold* permite que todos os dados sejam utilizados tanto para treinamento quanto para teste, mitigando o impacto da aleatoriedade na divisão inicial dos dados.

Com base na Figura 31 a seguir, inicialmente o conjunto de dados é dividido em K partes (normalmente 5 ou 10), chamadas de *folds*, de tamanho aproximadamente igual. O processo é repetido K vezes (nesse caso 5), K-1 *folds* são combinados para formar o conjunto de treinamento (*Folds* em verde) e o fold restante é utilizado como conjunto de teste (*fold* em azul). O modelo é treinado em cada iteração (*split*) e avaliado no conjunto de teste correspondente (*fold* azul). As métricas de desempenho (acurácia, precisão, *recall*, etc.) obtidas em cada iteração são calculadas e uma média é tomada. Essa média representa a estimativa do desempenho do modelo.

Figura 35 - Exemplo *k-fold*



Fonte: adaptada de [Scikit-learn \(2024\)](#).

Ao utilizar todos os dados tanto para treinamento quanto para teste, o *k-fold* reduz o viés na estimativa do desempenho do modelo, que poderia ocorrer devido a uma divisão inicial aleatória não representativa. A média das métricas obtidas em todas as iterações fornece uma estimativa mais robusta e confiável do desempenho do modelo. O *k-fold* pode ajudar a identificar modelos que estão sofrendo de *overfitting*, ou seja, modelos que se ajustam muito bem aos dados de treinamento, mas apresentam um desempenho pobre em dados não vistos.

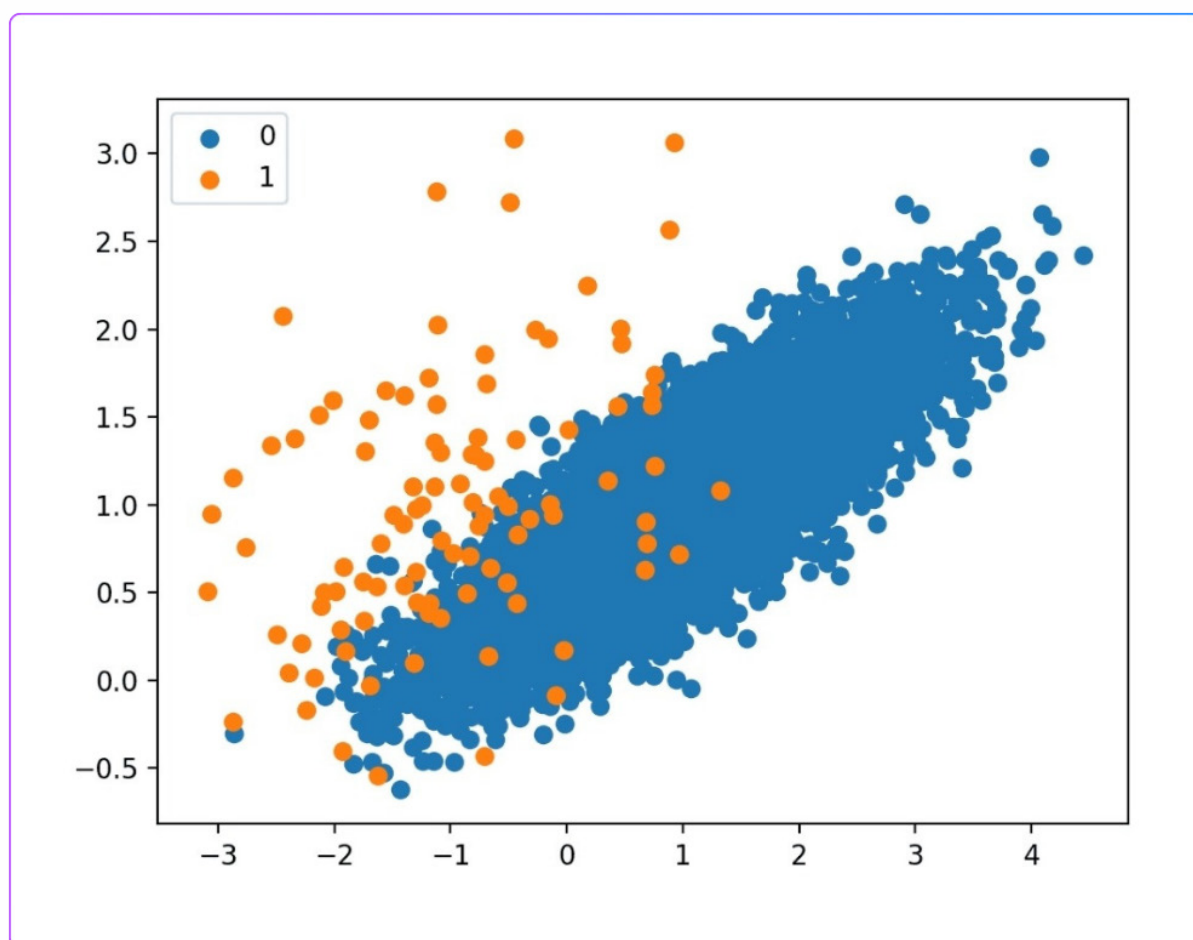
A validação cruzada *k-fold* é uma técnica poderosa e versátil para avaliar o desempenho de modelos de aprendizado de máquina. Ao garantir uma utilização mais eficiente dos dados e fornecer uma estimativa mais robusta do desempenho, o *k-fold* é uma ferramenta essencial no processo de desenvolvimento de modelos.

7.8 Dados Desbalanceados e Técnicas para Balanceamento

Conjuntos de dados desbalanceados ocorrem quando algumas classes têm muito mais exemplos do que outras, o que pode prejudicar o aprendizado do modelo. Técnicas como *oversampling* (aumentar a representatividade das classes minoritárias) e *undersampling* (reduzir a representatividade das classes majoritárias) são usadas para balancear o conjunto de dados e melhorar o desempenho do modelo.

Na figura abaixo há um exemplo de dados desbalanceados, nota-se a evidente diferença na quantidade de dados da classe 0 e 1, em azul e laranja respectivamente.

Figura 36 - Exemplo de dados desbalanceados



Fonte: adaptada de [Medium \(2022\)](#).

Modelos treinados com dados desbalanceados tendem a ser mais precisos na classificação da classe majoritária, ignorando ou classificando incorretamente os exemplos da classe minoritária. Isso ocorre porque o algoritmo busca minimizar o erro global, e a

classe majoritária, por ter mais exemplos, exerce maior influência na função de custo.

A acurácia, uma métrica comumente utilizada para avaliar o desempenho de modelos de classificação, pode ser enganosa em casos de desbalanceamento. Um modelo que simplesmente classifica todos os exemplos como pertencentes à classe majoritária pode obter uma alta acurácia, mesmo que tenha um desempenho muito ruim na classificação da classe minoritária. Métricas como precisão, *recall* e F1-score são mais indicadas para avaliar o desempenho em problemas de classificação com dados desbalanceados. Essas métricas serão detalhadas na unidade de Classificação.

Existem diversas técnicas para lidar com o desbalanceamento de dados, buscando equilibrar a distribuição das classes e melhorar o desempenho dos modelos. As duas principais abordagens são: *Oversampling* e *Undersampling*. A primeira consiste em aumentar a representatividade da classe minoritária, gerando novos exemplos sintéticos. Na segunda busca-se reduzir a representatividade da classe majoritária, removendo aleatoriamente exemplos dessa classe.

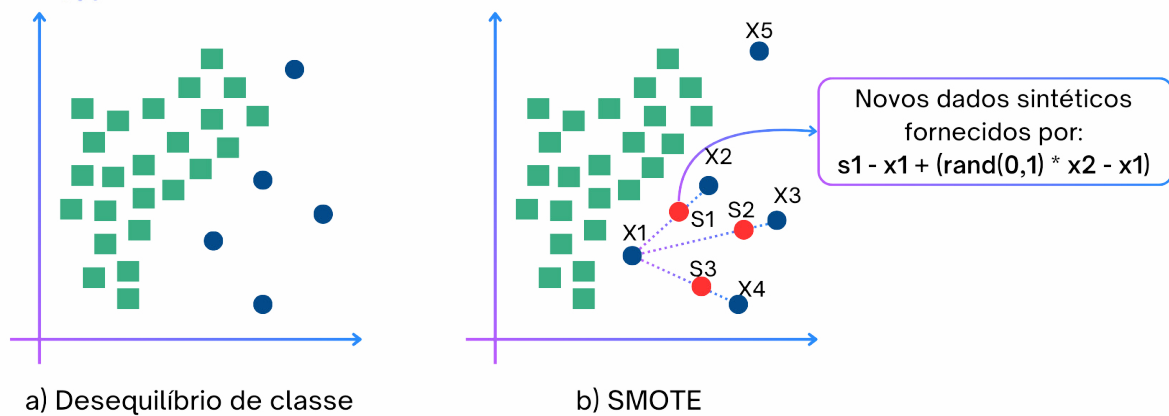
7.8.1 Técnica de *Oversampling*

O **SMOTE (Synthetic Minority Over-sampling Technique)** é uma das técnicas mais populares de *oversampling*. Ele cria novos exemplos sintéticos para a classe minoritária, interpolando os dados existentes em um espaço multidimensional. Essa técnica é eficaz em lidar com desbalanceamentos significativos e pode melhorar significativamente o desempenho do modelo na classificação da classe minoritária. Nela, dados artificiais são criados com base nos K vizinhos mais próximos. Observe a figura abaixo na qual são apresentadas duas classes, quadrados verdes e bolas azuis. O SMOTE segue os seguintes passos:

1. Seleciona a classe minoritária (bolas em azul).
2. Utiliza o KNN na classe minoritária com algum valor de K. Em outras palavras, seleciona os K vizinhos mais próximos de bola azul x_1 . Nesse caso o K é 3, por isso, os vizinhos mais próximos são x_2 , x_3 e x_4 .
3. Desenha-se linhas entre uma amostra da classe minoritária e seus K vizinhos.
4. Escolhe um valor aleatório sobre essa linha. Nesse caso as bolas em vermelho s_1 , s_2 e s_3 são os valores aleatórios sobre essas linhas. Elas são os dados criados artificialmente para o balanceamento da classe de bolas azuis.

O processo se repete até que a quantidade de bolas (vermelhas + azuis) seja igual a quantidade de quadrados verdes.

Figura 37 - Exemplo do funcionamento do *Synthetic Minority Over-sampling Technique* (SMOTE)



Fonte: adaptada de [Yang \(2022\)](#).

7.8.2 Técnica de *Undersampling*

O ***RandomUnderSampler*** é uma técnica simples de *undersampling* que remove aleatoriamente exemplos da classe majoritária até que as classes estejam balanceadas. Embora seja fácil de implementar, pode levar à perda de informações importantes, especialmente se a classe majoritária contiver padrões relevantes.

A escolha da técnica de balanceamento depende de diversos fatores, como:

- » **Grau de desbalanceamento:** Para desbalanceamentos extremos, o *oversampling* pode ser mais eficaz.
- » **Tamanho do conjunto de dados:** Em conjuntos de dados pequenos, o *oversampling* pode levar ao *overfitting* (sobreajuste).
- » **Natureza dos dados:** A complexidade dos dados e a relação entre as *features*

O uso prático se estende a diversas aplicações, porém um exemplo didático é a detecção de fraudes em cartões de crédito, onde a classe “fraude” é significativamente menor do que a classe “não fraude”. Ao aplicar o SMOTE, novos exemplos de fraudes sintéticas seriam gerados, tornando a distribuição das classes mais equilibrada. Isso permitiria que o modelo aprendesse a identificar padrões associados a fraudes com maior precisão.

O tratamento de dados desbalanceados é um desafio comum em aprendizado de máquina. A escolha da técnica de balanceamento adequada depende das características do conjunto de dados e dos objetivos da análise. Ao utilizar técnicas como *oversampling* e *undersampling*, é possível melhorar significativamente o desempenho dos modelos de classificação em problemas com classes desbalanceadas.

7.9 Subajuste (*Underfitting*) e Sobreajuste (*Overfitting*)

Subajuste, ou mais comumente conhecido como *Underfitting*, ocorre quando um modelo é muito simples e não consegue capturar as tendências dos dados de treino, resultando em baixo desempenho tanto no treino quanto no teste. Sobreajuste, ou melhor conhecido como *Overfitting*, por outro lado, acontece quando um modelo se ajusta excessivamente aos dados de treino, memorizando ruídos e peculiaridades, e falha em generalizar para novos dados. Técnicas como validação cruzada, regularização e seleção de modelos são usadas para encontrar um equilíbrio ideal e evitar esses problemas. Ambos representam situações em que o modelo não consegue capturar adequadamente as relações entre as variáveis, resultando em um desempenho insatisfatório.

O *underfitting* ocorre quando um modelo é muito simples para capturar a complexidade dos dados. Ele não consegue aprender as relações subjacentes entre as *features* (características) e o *target* (variável alvo), resultando em um alto erro tanto no conjunto de treinamento quanto no conjunto de teste. Um exemplo é o ajuste de uma linha reta a dados que seguem um padrão não linear, como o ilustrado na figura abaixo e a esquerda. As principais causas são:

- » **Modelo subespecificado:** O modelo escolhido não possui a complexidade necessária para representar os dados. Por exemplo, utilizar uma regressão linear para modelar dados não lineares.
- » **Poucos dados de treinamento:** A quantidade de dados disponíveis é insuficiente para o modelo aprender as relações complexas entre as *features*.
- » **Características irrelevantes:** A inclusão de características que não possuem relação com a variável alvo pode dificultar o aprendizado do modelo.

Existem algumas maneiras de mitigação desses problemas, algumas técnicas são:

- » **Aumentar a complexidade do modelo:** Utilizar modelos com mais parâmetros, como redes neurais com mais camadas ou polinômios de maior grau.
- » **Adicionar mais *features*:** Incluir características relevantes que possam ajudar o modelo a capturar as relações subjacentes.

O *overfitting* ocorre quando um modelo se ajusta excessivamente aos dados de treinamento, memorizando o ruído presente nos dados e perdendo a capacidade de generalizar para novos dados. É como ajustar uma curva muito complexa a um conjunto de pontos, capturando até mesmo as pequenas variações aleatórias, como o ilustrado na figura abaixo e a direita. As principais causas são:

- » **Modelo superparametrizado:** O modelo possui muitos parâmetros, o que permite que ele memorize os dados de treinamento em vez de aprender as relações gerais.

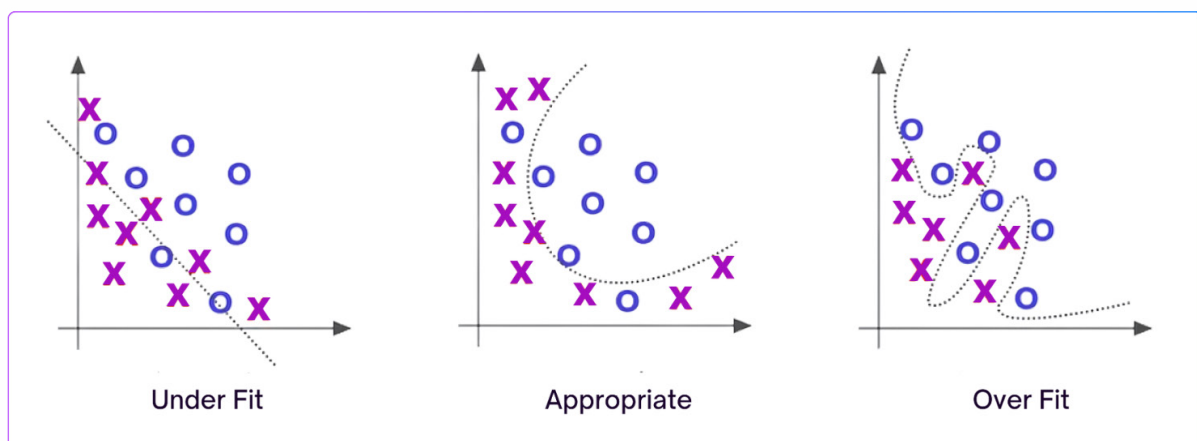
- » **Poucos dados de treinamento em relação à complexidade do modelo:** Quando o modelo é muito complexo em relação à quantidade de dados, ele tende a sobreajustar.
- » **Ruído nos dados:** A presença de ruído nos dados pode levar o modelo a ajustar-se a padrões aleatórios, em vez das relações subjacentes.

Existem algumas maneiras de mitigação desses problemas, algumas técnicas são:

- » **Reduzir a complexidade do modelo:** Utilizar modelos com menos parâmetros, como redes neurais com menos camadas ou polinômios de menor grau.
- » **Aumentar a regularização:** A regularização penaliza modelos complexos, evitando que eles se ajustem demais aos dados de treinamento.
- » **Coleta de mais dados:** Aumentar a quantidade de dados de treinamento pode ajudar a reduzir o *overfitting* (sobreajuste), pois o modelo terá mais exemplos para aprender.
- » **Validação cruzada:** A validação cruzada permite avaliar o desempenho do modelo em diferentes subconjuntos dos dados, ajudando a identificar o *overfitting* (sobreajuste).
- » **Dropout:** Uma técnica utilizada em redes neurais que desativa aleatoriamente neurônios durante o treinamento, forçando o modelo a não depender excessivamente de nenhum neurônio em particular.

O objetivo do aprendizado de máquina é encontrar um modelo que seja capaz de generalizar bem para novos dados, evitando tanto o *underfitting* quanto o *overfitting* (sobreajuste), como o ilustrado na figura abaixo e ao centro. A escolha da técnica adequada para mitigar esses problemas depende das características dos dados e do modelo utilizado.

Figura 38 - Exemplo de ajustes de curvas



Fonte: adaptada de [Medium \(2022\)](#).



7.10 Notebook Colab:

No [link do Colab](#) serão abordados exemplos práticos e exercícios de dados estruturados e não estruturados, tipos conjunto de dados; limpeza dos dados; pré processamento; divisão do conjuntos de dados em treino, validação e teste; dados balanceados e desbalanceados; *underfitting* e *overfitting* (sobreajuste).

Objetivos de Aprendizagem

- » Entender a diferença entre dados estruturados e não estruturados;
- » Entender a diferença entre conjunto de dados supervisionados, não supervisionados e semi-supervisionados;
- » Como importar alguns datasets famosos
- » Como fazer a limpeza dos dados
- » Como fazer o pré-processamento
- » Entender a importância da seleção das caraterísticas
- » Como fazer a divisão dos dados em treino, validação e teste
- » Como fazer a validação cruzada
- » Como tratar dados desbalanceados
- » Entender o sobreajuste ou overfitting

Dados Estruturados e Não Estruturados

Dados Estruturados

Os dados estruturados são organizados em um formato específico, como tabelas, **facilitando a sua análise e processamento.**

Dados Não Estruturados

Os dados não estruturados não possuem um formato específico e **são mais difíceis de organizar e analisar.**

Pacotes Python para Dados Estruturados e Não Estruturados

Dados Estruturados

Como exemplo de pacotes python para uso com dados estruturados podem ser citados:

1. **Pandas:** Pandas é um pacote essencial para manipulação e análise de dados

estruturados. Ele fornece estruturas de dados como DataFrames e Series, que permitem operações eficientes de filtragem, agregação e transformação de dados tabulares.

2. **NumPy:** NumPy é uma biblioteca fundamental para computação numérica em Python, oferecendo suporte a arrays multidimensionais e uma ampla gama de funções matemáticas para operações rápidas e eficientes em grandes conjuntos de dados.
3. **Scikit-learn:** Scikit-learn é uma biblioteca robusta para aprendizado de máquina, que inclui diversas ferramentas para pré-processamento de dados, como normalização, imputação de valores ausentes e codificação de variáveis categóricas, facilitando a preparação de dados estruturados para modelos preditivos.
4. **Dask:** Dask é uma biblioteca que permite a manipulação de grandes conjuntos de dados que não cabem na memória, dividindo-os em blocos e distribuindo as operações em paralelo. Ele estende a funcionalidade do Pandas para trabalhar com dados maiores do que a memória disponível.
5. **SQLAlchemy:** SQLAlchemy é uma ferramenta poderosa para a integração de bancos de dados relacionais em Python, facilitando a execução de consultas SQL e a manipulação de dados estruturados diretamente do banco de dados, mantendo a compatibilidade com diferentes sistemas de gerenciamento de banco de dados (SGBDs).
6. **Polars:** Polars é uma biblioteca de processamento de dados altamente eficiente e de alta performance, projetada para manipulação e análise de grandes conjuntos de dados tabulares. Ele utiliza uma abordagem baseada em colunas e oferece APIs semelhantes ao Pandas, mas com desempenho significativamente mais rápido devido ao seu uso de Rust como backend.
7. **PySpark:** PySpark é a interface Python para o Apache Spark, um framework de computação distribuída. Ele permite a manipulação e análise de grandes volumes de dados de forma paralela e distribuída, sendo ideal para processamento de Big Data em clusters, com suporte para SQL, machine learning e streaming em tempo real..

Pandas:

```
# Importação da biblioteca
import pandas as pd

# Definição dos dados de forma estruturada Colunas e Linhas
d = {'col1': [1, 2], 'col2': [3, 4]}

# Importação do dicionário de dados para dentro do DataFrame do pandas
df = pd.DataFrame(data=d)

df
```

continua

	col1	col2
0	1	3
1	2	4

Dados Não Estruturados

Como exemplo de pacotes python para uso com dados não estruturados podem ser citados:

1. **Scikit-learn:** Scikit-learn é uma biblioteca versátil, já abordada também para uso com dados estruturados. Ela oferece uma ampla gama de algoritmos para aprendizado não supervisionado, como clustering, redução de dimensionalidade e análise de agrupamentos. Ela fornece implementações eficientes de algoritmos como k-means, PCA, e DBSCAN, permitindo a descoberta de padrões em dados não rotulados.
2. **OpenCV:** OpenCV é uma biblioteca poderosa para processamento de imagens e visão computacional, frequentemente utilizada em tarefas não supervisionadas como segmentação de imagem e detecção de bordas. Sua vasta coleção de ferramentas permite a análise e transformação de imagens, além de suportar operações avançadas de processamento de vídeo.
3. **NLTK (Natural Language Toolkit):** NLTK é uma biblioteca essencial para o processamento de linguagem natural (NLP), oferecendo ferramentas para análise de texto não supervisionada, como tokenização, stemming, e análise de frequências. Ela também suporta tarefas como clustering de documentos e análise semântica, facilitando o trabalho com grandes corpora de texto.
4. **HDBSCAN:** HDBSCAN é uma biblioteca especializada em clustering hierárquico baseado em densidade, que pode identificar clusters de diferentes densidades em dados não supervisionados. Ele é particularmente útil para encontrar agrupamentos em conjuntos de dados complexos e de forma não paramétrica, sem necessidade de definir o número de clusters previamente.
5. **Gensim:** Gensim é uma biblioteca focada em modelagem de tópicos e análise de grandes corpora de texto. Ela é conhecida por suas implementações de modelos como LDA (Latent Dirichlet Allocation) e Word2Vec, que permitem a extração de tópicos e representações semânticas de documentos em cenários não supervisionados.

NLTK (Natural Language Toolkit):

```
# Importação das bibliotecas
import nltk
from nltk.tokenize import word_tokenize
nltk.download('punkt')

# Exemplo de texto
```

continua

```
texto = "Olá, como você está hoje?"
```

```
# Tokenizar o texto
```

```
tokens = word_tokenize(texto, language='portuguese')
```

```
print(tokens)
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
```

```
[nltk_data]   Unzipping tokenizers/punkt.zip.
```

```
['Olá', ',', 'como', 'você', 'está', 'hoje', '?']
```

OpenCV:

```
# Importação das bibliotecas
```

```
from skimage import io
```

```
import matplotlib.pyplot as plt
```

```
import cv2
```

```
# Carrega a imagem da internet
```

```
image = io.imread('https://akcit.ufg.br/_next/image?url=%2Flogo_akcit.png&w=1080&q=75')
```

```
# Salva a imagem
```

```
io.imwrite('image.jpg', image)
```

```
# Carrega a imagem salva utilizando o OpenCV
```

```
image = cv2.imread('image.jpg')
```

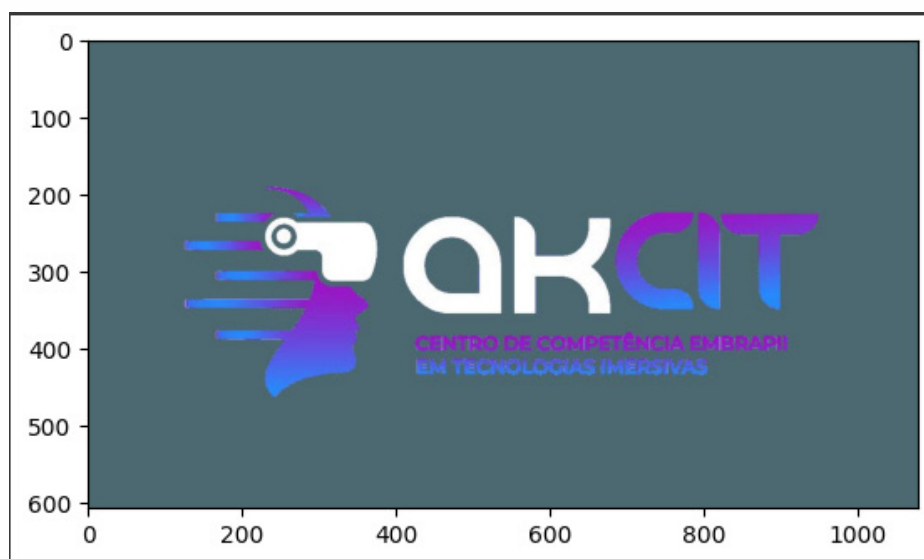
```
# Corrige as cores
```

```
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
```

```
plt.imshow(image)
```

```
plt.show()
```

Figura 39 - Exemplo de carregamento de imagem



Fonte: autoria própria.

Dados Supervisionados

Os dados supervisionados são aqueles em que cada exemplo de treinamento é composto por uma entrada e a saída desejada (rótulo). Exemplos incluem:

- » **MNIST:** Conjunto de dados de dígitos manuscritos com rótulos indicando os dígitos de 0 a 9.
- » **Iris:** Conjunto de dados de flores com características como comprimento da pétala e rótulos indicando a espécie da flor.
- » **CIFAR-10:** Conjunto de imagens divididas em 10 classes, como aviões, carros, gatos, etc.

Exemplo de um conjunto de dados supervisionado

O conjunto de dados Iris é amplamente reconhecido e utilizado em estatística e aprendizado de máquina, especialmente como exemplo introdutório para a aplicação de técnicas de classificação. Este conjunto de dados consiste em 150 amostras de flores pertencentes à espécie Iris, distribuídas igualmente entre três subespécies: **Iris setosa**, **Iris versicolor** e **Iris virginica**.

Cada amostra no conjunto de dados é caracterizada por quatro variáveis numéricas que descrevem atributos morfológicos das flores:

- » **Comprimento da sépala** (em centímetros);
- » **Largura da sépala** (em centímetros);
- » **Comprimento da pétala** (em centímetros);
- » **Largura da pétala** (em centímetros).

Devido à sua simplicidade, equilíbrio e bem-definidas classes, o conjunto de dados Iris é frequentemente empregado para ilustrar algoritmos de classificação e técnicas de análise exploratória de dados.

```
# Importa dados do scikitlearn
from sklearn.datasets import load_iris
data = load_iris(as_frame=True)
```

```
# Imprime a descrição do dataset
print(data.DESCR)
```

```
.. _iris_dataset:

Iris plants dataset
-----
```

continua

****Data Set Characteristics:****

:Number of Instances: 150 (50 in each of three classes)
:Number of Attributes: 4 numeric, predictive attributes and the class
:Attribute Information:
- sepal length in cm
- sepal width in cm
- petal length in cm
- petal width in cm
- class:
- Iris-Setosa
- Iris-Versicolour
- Iris-Virginica

:Summary Statistics:

	Min	Max	Mean	SD	Class Correlation
sepal length:	4.3	7.9	5.84	0.83	0.7826
sepal width:	2.0	4.4	3.05	0.43	-0.4194
petal length:	1.0	6.9	3.76	1.76	0.9490 (high!)
petal width:	0.1	2.5	1.20	0.76	0.9565 (high!)

:Missing Attribute Values: None
:Class Distribution: 33.3% for each of 3 classes.
:Creator: R.A. Fisher
:Donor: Michael Marshall (MARSHALL%PLU@io.arc.nasa.gov)
:Date: July, 1988

The famous Iris database, first used by Sir R.A. Fisher. The dataset is taken from Fisher's paper. Note that it's the same as in R, but not as in the UCI Machine Learning Repository, which has two wrong data points.

This is perhaps the best known database to be found in the pattern recognition literature. Fisher's paper is a classic in the field and is referenced frequently to this day. (See Duda & Hart, for example.) The data set contains 3 classes of 50 instances each, where each class refers to type of iris plant. One class is linearly separable from the other 2; the latter are NOT linearly separable from each other.

|details-start|
References
|details-split|

- Fisher, R.A. "The use of multiple measurements in taxonomic problems" Annual Eugenics, 7, Part II, 179-188 (1936); also in "Contributions to Mathematical Statistics" (John Wiley, NY, 1950).
- Duda, R.O., & Hart, P.E. (1973) Pattern Classification and Scene Analysis. (Q327.D83) John Wiley & Sons. ISBN 0-471-22361-1. See page 218.
- Dasarthy, B.V. (1980) "Nosing Around the Neighborhood: A New System Structure and Classification Rule for Recognition in Partially Exposed Environments". IEEE Transactions on Pattern Analysis and Machine Intelligence, Vol. PAMI-2, No. 1, 67-71.
- Gates, G.W. (1972) "The Reduced Nearest Neighbor Rule". IEEE Transactions

on Information Theory, May 1972, 431-433.
- See also: 1988 MLC Proceedings, 54-64. Cheeseman et al's AUTOCLASS II conceptual clustering system finds 3 classes in the data.
- Many, many more ...

|details-end|

Lista de características

data.feature_names

```
['sepal length (cm)',  
'sepal width (cm)',  
'petal length (cm)',  
'petal width (cm)']
```

Lista de rótulos

data.target_names

```
array(['setosa', 'versicolor', 'virginica'], dtype='<U10')
```

Apresenta os dados como Pandas DataFrame

data.frame

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	2
146	6.3	2.5	5.0	1.9	2
147	6.5	3.0	5.2	2.0	2
148	6.2	3.4	5.4	2.3	2
149	5.9	3.0	5.1	1.8	2

150 rows × 5 columns

Dados Não Supervisionados

Os dados não supervisionados são aqueles que não possuem rótulos, e o objetivo é encontrar padrões ou estrutura nos dados. Exemplos incluem:

- » **Google News Word Vectors:** O conjunto de dados consiste em vetores de palavras (word embeddings) treinados em um grande corpus de texto extraído do Google News. Esses vetores foram gerados utilizando o modelo Word2Vec, uma técnica de aprendizado não supervisionado que captura relações semânticas entre palavras ao mapeá-las em um espaço vetorial contínuo. Este conjunto é amplamente utilizado em tarefas de análise semântica, como busca por similaridade de palavras, análise de relações entre palavras, e construção de representações vetoriais para uso em modelos de aprendizado de máquina. Contém cerca de 3 milhões de vetores de palavras e frases, cada um com 300 dimensões. Sua aplicação inclui: Análise semântica, processamento de linguagem natural (NLP), busca por similaridade de palavras. Este conjunto de dados pode ser baixado publicamente em <https://code.google.com/archive/p/word2vec/>
- » **Análise de Sentimentos de Tweets:** consiste em uma coleção de tweets não rotulados, que podem ser utilizados para tarefas como clustering, redução de dimensionalidade ou identificação de padrões sem supervisão. Embora o conjunto de dados original seja não rotulado, ele é frequentemente utilizado em pesquisas de aprendizado não supervisionado para agrupar tweets com base em sua similaridade ou para extrair temas latentes. Dependendo da coleção possui tamanho variável, mas normalmente composto por milhares ou milhões de tweets. É normalmente aplicado em: Clustering, análise de tópicos, redução de dimensionalidade, detecção de tendências em redes sociais. O conjuntos de dados de tweets podem ser extraídos via API ou encontrados em repositórios públicos como o Kaggle em várias competições ou projetos, como por exemplo <https://www.kaggle.com/code/leandrodoze/sentiment-analysis-in-portuguese>.
- » **Digits Dataset:** é composto por 1,797 imagens de dígitos manuscritos, cada uma com uma resolução de 8x8 pixels, em escala de cinza. Cada imagem representa um dígito de 0 a 9, e é acompanhada por um rótulo que indica o dígito correspondente. Esse conjunto de dados é amplamente utilizado para desenvolver e avaliar algoritmos de aprendizado de máquina em tarefas de reconhecimento de caracteres e classificação de imagens.

Exemplo de um conjunto de dados não supervisionados

Como exemplo, será carregado o conjunto de dados Digits de dígitos manuscritos utilizando a função **load_digits()** do Scikit-learn e, em seguida, visualiza uma amostra de imagens desse conjunto.

Primeiro, o conjunto de dados é carregado e os rótulos são removidos para simular um cenário não supervisionado. Em seguida, utiliza-se a biblioteca **Matplotlib** para criar uma grade de subplots e exibir as primeiras 10 imagens do conjunto de dados, cada uma representando um dígito manuscrito em escala de cinza..

```
# Importar as bibliotecas necessárias
from sklearn import datasets
import matplotlib.pyplot as plt

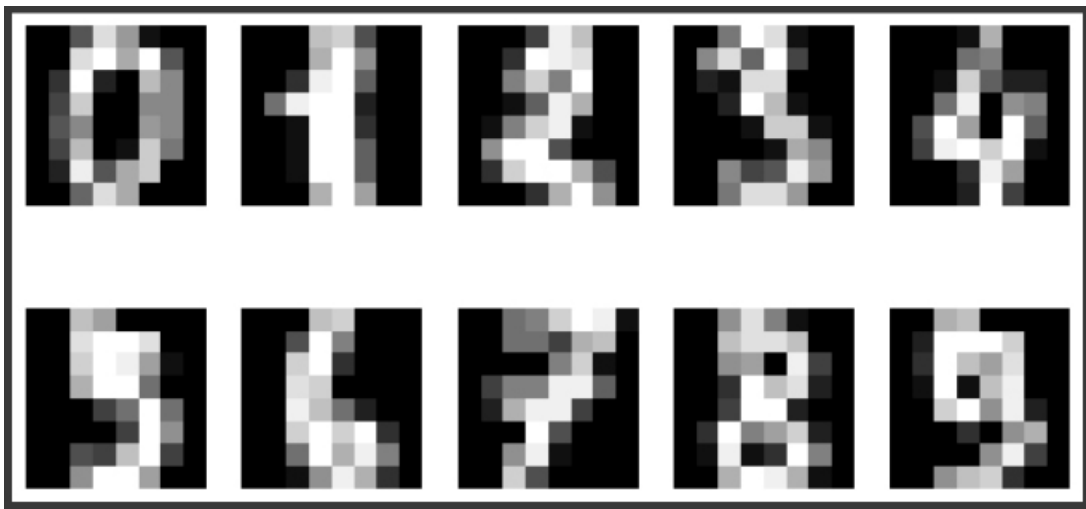
# Carregar o conjunto de dados de Digits
digits = datasets.load_digits()

# Remover os rótulos para simular um cenário não supervisionado
data = digits.data

# Visualizar algumas imagens do conjunto de dados
fig, axes = plt.subplots(2, 5, figsize=(10, 5))
for i, ax in enumerate(axes.ravel()):
    ax.imshow(data[i].reshape(8, 8), cmap='gray')
    ax.axis('off')

plt.show()
```

Figura 40 - Exemplo de um conjunto de dados não supervisionados



Fonte: autoria própria.

Dados Semi-Supervisionados

Os dados semi-supervisionados combinam uma pequena quantidade de dados rotulados com uma grande quantidade de dados não rotulados. Na prática esses datasets podem ser adaptados dos dataset supervisionados, retirando os rótulos de alguns caos e assim podendo ser empregado para teste em algoritmos que visam performance em

conjunto de dados semi-supervisionados.Exemplos incluem:

- » **Cifar-10 com Rótulos Parciais:** Apenas uma fração das imagens possui rótulos, enquanto o restante é utilizado para melhorar a generalização do modelo.
- » **Dados de Textos Anotados:** Pequeno subconjunto de documentos anotados manualmente, com o restante não anotado, usado em tarefas de classificação de texto.
- » **Vídeos com Quadros Anotados:** Alguns quadros de um vídeo são rotulados, enquanto a maioria não possui rótulos, utilizado para treinamento de modelos de reconhecimento de ações.

Exemplo de um conjunto de dados semi-supervisionado

Para criar um dataset semi-supervisionado artificialmente será utilizado o mesmo dataset **Digits** utilizado para o caso não supervisionado.

Inicialmente, carregam-se os dados e os rótulos associados, depois embaralha aleatoriamente os índices dos exemplos para garantir uma divisão não sistemática. Em seguida, apenas 10% dos rótulos originais são mantidos, sendo que o restante dos exemplos como são rotulados com o valor -1. A partir dessas etapas, pode-se observar como transformar um conjunto de dados rotulado em um formato semi-supervisionado.

Em **X** estarão os valores dos 64 pixels (imagem de 8x8 pixels) que representam a imagem de um numeral. Em **y** qual é o número de 0 a 9 que aquela imagem representa.

```
# Importação das bibliotecas
import numpy as np
import pandas as pd
from sklearn import datasets
import matplotlib.pyplot as plt

# Carregar o conjunto de dados de dígitos
digits = datasets.load_digits()
# X possui as características da imagem (valores dos 64 pixels --> imagem 8x8)
X = digits.data
# y possui o valor do rótulo (0 a 9) que representa aquela imagem (conjunto de pixels)
y = digits.target

print("Dados das imagens")
print(X)
print("Dados dos rótulos")
print(y)

print("Tamanho dos Dados das imagens:", X.shape)
print("Tamanho dos rótulos:", y.shape)
```

continua

```
Dados das imagens
[[ 0.  0.  5. ...  0.  0.  0.]
 [ 0.  0.  0. ... 10.  0.  0.]
 [ 0.  0.  0. ... 16.  9.  0.]
 ...
 [ 0.  0.  1. ...  6.  0.  0.]
 [ 0.  0.  2. ... 12.  0.  0.]
 [ 0.  0. 10. ... 12.  1.  0.]]
Dados dos rótulos
[0 1 2 ... 8 9 8]
Tamanho dos Dados das imagens: (1797, 64)
Tamanho dos rótulos: (1797,)
```

```
# Quantidade de exemplos por rótulo
y_ori = pd.DataFrame(data=y, columns=['y_ori'])
y_ori.groupby('y_ori')['y_ori'].count()
```

y_ori	
y_ori	
0	178
1	182
2	177
3	183
4	181
5	182
6	181
7	179
8	174
9	180

dtype: int64

```
# Embaralhar o dataset
random_state = np.random.RandomState(42)
indices = np.arange(len(y))
random_state.shuffle(indices)
```

continua

```
print("Amostra dos valores de indices: ", indices)
print("Comprimento do indice: ", len(indices))
```

```
Amostra dos valores de indices: [1245  220 1518 ...  860 1459 1126]
Comprimento do indice: 1797
```

```
# Manter rótulos somente de 10% dos dados iniciais
n_labeled_points = int(len(y) * 0.1)
print("Quantidade de rótulos que serão mantidos: ", n_labeled_points)
```

```
# Marcar a maioria dos rótulos como -1 (não rotulados)
unlabeled_indices = indices[n_labeled_points:]
y[unlabeled_indices] = -1
```

```
Quantidade de rótulos que serão mantidos: 179
```

```
# Quantidade de exemplos por rótulo
y_semi = pd.DataFrame(data=y, columns=['y_semi'])
y_semi.groupby('y_semi')['y_semi'].count()
```

y_semi	
y_semi	
-1	1618
0	17
1	11
2	16
3	17
4	25
5	22
6	19
7	19
8	8
9	25
dtype: int64	

Limpeza dos dados

Objetivos

- » Compreender a importância da limpeza de dados.
- » Aprender técnicas básicas de limpeza de dados.
- » Implementar essas técnicas utilizando bibliotecas populares do Python.
- » Praticar a aplicação das técnicas de limpeza de dados em conjuntos de dados reais.

1. Introdução

A limpeza de dados é uma etapa crucial no processo de análise de dados. Dados sujos podem levar a resultados enganosos e comprometer a qualidade da análise. Este notebook irá guiá-lo através dos conceitos e técnicas fundamentais para a limpeza de dados utilizando Python.

2. Explicação Teórica

Importância da Limpeza de Dados

- » **Confiabilidade:** Dados limpos aumentam a confiabilidade das análises.
- » **Qualidade:** Dados de alta qualidade são essenciais para obter insights precisos.
- » **Eficiência:** Dados bem estruturados facilitam a análise e economizam tempo.

Conceitos Principais

- » **Valores ausentes:** Dados faltantes ou nulos.
- » **Valores duplicados:** Registros idênticos que podem distorcer a análise.
- » **Outliers:** Valores extremos que podem influenciar desproporcionalmente os resultados.
- » **Formatação:** Uniformidade nos formatos de dados.

3. Exemplos Práticos

Nesse exemplo ilustra-se um processo abrangente de limpeza de dados, aplicando técnicas práticas para lidar com valores ausentes, duplicatas e outliers em um conjunto de dados. Primeiramente, o carrega-se um conjunto de dados do Titanic e visualiza-se a presença de valores ausentes utilizando a biblioteca **missingno**, substituindo os valores ausentes na coluna **age** pela média da coluna. Em seguida, verifica-se e remove regis-

troos duplicados para garantir a integridade dos dados. Para a detecção e tratamento de outliers na coluna **fare**, o utiliza-se um boxplot para visualizar os outliers e aplica o método do intervalo interquartil (IQR) para remover valores extremos. Finalmente, compara-se a média da coluna **fare** antes e depois da remoção dos outliers, demonstrando o impacto desses valores extremos nos resultados analíticos.

Carregando Bibliotecas e Dados

```
import pandas as pd
import numpy as np

# Carregar um conjunto de dados de exemplo
url = 'https://raw.githubusercontent.com/mwaskom/seaborn-data/master/titanic.csv'
df = pd.read_csv(url)
df.head()
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	embark_town	alive	alone
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	NaN	Southampton	no	False
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C	Cherbourg	yes	False
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	NaN	Southampton	yes	True
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C	Southampton	yes	False
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	NaN	Southampton	no	True

Tratamento de Valores Ausentes

```
# Visualizar valores ausentes
df.isnull().sum()
```

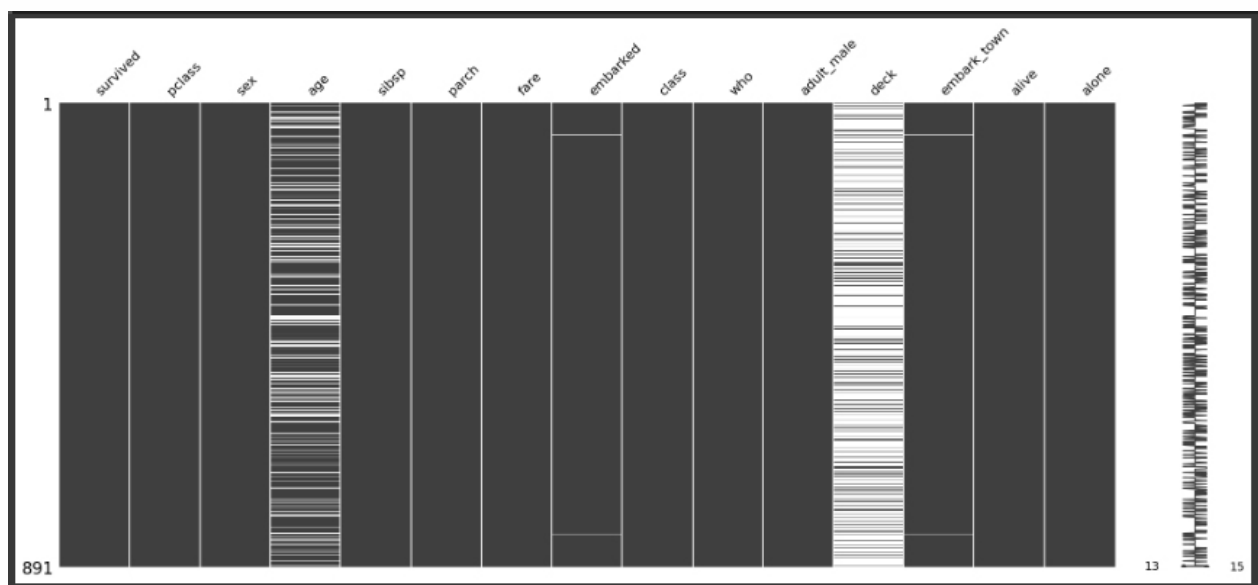
	0
survived	0
pclass	0
sex	0
age	177
sibsp	0
parch	0
fare	0
embarked	2
class	0
who	0
adult_male	0
deck	688
embark_town	2
alive	0
alone	0
dtype:	int64

Outra maneira de verificar os valores ausentes é visualizando graficamente. Abaixo o código apresenta graficamente os valores ausentes em cada coluna do dataset.

Em cor escura, os dados preenchidos, na cor branca os valores ausentes. Em alguns casos reais, pode ser que em determinados períodos não houve o registro de certa característica. Porém para chegar a essa conclusão pode não ser tão direto assim. Nesse caso, a visualização gráfica apontaria um grande bloco branco em meio a grandes blocos escuros, e facilitaria essa conclusão.

```
import missingno as msno
# Visualizar valores ausentes
msno.matrix(df)
plt.show()
```

Figura 41 - Visualização de valores ausentes



Fonte: autoria própria.

```
# Preencher valores ausentes
df['age'].fillna(df['age'].mean(), inplace=True)
```

```
# Visualizar valores ausentes
df.isnull().sum()
```

continua

```

0
survived    0
pclass     0
sex         0
age         0
sibsp       0
parch       0
fare        0
embarked    2
class       0
who         0
adult_male  0
deck       688
embark_town 2
alive       0
alone       0
dtype: int64

```

Remoção de Valores Duplicados

Dimensões do dataset antes da remoção de duplicatas

df.shape

```
(891, 15)
```

Verificar as duplicidades

df.loc[df.duplicated()]

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	embark_town	alive	alone
47	1	3	female	29.699118	0	0	7.7500	Q	Third	woman	False	NaN	Queenstown	yes	True
76	0	3	male	29.699118	0	0	7.8958	S	Third	man	True	NaN	Southampton	no	True
77	0	3	male	29.699118	0	0	8.0500	S	Third	man	True	NaN	Southampton	no	True
87	0	3	male	29.699118	0	0	8.0500	S	Third	man	True	NaN	Southampton	no	True
95	0	3	male	29.699118	0	0	8.0500	S	Third	man	True	NaN	Southampton	no	True
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
870	0	3	male	26.000000	0	0	7.8958	S	Third	man	True	NaN	Southampton	no	True
877	0	3	male	19.000000	0	0	7.8958	S	Third	man	True	NaN	Southampton	no	True
878	0	3	male	29.699118	0	0	7.8958	S	Third	man	True	NaN	Southampton	no	True
884	0	3	male	25.000000	0	0	7.0500	S	Third	man	True	NaN	Southampton	no	True
886	0	2	male	27.000000	0	0	13.0000	S	Second	man	True	NaN	Southampton	no	True

107 rows x 15 columns

continua

```
# Remover duplicatas
df.drop_duplicates(inplace=True)

# Dimensões do dataset antes da remoção de duplicatas
df.shape

(784, 15)
```

Tratamento de Outliers

A técnica apresentada chama-se Tukey's Fence, ela e outras técnicas podem ser encontradas em <https://en.wikipedia.org/wiki/Outlier>

O Tukey's Fence é um método estatístico utilizado para identificar e remover outliers em um conjunto de dados, particularmente em conjunto com a análise de boxplots. Este método baseia-se em extensões do intervalo interquartil (IQR) e utiliza um critério de "cerca" para determinar quais observações são consideradas outliers.

O processo envolve calcular o IQR, que é a diferença entre o terceiro quartil (Q3) e o primeiro quartil (Q1), e então aplicar um fator multiplicativo, geralmente 1.5 ou 3, para estabelecer limites superior e inferior para a identificação de outliers. Dados que estão além desses limites são considerados como outliers.

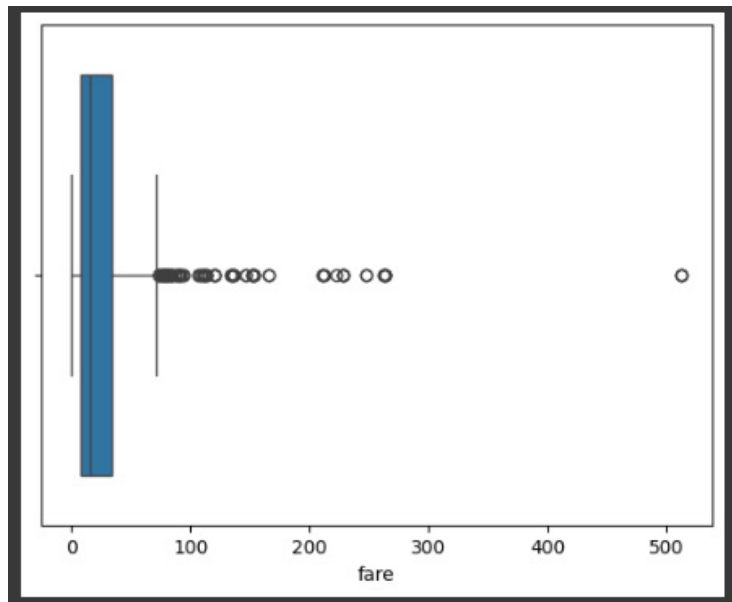
Em um boxplot, os outliers são visualmente representados como pontos fora das "cercas" do boxplot, facilitando a identificação e a decisão sobre quais valores devem ser removidos para melhorar a qualidade dos dados e a precisão das análises subsequentes.

```
# Visualizar outliers
import seaborn as sns
import matplotlib.pyplot as plt

sns.boxplot(x=df['fare'])
plt.show()

# Remover outliers
Q1 = df['fare'].quantile(0.25)
Q3 = df['fare'].quantile(0.75)
IQR = Q3 - Q1
df_sem_outlier = df[~((df['fare'] < (Q1 - 1.5 * IQR)) | (df['fare'] > (Q3 + 1.5 * IQR)))]
```

Figura 42 - Tratamento de Outliers



Fonte: autoria própria.

```
# Comparação do efeito dos outliers
print("Média dos valores de fare originais: ", df['fare'].mean())
print("Média dos valores de fare sem outliers: ", df_sem_outlier['fare'].mean())
```

Média dos valores de fare originais: 34.71173966836735
Média dos valores de fare sem outliers: 19.50749002932551

4. Exercícios e Desafios

Exercício 1: Identificação e Tratamento de Valores Ausentes

Rode o código abaixo que introduz alguns valores ausentes ao conjunto de dados de habitação da Califórnia.

Com base no df gerado, utilize o que aprendeu e identifique as colunas com valores ausente com preenchendo com os valores da mediana.

```
import pandas as pd
import numpy as np
from sklearn.datasets import fetch_california_housing

# Carregar o conjunto de dados California Housing
data = fetch_california_housing(as_frame=True)
df = data.frame

# Introduzir valores ausentes artificialmente para fins de demonstração
df.loc[::10, 'MedInc'] = np.nan # Introduzir valores ausentes na coluna 'MedInc'
df.loc[::15, 'HouseAge'] = np.nan # Introduzir valores ausentes na coluna 'HouseAge'

df.head()
```

continua

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude	MedHouseVal
0	NaN	NaN	6.984127	1.023810	322.0	2.555556	37.88	-122.23	4.526
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	-122.22	3.585
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	-122.24	3.521
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	-122.25	3.413
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	-122.25	3.422

Exercício resolvido

O código realiza a identificação de colunas numéricas que contêm valores ausentes, preenche esses valores com a mediana de cada coluna e, finalmente, verifica se ainda há valores ausentes no DataFrame.

```
# Exercício Resolvido
# Identificar colunas numéricas com valores ausentes
# Seleciona todas as colunas do DataFrame df que possuem dados numéricos.
numeric_columns = df.select_dtypes(include=[np.number]).columns

# Identifica quais dessas colunas numéricas possuem valores ausentes.
# O método isnull() é usado para verificar valores ausentes,
# any() verifica se há pelo menos um valor ausente em cada coluna.
# O resultado é uma série de colunas que contêm valores ausentes,
# armazenadas na variável missing_numeric_columns.
missing_numeric_columns = numeric_columns[df[numeric_columns].isnull().any()]

# Apresenta as colunas com valores ausentes
print("Essas são as colunas com valores ausentes: ", missing_numeric_columns)

# Preencher valores ausentes com a mediana
# Percorre cada coluna identificada como contendo valores ausentes.
for column in missing_numeric_columns:
    # Para cada coluna, o método fillna() é utilizado para preencher os valores
    # ausentes da coluna.
    # O método median() calcula a mediana dos valores não ausentes da coluna.
    # O parâmetro inplace=True garante que as mudanças sejam aplicadas diretamente ao
    # DataFrame df,
    # sem a necessidade de criar uma cópia.
    df[column].fillna(df[column].median(), inplace=True)

# Imprime na tela o número de valores ausentes restantes em cada coluna.
# A função isnull() cria uma matriz booleana indicando a presença de valores ausentes,
# sum() calcula o total de valores ausentes por coluna.
print(df.isnull().sum())
```

continua

```
Essas são as colunas com valores ausentes: Index(['MedInc', 'HouseAge'],
dtype='object')
MedInc      0
HouseAge    0
AveRooms    0
AveBedrms   0
Population  0
AveOccup    0
Latitude    0
Longitude   0
MedHouseVal 0
dtype: int64
```

Pré Processamento

Objetivos

- » Compreender a importância do pré-processamento de dados.
- » Aprender técnicas básicas de normalização e codificação de variáveis categóricas.
- » Implementar essas técnicas utilizando bibliotecas populares do Python.
- » Praticar a aplicação das técnicas de pré-processamento em conjuntos de dados reais.

1. Introdução

O pré-processamento de dados é uma etapa essencial no *pipeline* (sequência de etapas) de *machine learning*. Ele prepara os dados para a modelagem, melhorando a eficiência e a precisão dos modelos. Esta aula abordará duas técnicas fundamentais de pré-processamento: normalização e codificação de variáveis categóricas.

Normalização

A normalização é o processo de ajustar a escala dos dados para que estejam em um intervalo específico, geralmente entre 0 e 1. Isso é importante porque muitos algoritmos de *machine learning* se beneficiam de dados normalizados, resultando em melhor desempenho e convergência mais rápida.

Métodos de Normalização

Saiba mais: https://scikit-learn.org/stable/auto_examples/preprocessing/plot_all_scaling.html#plot-all-scaling-minmax-scaler-section

Aqui serão abordados os 2 principais métodos:

- » **Min-Max Scaling:** Transforma os dados para que seus valores fiquem entre 0 e 1.
- » **Z-score Standardization:** Transforma os dados para que tenham média 0 e desvio padrão 1.

Exemplo de aplicação de Normalização

O exemplo abaixo tem como objetivo principal demonstrar técnicas de pré-processamento, especificamente no contexto de um conjunto de dados de imóveis da Califórnia. As etapas envolvem:

- » **Carregamento dos dados:** O conjunto de dados da Califórnia Housing é carregado e armazenado em um DataFrame Pandas para facilitar a manipulação.
- » **Introdução de variáveis categóricas:** Uma nova coluna categórica é criada aleatoriamente para simular uma situação real onde os dados podem conter variáveis desse tipo.
- » **Normalização dos dados:** As variáveis numéricas contínuas 'MedInc' e 'AveOccup' são normalizadas utilizando os métodos MinMaxScaler e StandardScaler, com o objetivo de escalar os valores para um intervalo específico ou com média zero e desvio padrão unitário, respectivamente.
- » **Visualização dos dados:** Gráficos de dispersão são utilizados para visualizar a distribuição dos dados antes e após a normalização.

```
### Carregando Bibliotecas e Dados

import pandas as pd
import numpy as np
from sklearn.datasets import fetch_california_housing
from sklearn.preprocessing import MinMaxScaler, StandardScaler, OneHotEncoder,
LabelEncoder

# Carregar o conjunto de dados California Housing
data = fetch_california_housing(as_frame=True)
df = data.frame

# Introduzir variáveis categóricas artificiais

# A função np.random.choice do NumPy é utilizada para gerar amostras aleatórias
# a partir de uma determinada população.
# Ela é usada para criar a coluna 'ocean_proximity', atribuindo aleatoriamente
# uma das três categorias possíveis ('<1H OCEAN', 'INLAND', 'NEAR OCEAN') a cada
# observação do DataFrame.
# em size: O número de amostras a serem geradas (igual ao número de linhas do
DataFrame).
```

continua

```
# Essa etapa será necessária quando do tratamento de variáveis categóricas.
df['ocean_proximity'] = np.random.choice(['<1H OCEAN', 'INLAND', 'NEAR OCEAN'],
size=len(df))

df.head()
```

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude	MedHouseVal	ocean_proximity
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	-122.23	4.526	NEAR OCEAN
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	-122.22	3.585	NEAR OCEAN
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	-122.24	3.521	INLAND
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	-122.25	3.413	<1H OCEAN
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	-122.25	3.422	<1H OCEAN

Min-Max Scaling

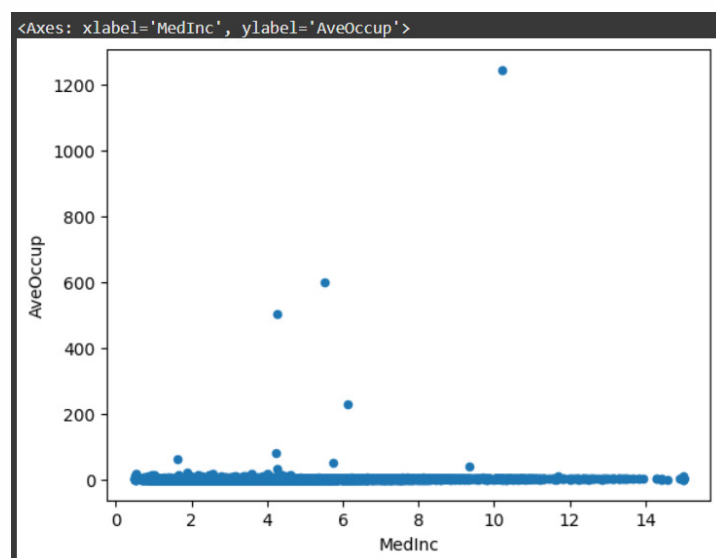
- » **MedInc:** Representa a renda média dos blocos censitários. Essa variável numérica contínua indica o nível de renda da região onde o imóvel está localizado.
- » **AveOccup:** Representa a ocupação média por unidade habitacional. Essa variável numérica contínua fornece informações sobre a densidade populacional nos imóveis.

Utilizaremos essas colunas para testar as técnicas de normalização.

Para melhor compreender, será visto a distribuição desses dados.

```
# Distribuição dos dados antes da normalização
# Atente-se aos valores do eixo x, variando aproximadamente entre 0 a 14
# Atente-se aos valores do eixo y, variando aproximadamente entre 0 e 1200
df.plot.scatter('MedInc', 'AveOccup')
```

Figura 43 - Distribuição de dados antes da normalização - MinMax Scaling



Fonte: autoria própria.

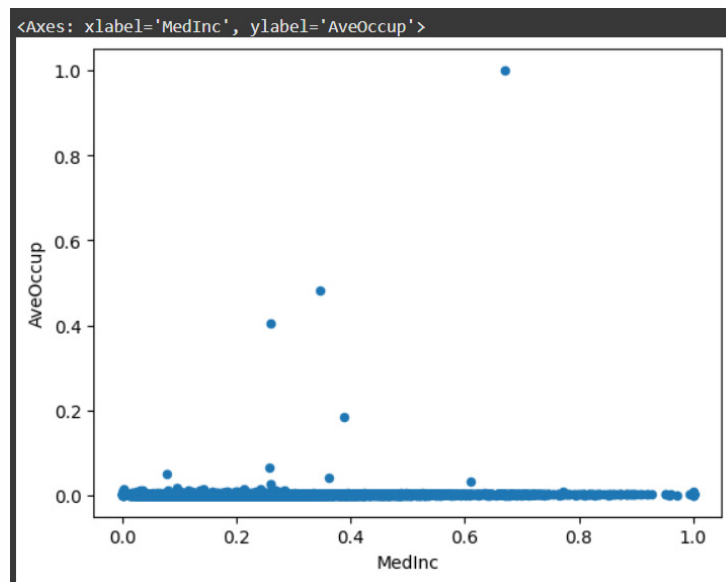
O **min-max scaling** escala os valores de cada característica para o intervalo [0,1]. É útil quando se deseja que todas as características tenham a mesma escala.

```
# Cria um objeto da classe MinMaxScaler.  
# Esse objeto armazenará os parâmetros de transformação que serão aprendidos dos dados.  
scaler = MinMaxScaler()  
  
# Combina duas etapas em uma:  
# fit: Calcula os valores mínimo e máximo das colunas MedInc e AveOccup para determinar a escala.  
# transform: Aplica a transformação aos dados, ajustando cada valor para o intervalo [0, 1]  
# Os valores transformados substituem os valores originais nas colunas especificadas.  
df[['MedInc', 'AveOccup']] = scaler.fit_transform(df[['MedInc', 'AveOccup']])  
  
# Exibe as primeiras linhas do DataFrame após a transformação, mostrando os novos valores normalizados.  
df[['MedInc', 'AveOccup']].head()
```

	MedInc	AveOccup
0	0.539668	0.001499
1	0.538027	0.001141
2	0.466028	0.001698
3	0.354699	0.001493
4	0.230776	0.001198

```
# Distribuição dos dados após a normalização  
# Compare esse gráfico com o primeiro, note como os valores dos eixos x e y  
# agora estão entre 0 e 1  
df.plot.scatter('MedInc', 'AveOccup')
```

Figura 44 - Distribuição de dados após normalização - MinMaxScaling



Fonte: autoria própria.

Z-score Standardization

De forma análoga ao que foi realizado com o **Min-max Scaling**, agora será aplicado outra forma de normalização, o **Z-score Standardization**.

StandardScaler: Centraliza os dados (média = 0) e escala para ter variância unitária.

```
# Cria um objeto da classe StandardScaler.
# Esse objeto armazenará os parâmetros de transformação que serão aprendidos dos
dados.
scaler = StandardScaler()

# Combina duas etapas em uma:
# fit: Calcula os valores mínimo e máximo das colunas MedInc e AveOccup para determinar
a escala.
# transform: Aplica a transformação aos dados, ajustando cada valor para que ao final
# o conjunto possua a média 0 e variância unitária.
# Os valores transformados substituem os valores originais nas colunas especificadas.
df[['MedInc', 'AveOccup']] = scaler.fit_transform(df[['MedInc', 'AveOccup']])

# Exibe as primeiras linhas do DataFrame após a transformação, mostrando os novos
valores normalizados.
df[['MedInc', 'AveOccup']].head()
```

	MedInc	AveOccup
0	2.344766	-0.049597
1	2.332238	-0.092512
2	1.782699	-0.025843
3	0.932968	-0.050329
4	-0.012881	-0.085616

continua

```
# Imprime os valores da média
# Nota-se que os valores obtidos são muito proximo de 0
print("Média dos valores após a normalização: ")
print(df[['MedInc', 'AveOccup']].mean())
```

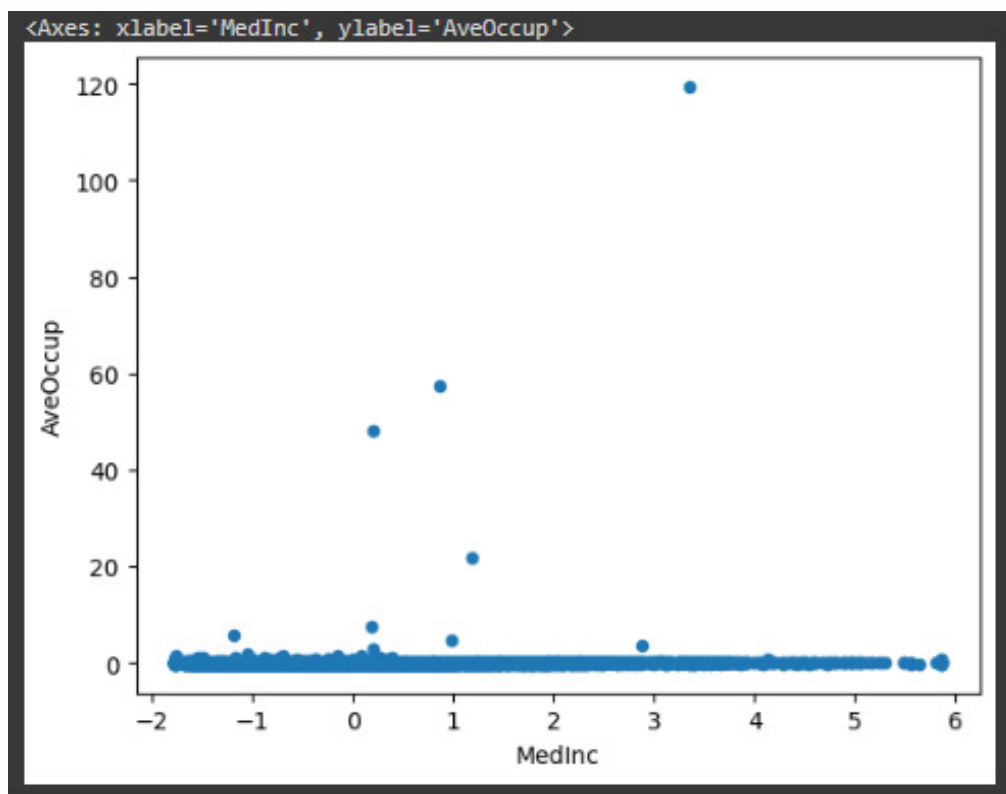
```
Média dos valores após a normalização:
MedInc      -1.652425e-17
AveOccup    -1.377021e-18
dtype: float64
```

```
# Imprime os valores do desvio padrão
# Nota-se que os valores obtidos são muito proximo de 1
print("Desvio padrão dos valores após a normalização: ")
print(df[['MedInc', 'AveOccup']].std())
```

```
Desvio padrão dos valores após a normalização:
MedInc       1.000024
AveOccup     1.000024
dtype: float64
```

```
# Distribuição dos dados antes da normalização
# Compare esse gráfico com o primeiro e o do min max scaling,
# Note como os valores dos eixos x e y não estão entre 0 e 1
# nem estão nos valores originais, porém, como visto anteriormente
# mantém média 0 e desvio padrão 1
df.plot.scatter('MedInc', 'AveOccup')
```

Figura 45 - Distribuição de dados após normalização - ZCore



Fonte: autoria própria.

Codificação de Variáveis Categóricas

Variáveis categóricas são aquelas que contêm valores discretos e não numéricos. Para usar essas variáveis em algoritmos de *machine learning*, precisamos convertê-las em uma forma numérica.

Métodos de Codificação

- » **One-Hot Encoding:** Cria colunas binárias para cada categoria.
- » **Label Encoding:** Atribui um número inteiro único a cada categoria.

One-Hot Encoding

Cria colunas binárias para cada categoria.

Serão utilizados os mesmos dados da etapa de normalização, por isso é importante não pular as etapas anteriores.

```
# Visualização dos dados  
df.head()
```

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude	MedHouseVal	ocean_proximity
0	2.344766	41.0	6.984127	1.023810	322.0	-0.049597	37.88	-122.23	4.526	NEAR OCEAN
1	2.332238	21.0	6.238137	0.971880	2401.0	-0.092512	37.86	-122.22	3.585	NEAR OCEAN
2	1.782699	52.0	8.288136	1.073446	496.0	-0.025843	37.85	-122.24	3.521	INLAND
3	0.932968	52.0	5.817352	1.073059	558.0	-0.050329	37.85	-122.25	3.413	<1H OCEAN
4	-0.012881	52.0	6.281853	1.081081	565.0	-0.085616	37.85	-122.25	3.422	<1H OCEAN

Como abordado no ebook, muitos os algoritmos necessitam que colunas categóricas possuam números ao invés de textos.

No trecho abaixo do código o objetivo é transformar os rótulos a coluna `ocean_proximity` em números. Relembrando, esses rótulos foram criados artificialmente no começo da etapa de pré processamento com rótulos do formato **string** podendo assumir valores: `<1H OCEAN`, `INLAND` ou `NEAR OCEAN`. No entanto, em alguns casos reais esses rótulos já podem vir nativamente no dataset.

Abaixo será visto:

- » Criação de um one-hot encoder.
- » Aplicação do one-hot encoding à coluna `ocean_proximity`.
- » A agregação do DataFrame codificado ao original.
- » Remoção da coluna original codificada.
- » Exibição das primeiras linhas do DataFrame resultante.

O resultado final é um DataFrame onde a coluna categórica `ocean_proximity` foi substituída por novas colunas binárias, representando cada categoria possível.

```
# Cria um codificador one-hot
encoder = OneHotEncoder(sparse_output=False)

# Aplica o one-hot encoding à coluna 'ocean_proximity'
# - fit_transform: Ajusta o encoder aos dados e realiza a transformação de uma só vez
encoded_features = encoder.fit_transform(df[['ocean_proximity']])

# Cria um novo DataFrame com as features codificadas
# - get_feature_names_out: Obtém os nomes das novas colunas geradas

encoded_df = pd.DataFrame(encoded_features, columns=encoder.get_feature_names_out(['ocean_proximity']))

# Combina o DataFrame original com o DataFrame codificado
# - drop: Remove a coluna original 'ocean_proximity'
df_onehot = df.join(encoded_df).drop(columns=['ocean_proximity'])

# Visualiza as primeiras linhas do DataFrame com as features codificadas
# Note as últimas 3 colunas e os valores que elas assumem (0 ou 1)
df_onehot.head()
```

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude	MedHouseVal	ocean_proximity<1H OCEAN	ocean_proximity_INLAND	ocean_proximity_NEAR OCEAN
0	2.344766	41.0	6.984127	1.023810	322.0	-0.049597	37.88	-122.23	4.526	0.0	0.0	1.0
1	2.332238	21.0	6.238137	0.971880	2401.0	-0.092512	37.86	-122.22	3.585	0.0	0.0	1.0
2	1.782699	52.0	8.288136	1.073446	496.0	-0.025843	37.85	-122.24	3.521	0.0	1.0	0.0
3	0.932968	52.0	5.817352	1.073059	558.0	-0.050329	37.85	-122.25	3.413	1.0	0.0	0.0
4	-0.012881	52.0	6.281853	1.081081	565.0	-0.085616	37.85	-122.25	3.422	1.0	0.0	0.0

Label Encoding

De forma análoga ao visto com o **OneHotEncoder** aqui é utilizado o **LabelEncoder**, que atribui um número inteiro único a cada categoria.

Cada categoria da coluna `ocean_proximity` receberá um número correspondente. Esse número substituirá o rótulo da coluna `ocean_proximity`, retirando permitindo que os algoritmos de machine learning possam funcionar adequadamente.

```
# Cria um codificador LabelEncoder
encoder = LabelEncoder()

# Aplica o LabelEncoder à coluna 'ocean_proximity', substituindo os valores existentes
# - fit_transform: Ajusta o encoder aos dados e realiza a transformação de uma só vez
df['ocean_proximity'] = encoder.fit_transform(df['ocean_proximity'])

# Visualiza as primeiras linhas do DataFrame com as features codificadas
# Note as última colunas e os valores que elas assumem (0 a 2)
df.head()
```

continua